

中国研究生创新实践系列大赛

“华为杯”第二十二届中国研究生

数学建模竞赛

题 目： 基于迁移学习的高速列车轴承故障诊断研究

摘 要：

高速列车作为我国综合运输体系中的骨干，具有效率高、体验好、安全环保等方面优点。滚动轴承作为高速列车运行的关键部件，对列车的平稳运行起到重要作用。一旦轴承发生故障，轻则降低列车的运行效率，重则导致列车发生脱轨等严重事故。因此对轴承故障的诊断具有重要的现实价值。本文针对高速列车轴承故障诊断中实际运营环境下数据稀缺与工况复杂的问题，提出了一套完整的研究方案。该方案包括数据筛选与预处理、多维特征提取、故障诊断模型构建及迁移学习模型的设计，并结合可解释性分析，为高速列车轴承故障诊断的实际应用提供了坚实的理论基础和技术支持。

针对问题一，基于目标域特征构建样本均衡且具有代表性的源域数据集并进行数据预处理。从时域、频域、时频域和包络谱分析四个维度选择 39 个备用特征，之后使用皮尔逊和斯皮尔曼相关性分析、主成分分析 PCA、互信息和卡方检验筛选出 14 个重要特征（见表 4.8）并对其的进行可视化分析。

针对问题二，在五折交叉验证下，使用支持向量机 SVM、随机森林 RF 和多层感知机 MLP 等机器学习算法对问题一中三个特征筛选方法得到的 14 个重要特征进行故障诊断，同时采用 VGG、CNN 等深度学习算法对经过小波变换得到的时频域二维特征进行诊断分析，从多个维度对模型性能进行评估（见表 5.2），其中 MLP 在所有指标上都达到了超过 99.6% 的最优性能，尤其是 ACC 高达 99.66%，验证了本文所采用模型的有效性。

针对问题三，基于卷积神经网络设计决策边界对齐策略，通过调整源域和目标域的特征分布，减少由于分布差异导致的性能下降。根据本文提出的基于边界决策优化的迁移学习模型对所给出的目标域样本进行故障诊断，识别出了 4 个外圈故障、4 个内圈故障、5 个滚动体故障和 3 个正常工作状态（具体见表 6.2）。

针对问题四，在轴承诊断任务中，引入分布距离度量、最大均值差异（MMD）和类间相关度量，能够显著提升迁移学习模型的性能。通过减小源域与目标域数据分布之间的差异，增强模型的泛化能力，从而提高故障诊断的准确性和鲁棒性。这些指标的引入为迁移学习在实际轴承诊断中的有效性提供了有力的理论支持。

关键词： 轴承故障诊断 数据不平衡 特征提取 深度学习 迁移学习 可解释性

目录

1	研究背景及问题重述	5
1.1	研究背景	5
1.2	问题重述	5
1.3	总体解题思路	6
2	模型假设与符号说明	7
2.1	模型假设	7
2.2	定义与符号说明	7
3	数据集介绍与可视化	8
3.1	数据集介绍	8
3.2	数据集可视化	8
3.2.1	源域数据集	9
3.2.2	目标域数据集	10
4	问题一求解及分析	12
4.1	问题分析	12
4.2	源域样本筛选与数据集构建	13
4.2.1	数据集建立	13
4.2.2	采样频率与故障尺寸选择	14
4.2.3	传感器位置选择	14
4.2.4	外圈故障采样点位选择	15
4.3	数据预处理	17
4.3.1	采样频率设置	17
4.3.2	数据扩充与窗口划分	18
4.4	特征提取	19
4.4.1	时域特征提取	19
4.4.2	频域特征提取	21
4.4.3	时频域特征提取	22
4.4.4	包络谱分析特征提取	22
4.5	特征选择与分析	23
4.5.1	描述统计性分析	23
4.5.2	基于皮尔逊与斯皮尔曼的相关性分析	23

4.5.3	PCA 主成分分析	27
4.5.4	基于随机森林算法的特征重要性分析	28
4.5.5	基于互信息与卡方检验的特征筛选	31
4.6	本章小结	32
5	问题二求解及分析	34
5.1	问题分析	34
5.2	源域特征输入	35
5.2.1	提取后的特征	35
5.2.2	二维图像表示与处理	35
5.3	模型建立	37
5.3.1	支持向量机 (Support Vector Machine, SVM)	37
5.3.2	多层感知机 (Multilayer Perceptron, MLP)	38
5.3.3	VGG (Visual Geometry Group) 模型	39
5.4	实验设置与评价指标	40
5.5	实验结果分析与可视化	41
5.6	本章小结	43
6	问题三求解及分析	45
6.1	问题分析	45
6.2	域适应 (Domain Adaptation, DA)	46
6.3	基于决策边界优化的迁移学习模型	46
6.4	结果展示	49
6.5	本章小结	50
7	问题四求解及分析	54
7.1	问题分析	54
7.2	基于迁移过程的可解释性分析	54
7.3	本章小结	56
8	模型评价与展望	57
8.1	模型优点	57
8.2	模型缺点	57
8.3	未来展望	57

参考文献	58
附录 A Python 源程序	59
A.1 第 1 问程序	59
A.2 第 2 问程序	65
A.3 第 3 问程序	78
A.4 第 4 问程序	93

1 研究背景及问题重述

1.1 研究背景

高速铁路作为综合交通运输体系的重要组成部分，具有运输能力大、快速正点、全天候和环境友好的优势 [1]，在全球范围内得到了广泛的应用和推广，人们的出行方式和区域经济发展格局得到了深刻改变。尤其在中国，其高速铁路不仅承担着大量旅客的运输任务，同时作为经济大动脉的骨干，在促进了区域经济一体化方面起到了至关重要的作用。中国高速列车当前已经发展到世界领先水平，高速列车由于其在交通效率、出行体验和安全环保等方面优点，已经成为了我国综合运输体系中的骨干 [2]。滚动轴承具有摩擦小、效率高优势，作为高速列车运行的关键部件，对列车的平稳运行起到重要作用 [3]。然而，由于轴承长期工作在恶劣环境，持续性损伤诱因，极有可能导致轴承发生故障 [4]，轻则降低列车运行效率，甚而导致列车发生脱轨等严重事故 [5]。因此，对高铁轴承运行状态的**诊断和故障预测**，对保障列车安全平稳运行具有重要的**理论研究意义和现实价值**。

近年来，轴承故障检测方法如雨后春笋般层出不穷，如将信号故障分析与时频分析相结合、故障分析与浅层**机器学习**相结合、直接使用**深度学习**对振动信号的特征进行提取 [6]，前两种方法虽对特定方面有适用性高的优点，但在复杂工况下的诊断效果欠佳。深度学习与故障分析结合虽能满足对复杂工况下的故障诊断，但由于实际工作环境中，滚动轴承的振动数据往往**获取困难，样本少且分布不均衡，且易受噪音等干扰**，使得深度学习对故障特征的提取较为困难。

与实际运行数据难提取不同，在实验室台架环境下的到的振动数据有**完备的故障标注和丰富的状态类别**的特点，同时其故障原理也和高铁轴承相似。**迁移学习**应运而生，利用源任务与目标任务之间的相似性，迁移学习运用已经学习到的知识作为先验知识去学习新知识 [7]，这一过程不仅能够加速目标任务的学习速度，还能提高网络的整体性能 [8]。在此背景下，通过分析提取源域（轴承实验台的数据）的故障特征，通过迁移学习将其特征信息迁移到目标域（高铁轴承的数据），从而实现对**实际轴承故障的诊断和预测**。

1.2 问题重述

基于上述研究背景，本文需要完成以下问题：

- **问题一**：从源域中选出部分数据，根据三种**轴承故障的原理**，对源域数据集进行**特征分析**，并提取出整体数据集的特征，在提取特征时需要考虑所提取特征在**目标域是否适用**，以更好的完成迁移任务。
- **问题二**：根据问题一从源域中提取的**轴承故障特征**，将源域数据集划分为**训练集和测试集**，建立数学模型，对源域数据进行**轴承故障诊断**，然后对诊断结果进行分析和评价。
- **问题三**：根据问题二建立的轴承故障诊断模型，分析目标域和源域的相同和不同的特

征，使用**迁移学习**的方法，建立数学模型，完成对目标域中**未知标签数据**的故障诊断任务，展示评定结果，同时展示迁移结果并对其进行分析。

- **问题四**：根据问题三中建立的目标域轴承诊断模型，根据迁移学习的方法和过程，考虑轴承故障的特点和原理，说明迁移诊断的**事前/迁移过程/事后的可解释性**。

1.3 总体解题思路

本文的总体解题思路如图1.1所示。

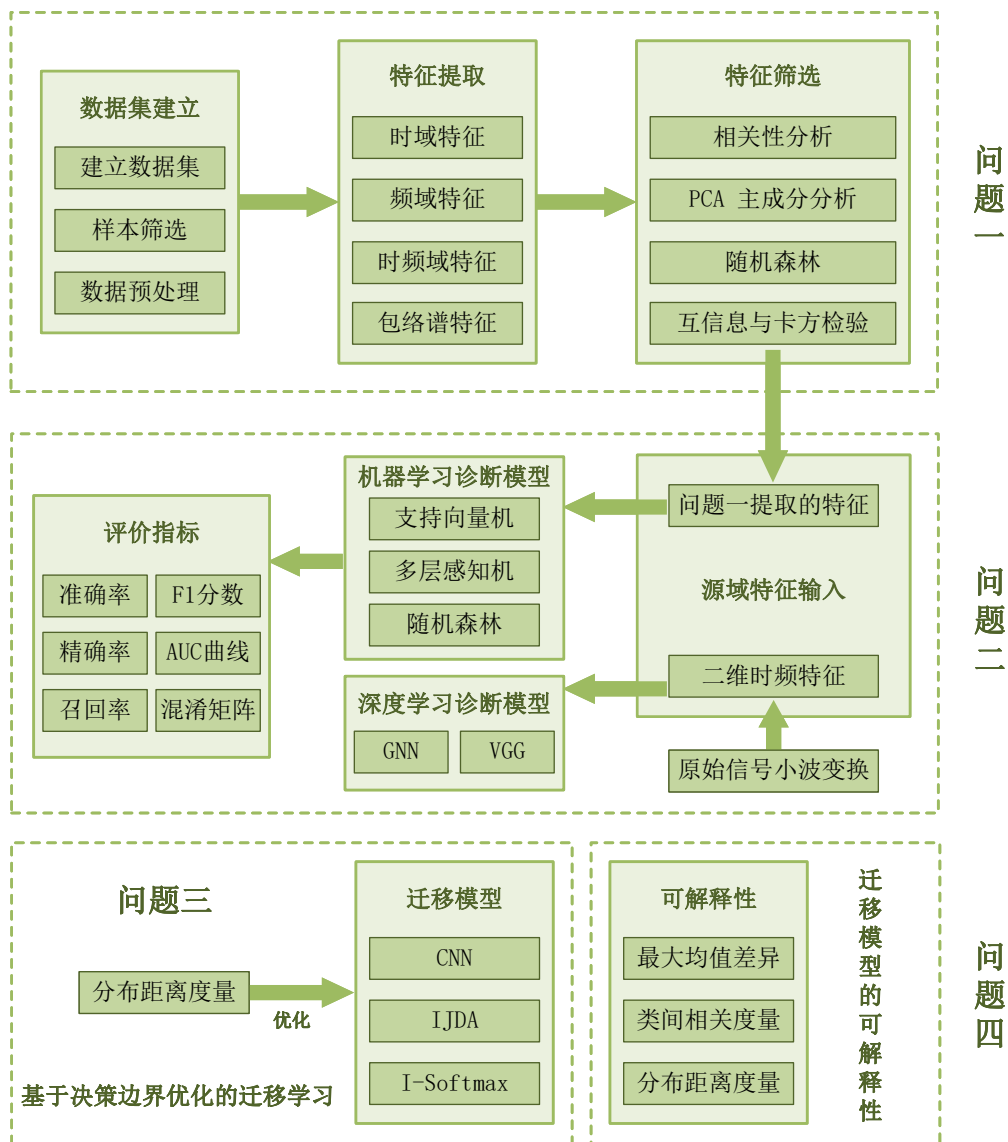


图 1.1 总体流程图

2 模型假设与符号说明

2.1 模型假设

1. 轴承故障形式为单点局部缺损（非复合故障），每组数据仅有一个轴承损坏。
2. 不同故障尺寸下同一工作状态不存在明显差异。
3. 试验台架数据与实际列车数据的故障物理机理相似，满足迁移学习的适用条件。

2.2 定义与符号说明

本文中出现的符号及其相关含义说明如下表 2.1 所示。

表 2.1 本文涉及的符号及说明

符号	意义	符号	意义
X	源域	Y	目标域
N	样本数量	f_s	采样频率
f_{s_in}	原采样频率	f_{s_out}	目标采样频率
T	采样个数	x_i	采样点
f_{max}	最高频率成分	f_r	轴承转频
N_d	滚动体数	d	滚动体直径
D	轴承节径	$\phi(x)$	高维特征空间映射
P_{signal}	残差信号能量	P_{noise}	谱基线能量
s_k	采样点频谱	f_k	第 k 条谱线的频率
M	信号总时长	d_i	第 i 个数据对位次差
σ	均值	$cov(\cdot)$	协方差
$W^{(l)}$	第 l 层权重	$b^{(l)}$	第 l 层偏置
$f(\cdot)$	激活函数	$\mathcal{L}$	损失
δ	梯度参数	η	学习率
$\mathcal{D}_S$	源域	$\mathcal{D}_T$	目标域
P	概率分布	$\mathbb{E}$	期望值
T	特征表示	F_i	特征向量

3 数据集介绍与可视化

3.1 数据集介绍

本研究所用数据采集于一套电机驱动的轴承转子实验平台，该平台在驱动端（DE, Drive End）和风扇端（FE, Fan End）分别布置加速度传感器，用于采集轴承运行过程中的振动信号。实验中通过人为破坏在轴承内圈、外圈和滚动体上制造不同尺寸的局部缺陷，从而得到典型的轴承故障数据。源域与目标域数据集的基本情况如表 3.1 所示。

表 3.1 数据集概况

数据集	故障类型	样本数量	平均轴承转速	采样频率 - 时间
源域	外圈故障 OR	77	1762 rpm	12kHz - 8s 48kHz - 4s
	内圈故障 IR	40		
	滚动体故障 B	40		
	正常状态 N	4		
目标域	—	16	600 rpm	32kHz - 8s

由表 3.1 可知，源域数据集包含外圈故障、内圈故障、滚动体故障以及正常工作状态四类运行状态。目标域数据集则在不同转速（600 rpm）和采样频率（32 kHz）的条件下采集的真实列车运行数据，用于验证模型的跨域适应性能。外圈故障、内圈故障和滚动体故障是最常见的三类轴承故障，其形成机理与振动特征如图 3.1 所示。

- **外圈故障**通常由长期磨损、装配不良或过载应力引起，导致外圈表面产生裂纹或不规则磨损区。当滚动体通过缺陷区域时，会产生周期性冲击，振动信号中在 BPFO（Ball Pass Frequency Outer race）及其谐波处出现明显特征。
- **内圈故障**主要由于摩擦、过载或材料疲劳导致裂纹或凹陷。当滚动体与内圈接触时，周期性撞击会激发较强的冲击成分，在 BPFI（Ball Pass Frequency Inner race）及其倍频处产生突出峰值。
- **滚动体故障**是由于循环应力作用下的疲劳破坏，使得滚动体表面出现裂纹或剥落。当滚动体在工作过程中与滚道接触不规则时，会在 BSF（Ball Spin Frequency）附近产生特征频率分量，同时在调制后的包络谱中显现明显冲击特征。

3.2 数据集可视化

在本研究中，为进一步分析不同采样频率和传感器位置下的故障信号特征，对源域与目标域数据进行了可视化。

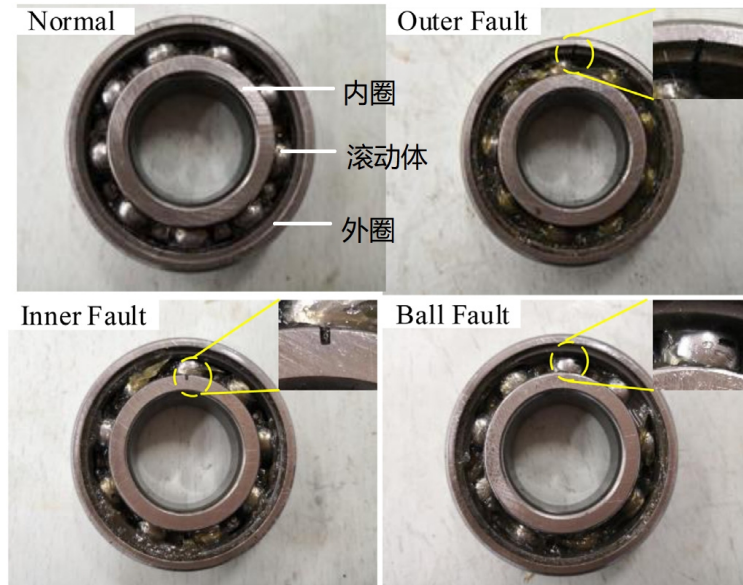


图 3.1 轴承故障示意图

3.2.1 源域数据集

当载荷为 0 时，驱动端采集的信号能够直接反映转子激励与轴承缺陷的相互作用。从图 3.2 中可以观察到，正常工况下的振动波形呈现平稳的周期性，幅值较小，没有明显突发冲击。而在外圈故障情况下，波形出现规律性尖峰，幅值最高可达约 $5m/s^2$ ，并呈明显周期性，表明外圈缺陷对振动信号具有显著影响。内圈故障波形也表现出类似的周期性冲击，但峰值幅度较小（约 $3m/s^2$ ），相较外圈故障影响较弱。滚动体故障的波形则更为复杂和不规则，振幅高于正常状态，但冲击规律性不如外圈与内圈故障明显。

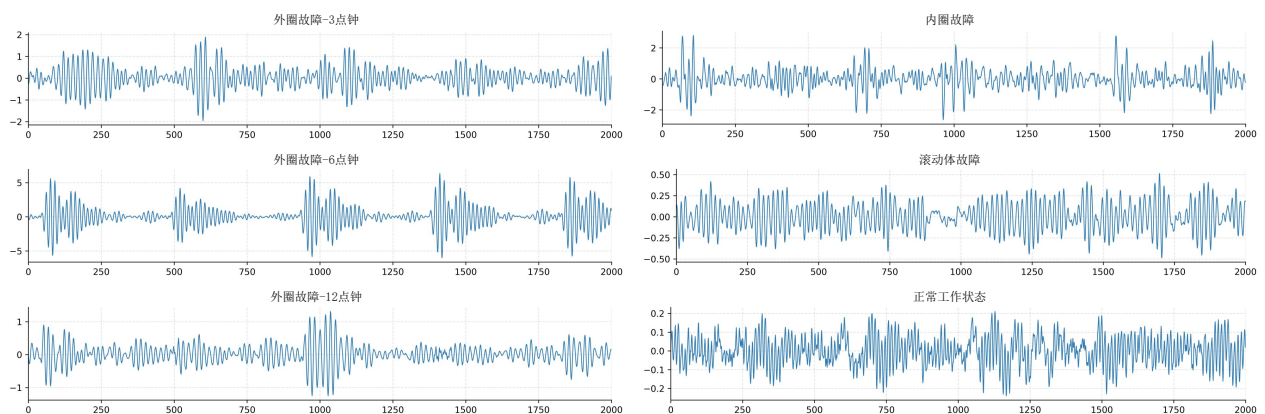


图 3.2 驱动端（DE）采样，48kHz，SKF6205 轴承，故障尺寸 0.007 英寸

在风扇端采集的信号波动情况如图 3.3 所示。外圈故障 6 点钟方向的冲击特征尤为显著，最大振动加速度约为 $3m/s^2$ ，表现出清晰的周期性。内圈故障信号也呈现规律性波动，但冲击峰值约为 $2m/s^2$ ，幅值小于外圈故障。滚动体故障的波形较为不规则，难以形成稳

定的冲击周期，体现出滚动体缺陷对振动响应的复杂性。

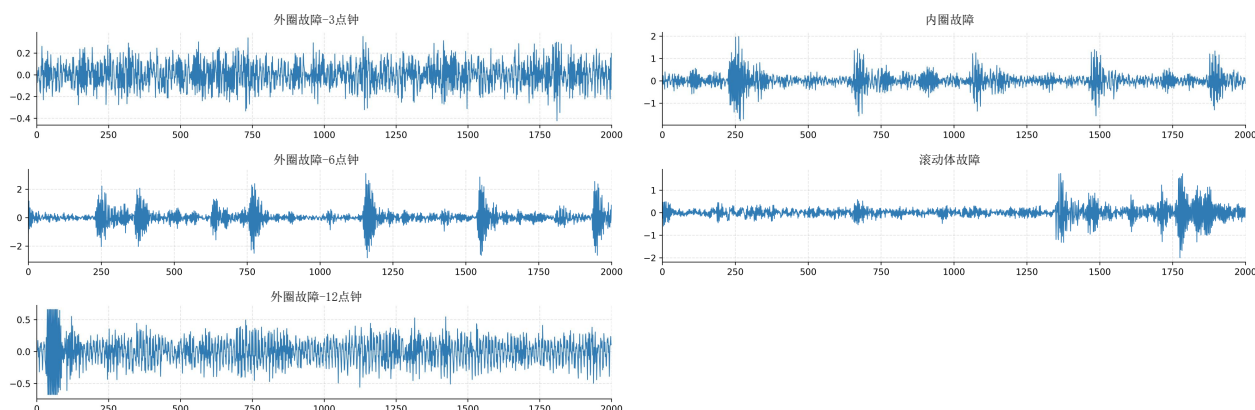


图 3.3 风扇端 (FE) 采样, 12 kHz, SKF6203 轴承, 故障尺寸 0.007 英寸

由于外部噪声干扰、传感器故障或设备本身的非线性振动等原因，在基座位置采集的信号出现频繁波动（如图 3.4所示），未呈现出显著的故障特征，常用于辅助分析。

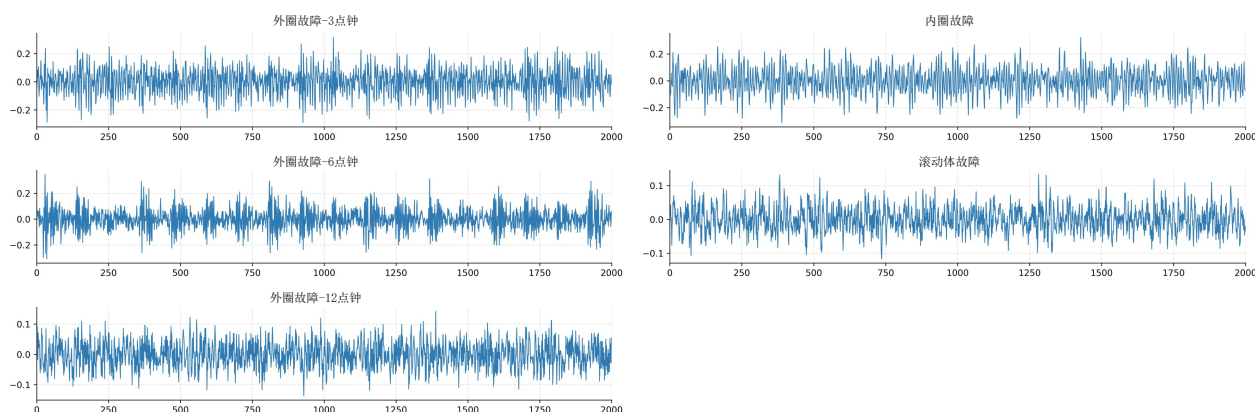


图 3.4 基座 (BA) 采样, 12kHz, SKF6205 轴承, 故障尺寸 0.007 英寸

3.2.2 目标域数据集

为深入分析目标域数据的特征分布，本研究对其进行了多维度可视化，包括时域、频域和包络谱，结果如图 3.5 所示。具体而言，将目标域数据集中的 16 个样本（标记为 A-P）进行如下处理：

- **时域波形**：截取信号前 0.2s 的时域波形，直观反映瞬态特征及幅值差异。
- **Welch 功率谱密度 (PSD)**：采用 Welch 方法估计功率谱密度，频率范围 0–10 kHz，用以展示各信号在频谱能量分布上的差异。
- **包络谱**：对信号进行 Hilbert 变换提取包络，并计算其频谱（0–500 Hz），以突出调制成分和低频特征。

从结果可见，部分样本在时域中存在明显冲击与周期性成分，在 PSD 中体现为能量峰

值的频段差异，而包络谱则揭示了调制频率分布的不同。这些差异说明目标域数据不仅在幅值动态范围上存在变化，也在频率分布和调制特性上具有异质性，为后续的特征提取与迁移学习研究提供直观参考。

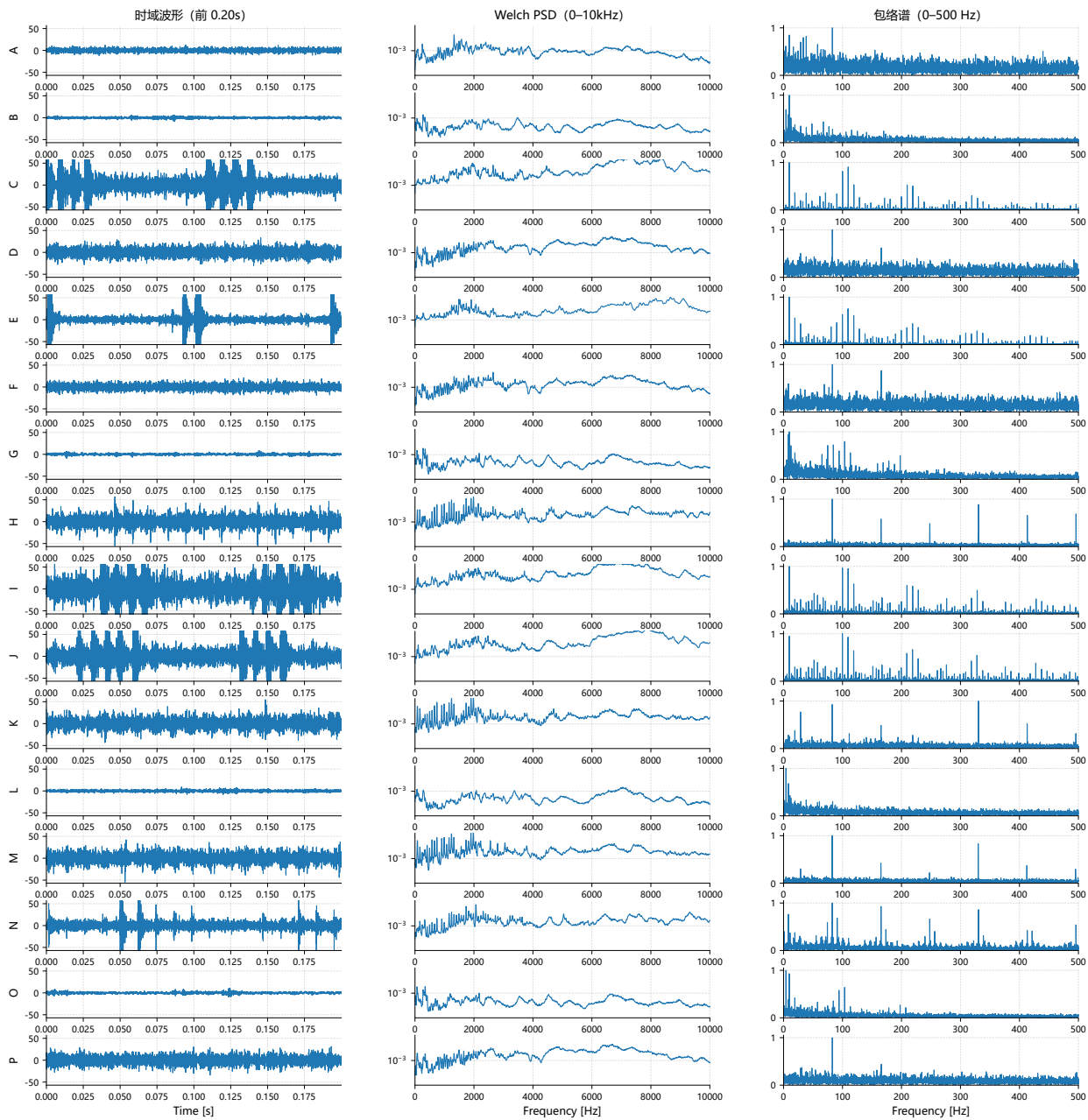


图 3.5 目标域数据集的可视化结果

4 问题一求解及分析

4.1 问题分析

在高速列车走行系统中，轴承作为典型的高转速、高负载部件，其外圈、内圈或滚动体一旦出现缺陷，往往会产生周期性冲击与调制效应。若不能及时识别，将直接威胁运行安全与系统可靠性。然而，现实工况下的振动信号受到载荷波动、转速变化及环境噪声等多重因素干扰，使得故障特征易被削弱甚至掩盖，从而增加了诊断难度。传统依赖人工经验构造特征或单一信号处理手段的方法，在跨工况、跨设备的应用场景中往往泛化不足，难以适应复杂多变的运行环境。

针对以上问题，本文在“源域—目标域”迁移诊断的框架下，聚焦完成三项核心任务，即从提供的源域数据中筛选出具有代表性、且最有助于迁移的子集，并结合目标域数据的特性开展特征分析与特征提取。在数据筛选方面，本文充分考虑了源域与目标域之间的差异与关联，综合分析了源域与目标域的采样频率差异、故障尺寸的合理性、如何放置传感器可以得到更优信号、外圈故障中不同点位的差异性，得到了具有代表性且最有助于迁移的源域数据集。考虑到原始样本存在数量稀缺和类别极度不平衡的问题，本文采用了滑动窗口重采样策略。通过设计不同重叠尺寸的窗口，将连续振动信号划分为多个片段，一方面扩充了样本数量，另一方面平衡了不同类别的比例，为后续模型训练提供了充足且均衡的数据基础。在特征提取方面，本文从时域、频域、时频域以及包络谱四个角度全面刻画轴承故障信号特征。在特征选择环节，本文结合多种方法对冗余或低相关特征进行筛选，包括基于皮尔逊相关系数与斯皮尔曼秩相关系数的相关性分析、PCA 主成分分析、随机森林特征重要性排序以及互信息与卡方检验。经过多方法交叉验证后，本文最终确定了一组既具物理可解释性又能显著提升诊断性能的有效特征，为后续的源域故障诊断及跨域迁移学习奠定了坚实基础。整体思路流程如图 4.1 所示。

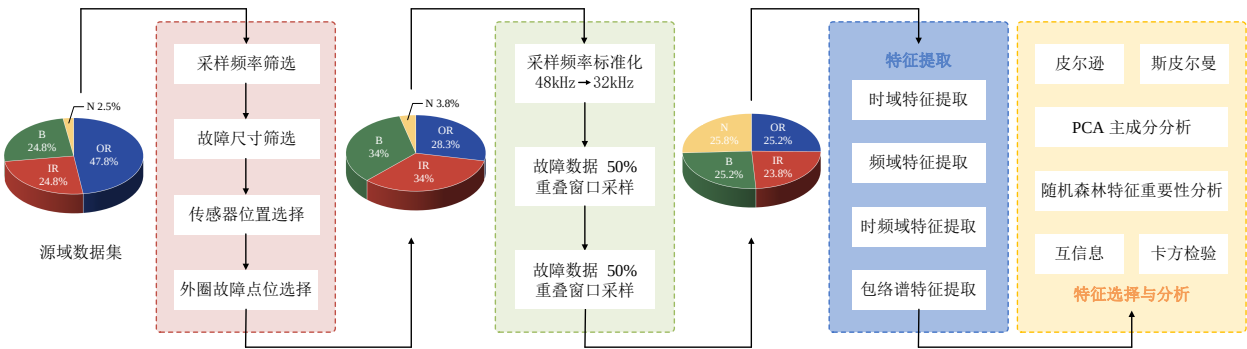


图 4.1 问题 1 求解流程图

4.2 源域样本筛选与数据集构建

4.2.1 数据集建立

根据源域数据集的样本分布（见表 4.1），轴承共有四类工作状态：外圈故障（OR）、内圈故障（IR）、滚动体故障（B）以及正常工作状态（N）。其中，外圈故障包含三种故障尺寸（0.007、0.014、0.021 英寸），内圈与滚动体故障各包含四种故障尺寸（0.007、0.014、0.021、0.028 英寸）。为覆盖不同工况，实验在 0、1、2、3 马力载荷下采集数据，对应主轴转速约为 1797、1772、1750、1730rpm。由于外圈位置固定不随轴承旋转，因此其振动信号在三个点位采集：3 点钟（Orthogonal）、6 点钟（Centered）、12 点钟（Opposite）。

表 4.1 源域数据集各类别样本量

故障尺寸	马力	轴承转速	OR			IR	B	N
			orth	cen	opp			
0.007	0	1797	3	3	3	3	3	—
	1	1772	3	3	3	3	3	—
	2	1750	3	3	3	3	3	—
	3	1730	3	3	3	3	3	—
0.014	0	1797	1	3	—	3	3	—
	1	1772	1	2	—	3	3	—
	2	1750	1	2	—	3	3	—
	3	1730	1	2	—	3	3	—
0.021	0	1797	2	3	2	3	3	—
	1	1772	3	2	2	3	3	—
	2	1750	3	2	2	3	3	—
	3	1730	3	2	2	3	3	—
0.028	0	1797	—	—	—	1	1	—
	1	1772	—	—	—	1	1	—
	2	1750	—	—	—	1	1	—
	3	1730	—	—	—	1	1	—
None	0	1796	—	—	—	—	—	1
	1	1772	—	—	—	—	—	1
	2	1750	—	—	—	—	—	1
	3	1725	—	—	—	—	—	1

在本研究中，考虑到目标域仅包含 OR、IR、B、N 四类状态，我们在源域中也保持相同的类别划分，不再根据故障尺寸进行进一步细分。最终仅保留部分 $48kHz$ 下的外圈故障（6 点钟）、内圈故障、滚动体故障，以及正常状态的四类数据。这样既保证了源域与目标域类别的一致性，又避免了故障尺寸差异带来的不必要干扰。

4.2.2 采样频率与故障尺寸选择

在轴承故障诊断任务中，采样频率直接决定了故障特征是否能够被完整捕获。根据奈奎斯特采样定理，采样频率必须满足：

$$f_s \geq 2f_{max} \quad (4.1)$$

其中， f_{max} 为信号中最高频率成分。以轴承转速为 $1797rpm \approx 29.95Hz$ 为例，根据 SKF6205 轴承尺寸参数（滚动体数 $N_d = 9$ ，滚动体直径 $d = 0.3126in$ ，轴承节径 $D = 1.537in$ ），可计算理论故障特征频率如下：

- 外圈故障特征频率（Ball Pass Frequency on the Outer Race, BPFO）

$$f_r \cdot \frac{N_d}{2} \cdot \left(1 - \frac{d}{D}\right) \approx 107.36Hz \quad (4.2)$$

- 内圈故障特征频率（Ball Pass Frequency on the Inner Race, BPFI）

$$f_r \cdot \frac{N_d}{2} \cdot \left(1 + \frac{d}{D}\right) \approx 162.19Hz \quad (4.3)$$

- 滚动体故障特征频率（Ball Spin Frequency, BSF）

$$f_r \cdot \frac{D}{d} \left[1 - \left(\frac{d}{D}\right)^2\right] \approx 141.17Hz \quad (4.4)$$

由上述公式可以得出，这些故障特征频率本身处于低频段。然而，轴承局部缺陷在运行中会激发强烈的冲击响应，该冲击信号会调制到轴承的结构共振区，通常落在 $8 \sim 15kHz$ 的频带内。而 $12kHz$ 采样仅能保留至 $6kHz$ ，必然导致关键信息丢失。相比之下， $48kHz$ 采样可完整覆盖宽频带冲击信号，更适合后续特征提取与分类建模。因此，我们舍弃了 $12kHz$ ，仅保留 $48kHz$ 数据。

在故障尺寸方面，我们选择了 0.007、0.014 和 0.021 英寸三种典型损伤。它们覆盖了从轻微到中等程度的常见损伤范围，能够提供清晰的类别区分。而 0.028 英寸属于严重损伤，数据量有限，且特征过于极端，不利于构建具有普适性的分类模型。

4.2.3 传感器位置选择

在轴承故障诊断中，传感器位置的选择直接影响到故障信号的检测和特征提取的效果。从轴承故障机理的角度来看，驱动端（DE）是位于动力源附近，故障信号最先在此

产生并直接影响系统。由于传递路径较短，DE 能够在故障的早期阶段捕捉到较为敏感的故障特征，特别是在频域中，DE 能够清晰展现轴承故障的特征频率。相比之下，风扇端（FE）和基座端（BA）由于信号传播衰减或混叠，往往难以保持同样的精确度。时域分析也显示，DE 端的振动变化更为明显，有利于及时检测出故障。

为了量化源域与目标域之间的差异，并客观地选择最合适的传感器位置，我们采用了最大均值差异（Maximum Mean Discrepancy, MMD）作为评估指标。MMD 是一种常用于比较两个数据分布差异的统计方法，尤其适用于跨域学习和迁移学习。MMD 通过计算源域数据与目标域数据在高维特征空间中的均值差异来度量它们的分布差异。MMD 的值越小，说明源域与目标域之间的分布越接近，迁移学习的效果也越好；反之，MMD 值越大，分布差异越大，迁移学习的困难度就越高。具体来说，我们将源域数据 X 和目标域数据 Y 映射到一个特征空间中，计算它们的均值差异，计算公式如下：

$$\text{MMD}(X, Y) = |\mathbb{E}_{x \sim X}[\phi(x)] - \mathbb{E}_{y \sim Y}[\phi(y)]| \quad (4.5)$$

其中， $\phi(x)$ 表示数据点 x 在高维特征空间中的映射， $\mathbb{E}$ 表示均值运算。我们采用了时域特征（如均值、均方根、峰值因子等）和包络谱特征（如 BPFO、BPFI、BSF 能量）对源域和目标域数据进行了比较分析。根据各指标的 MMD 值（如图 4.2 所示），我们得出以下结论：

1. 在脉冲因子和裕度因子两个指标上，DE 显著优于 FE，且在均值和峰值因子指标上 DE 也略优于 FE。这表明 DE 对于捕捉由轴承故障引发的冲击特征有较强的灵敏性，适合早期故障检测。
2. 在 BPFO、BPFI 和 BSF 能量上，DE 的 MMD 值显著低于 FE，可见 DE 对轴承故障特征频率的敏感性更强，能够更准确地检测故障特征频率。
3. 在均方根和峭度两个指标上，FE 与目标域更接近，表明目标域的信号整体分布情况与 FE 更相似。然而，需要注意的是，源域数据是在台架实验环境下采集的，而目标域数据来自真实的列车运行过程。由于这两种环境存在一定差异，均方根和峭度的 MMD 差异可能是信号特性的变化，因此在这两个数据下的 MMD 值的置信度有所下降。

综上所述，DE 端由于更靠近激励源，传递增益更高，且能够更好地捕捉外圈故障的脉冲信号，具有较高的故障诊断灵敏度。相较于 FE 和 BA，DE 端在外圈故障诊断中的优势更加明显。因此，我们选择 DE 作为主要的故障诊断通道，确保能够捕捉到更精确的故障特征。

4.2.4 外圈故障采样点位选择

传统信噪比（Signal-to-Noise Ratio, SNR）通常定义为信号功率与噪声功率之比（dB），是衡量信号中有用信息相比于背景噪声强度的指标，高 SNR 表示信号清晰，噪音影响小；反之则表示信号模糊，噪音影响大。然而在轴承振动诊断中，原始信号中包含大量背景噪声和非故障成分，直接采用整体功率比容易受到偶发尖峰和背景能量波动的干扰。为此，

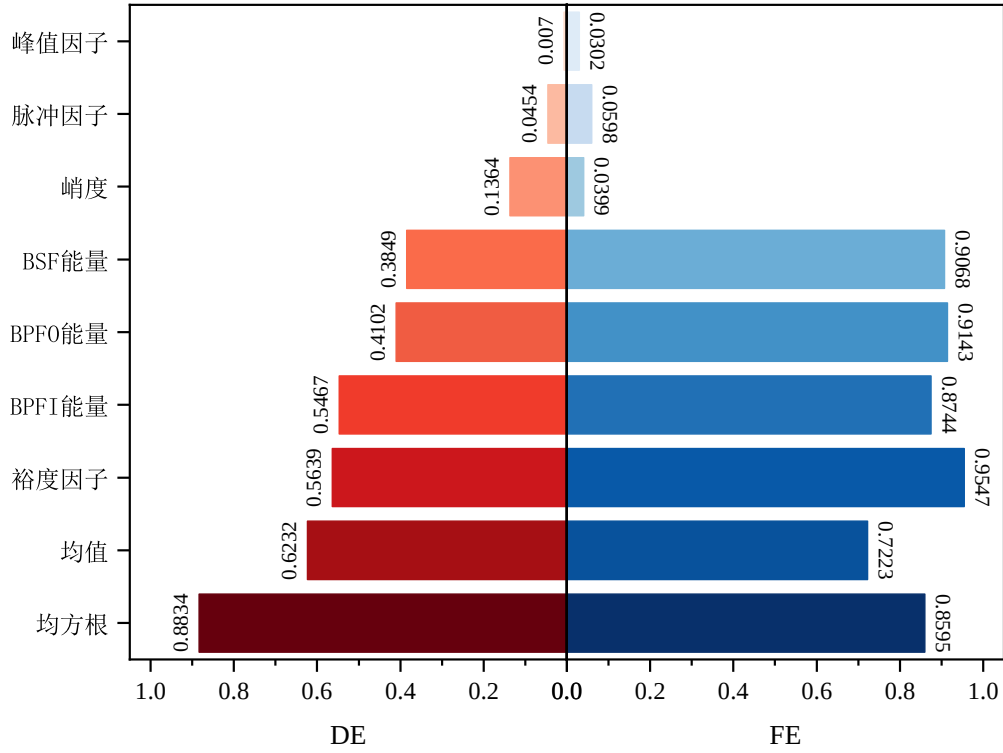


图 4.2 目标域数据集的可视化结果

本文在此基础上，在包络谱中对外圈故障特征频率（BPFO，见式 4.2）及其若干倍谐波与转频侧带附近的残差信号能量与噪声基线进行对比，从而获得更稳定、可比性更强的信噪比度量。其计算公式为：

$$SNR = 10 \log_{10} \frac{P_{\text{signal}}}{P_{\text{noise}}} \quad (4.6)$$

其中， P_{signal} 为在 BPFO 及其谐波和侧带处的残差信号能量，残差通过从包络谱中减去滚动中位数滤波获得的谱基线后取正值得到； P_{noise} 则为去除特征频率邻域后，基于滚动中位数估计得到的谱基线能量。具体来说，首先对原始振动信号进行带通滤波以保留轴承共振频带，随后通过 Hilbert 变换得到包络信号，并采用 Welch 方法计算包络谱。最后依据上述方法分离信号与噪声能量，得到 SNR 指标。

本文利用 SNR 指标对 3 点位、6 点位和 12 点位的振动信号进行了比较。图 4.3 的结果表明，三个点位在总体趋势上保持一致，说明单点测量即可反映故障的发展情况。在此基础上，6 点位表现出更显著的动态范围与信噪比优势，能够在不同工况下捕捉到最清晰的故障特征。相比之下，3 点与 12 点位虽然响应更稳定，但缺乏突出的敏感性。由此可见，6 点位既能代表整体趋势，又具有更强的故障敏感性，是外圈故障研究与诊断的最合适代表位置。需要指出的是，在高载荷（3 马力）工况下，三点位的 SNR 值均出现大幅下降，这可能与信号激励特性变化及所用特征提取方法的局限性有关。由于三个点位在该工况下表现一致，该现象可视为方法在极端条件下的边界情况，而不影响低至中载荷条件下得出的主要结论。

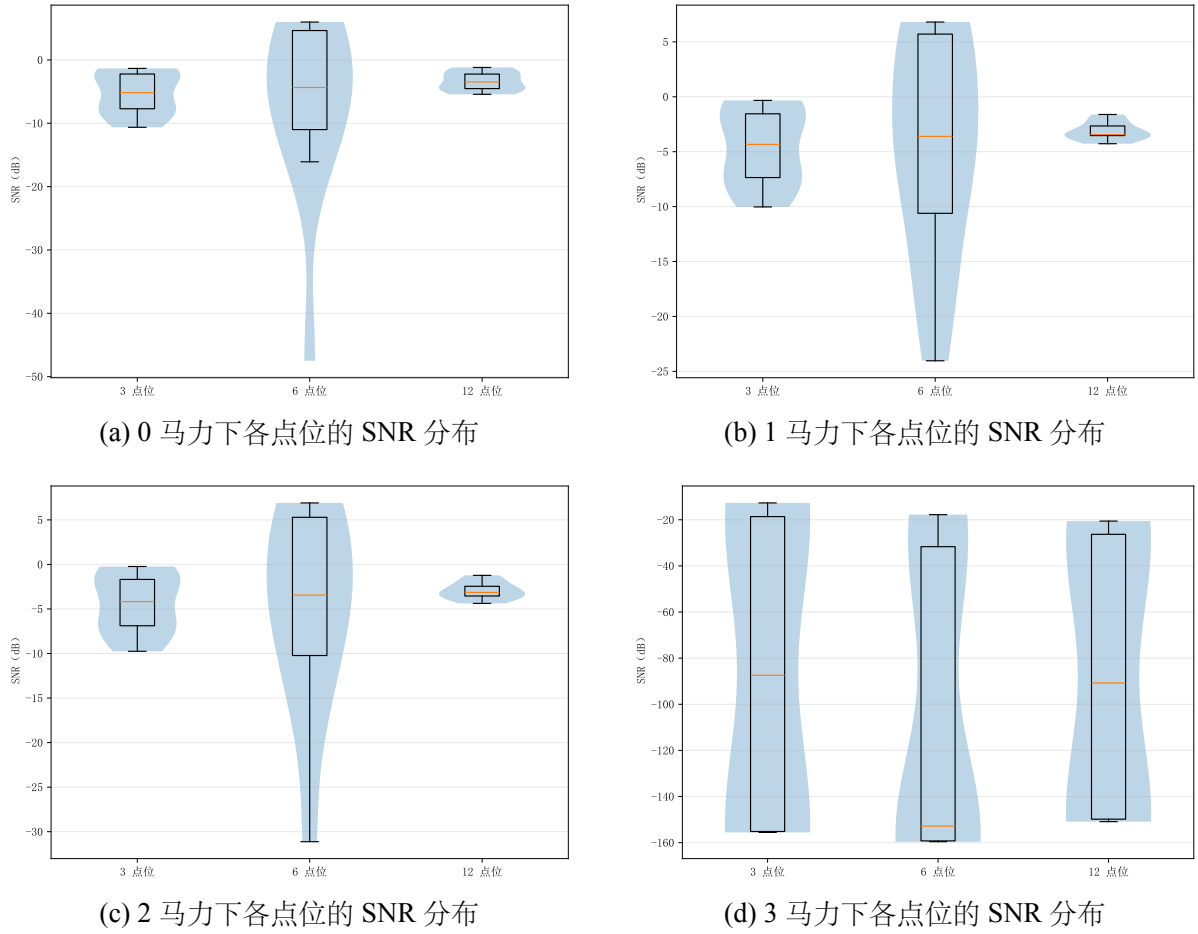


图 4.3 不同载荷下 3/6/12 点位的 SNR 分布情况

4.3 数据预处理

为降低源域和目标域的分布偏移，两个数据集采用完全一致的预处理流程，以便后续特征对齐。

4.3.1 采样频率设置

在数据预处理阶段，源域和目标域之间存在采样频率不匹配的问题。根据 4.2 节的内容，本文选取了源域数据集中采样频率为 48kHz 的样本，而目标域数据集的采样频率为 32kHz。这一频率差异可能导致信号在时域和频域特性上的不同，从而影响后续域迁移学习的效果。为了确保源域和目标域的数据在同一频率尺度下进行有效比较和学习，本文将源域数据集的采样频率从 48kHz 重采样到 32kHz。

为了避免在降采样过程中出现混叠现象，我们采用了抗混叠滤波技术。我们首先通过计算原采样频率 ($f_{s_in} = 48kHz$) 和目标采样频率 ($f_{s_out} = 32kHz$) 之间的最大公约数 ($GCD = 16$) 来确定上采样 ($up = f_{s_in}/GCD = 2$) 和下采样因子 ($down = f_{s_out}/GCD = 3$)。在重采样过程中，信号首先会进行上采样，然后通过低通滤波器去除信号中的高频成

分，最后进行下采样。我们选择的低通滤波器的截止频率为 16kHz，这一设置符合奈奎斯特采样定理的要求，可以有效避免信号中高频部分的混叠。通过这一过程，信号的频谱信息得以保留，同时避免了不必要的失真和干扰。

本文所采用的抗混叠重采样方法，能够在降低采样频率的同时尽量保留信号的原始信息，特别是那些在 32kHz 频率范围内依然具有有效信息的部分。此外，重采样后的源域数据集采样频率与目标域数据集的采样频率一致，均为 32kHz。这一统一处理使得源域和目标域数据能够在相同的频率尺度上进行对比和学习，从而避免了频率差异对后续域迁移学习模型的影响，提高了模型的学习效果和准确性。

4.3.2 数据扩充与窗口划分

由于原始数据量相对较少，考虑到数据的多样性和变异性，本文决定采用窗口划分的方式来扩充样本量。窗口划分不仅可以有效增加样本数量，还能够通过合理设置窗口重叠率来缓解数据不平衡的问题，从而提供更多的训练数据供模型进行学习。当原始数据量不足时，简单地将数据集划分成多个较小的片段可以有效增加样本量，从而提供更多的训练样本。然而，直接将数据划分成不重叠的片段可能会导致数据不平衡或简化信号的时序信息。因此，本文通过设置不同的窗口重叠率来确保数据的平衡性，同时最大限度保留信号的时序特征，避免信息丢失。

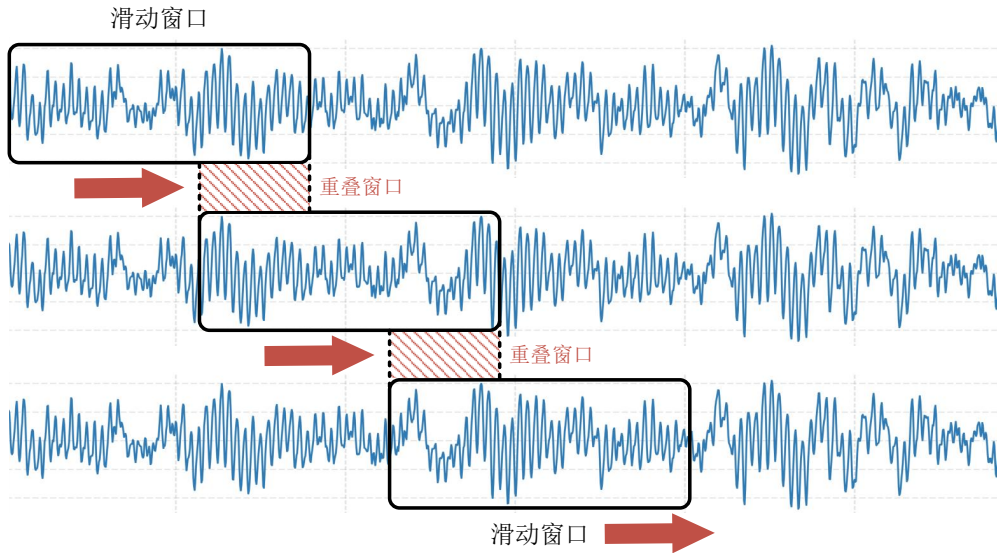


图 4.4 重叠滑动窗口

本文采用固定时间长度（即固定时间点数）的方式来划分信号，有利于规范化信号的长度，确保所有的窗口都具有相同的时长，避免由于信号长度不一致而产生数据不平衡问题。考虑到原始数据的不平衡比 ($\rho = N_{max}/N_{min}$) 高达 19.25，正常工作状态样本严重缺失，本文通过调整窗口重叠率来缓解数据不平衡带来的问题。窗口重叠不仅能够调整样本数量，还能增强不同数据片段之间的共享信息，从而减少模型对单一数据片段的过拟合，

使得模型能够更加稳健地学习到信号的全局特性。图 4.4 展示了信号划分方式以及如何通过滑动窗口的重叠部分来扩充样本量。窗口的重叠部分使得每个窗口之间共享了一定的信息，从而保证了信号时序信息的平滑过渡和充分利用。具体参数设置如表 4.2 所示。通过合理设置窗口大小与重叠率，能够确保每类信号的样本数量得到均衡，尤其是在正常工作状态样本较为稀缺的情况下，重叠窗口的使用极大地缓解了这一问题。

表 4.2 滑动窗口参数设置

类别	窗口重叠率	划分后样本量	窗口大小
外圈故障	50%	732	4096
内圈故障		690	
滚动体故障		732	
正常工作状态	70%	748	

4.4 特征提取

滚动轴承发生故障时，其振动特征会随之变化，因此可根据其振动信号，分别从时域、频域、时频域和包络谱对其相关特征进行提取和分析。以下将从这四个方面展开介绍对应的特征指标。

4.4.1 时域特征提取

时域特征值是衡量振动信号特征的重要指标，通常分为有量纲特征和无量纲特征。其中有量纲特征是最常用的特征指标，因为往往具有直观地物理含义，以下将从均值、方差、标准差、均方根（RMS）、峰值、峰对峰值、均值绝对值对其进行分析，其中 x_i 表示振动信号的每一个采样点， M 表示采样点总数。

- **均值（Mean）** 代表信号的平均，作为对真值的一种估计，计算公式如下：

$$\text{Mean} = \frac{1}{M} \sum_{i=1}^M x_i \quad (4.7)$$

- **标准差（Standard Deviation）** 描述信号对均值的集中程度，计算公式如下：

$$\text{Variance} = \frac{1}{M} \sum_{i=1}^M (x_i - \text{Mean})^2 \quad \text{Standard Deviation} = \sqrt{\text{Variance}} \quad (4.8)$$

- **均方根（RMS）** 又称有效值，常用于表示振动信号强度，计算公式如下：

$$\text{RMS} = \sqrt{\frac{\sum_{i=1}^M x_i^2}{M}} = \sqrt{\frac{x_1^2 + x_2^2 + \cdots + x_M^2}{M}} \quad (4.9)$$

- **峰值 (Peak Value)** 描述信号的最大变化，计算公式如下：

$$\text{Peak} = \max(|x_1, x_2, \dots, x_M|) \quad (4.10)$$

- **峰对峰值 (P2P)** 反映信号极限波动，计算公式如下：

$$\text{P2P} = \max(|x_1, x_2, \dots, x_M|) - \min(|x_1, x_2, \dots, x_M|) \quad (4.11)$$

- **均值绝对值 (Mean Absolute)** 反映信号的整体幅度，计算公式如下：

$$\text{Mean Absolute} = \frac{1}{N} \sum_{i=1}^N |x_i| \quad (4.12)$$

- **均方值 (SQR Mean)** 反映信号的有效幅度，计算公式如下：

$$\text{SQR Mean} = \left[\frac{1}{N} \sum_{i=1}^N \sqrt{|x_i|} \right]^2 \quad (4.13)$$

有量纲特征虽有直观的物理含义且对振动信号特征比较敏感，但是易受环境干扰，其不仅与轴承运动状态有关，还与轴承本身参数有关。而无量纲特征能有效地排除这些干扰的影响，因此无量纲特征也被广泛应用与特征提取中。以下将从峰度、偏度、形状因子、峰值因子、脉冲因子、裕度因子和清除因子方面对振动信号进行分析：

- **峰度 (Kurtosis)** 反映信号的尖锐程度，计算公式如下：

$$\text{Kurtosis} = \frac{1}{M} \sum_{i=1}^M \left(\frac{x_i - \text{Mean}}{\text{Standard Deviation}} \right)^4 \quad (4.14)$$

- **偏度 (Skewness)** 反映了振动信号分布的不对称性，计算公式如下：

$$\text{Skewness} = \frac{1}{M} \sum_{i=1}^M \left(\frac{x_i - \text{Mean}}{\text{Standard Deviation}} \right)^3 \quad (4.15)$$

- **形状因子 (Shape Factor)** 反映信号的平均幅度，计算公式如下：

$$\text{Shape Factor} = \frac{\text{RMS}}{\text{Mean Absolute}} \quad (4.16)$$

- **峰值因子 (Crest Factor)** 反映信号瞬时波动情况，计算公式如下：

$$\text{Crest Factor} = \frac{\text{Peak Value}}{\text{RMS}} \quad (4.17)$$

- **脉冲因子 (Impulse Factor)** 反映振动信号脉冲的显著性，计算公式如下：

$$\text{Impulse Factor} = \frac{\text{Peak Value}}{\text{Mean Absolute}} \quad (4.18)$$

- **裕度因子 (Margin Factor)** 反映振动信号中的冲击特性，计算公式如下：

$$\text{Impulse Factor} = \frac{\text{Peak Value}}{\text{SQR Mean}} \quad (4.19)$$

- **清除因子 (Clearance Factor)** 反映信号的平稳性，计算公式如下：

$$\text{Clearance Factor} = \frac{\text{RMS}}{\text{Mean}} \quad (4.20)$$

4.4.2 频域特征提取

尽管时域分析能够提供一些振动信号的基本特征，但信号的变化不仅仅依赖于时间，还受到频率、相位等因素的影响。因此，单纯的时域分析可能不足以揭示信号的深层次信息，若利用傅里叶变换等方法，将复杂的振动信号分解为不同频率的正弦波，能够更清晰地揭示信号的频率特征，由于频谱均值、频谱方差、频谱峰度和频谱偏度计公式与 4.4.1 节相似，因此以下着重介绍频谱重心、频域带宽和频谱熵三个特征，并做进一步分析，其中 $s(k)$ 表示振动信号 x_i 的频谱， $k = 1, 2, \dots, K$ ； K 是谱线数； f_k 是第 k 条谱线的频率； S_{Mean} 表示频谱均值。

- **频谱重心 (Spectral Centroid)** 反映频率分布的中心位置，计算公式如下：

$$\text{Spectral Centroid} = \frac{\sum_{k=1}^K s(k) f_k}{\sum_{k=1}^K s(k)} \quad (4.21)$$

- **频谱带宽 (Spectrum Bandwidth)** 反映了频谱成分的扩展范围，计算公式如下：

$$\text{Spectral Variance} = \sqrt{\frac{\sum_{k=1}^K (f_k - S_{Mean})^2 s(k)}{\sum_{k=1}^K s(k)}} \quad (4.22)$$

- **频谱熵 (Spectral Entropy)** 反映频谱能量分布的混乱程度（如图 4.5 所示），计算公式如下：

$$\text{Spectral Entropy} = \sum_{k=1}^K p_k \log(p_k), \quad p_k = \frac{s(k)}{\sum_{k=1}^K s(k)} \quad (4.23)$$

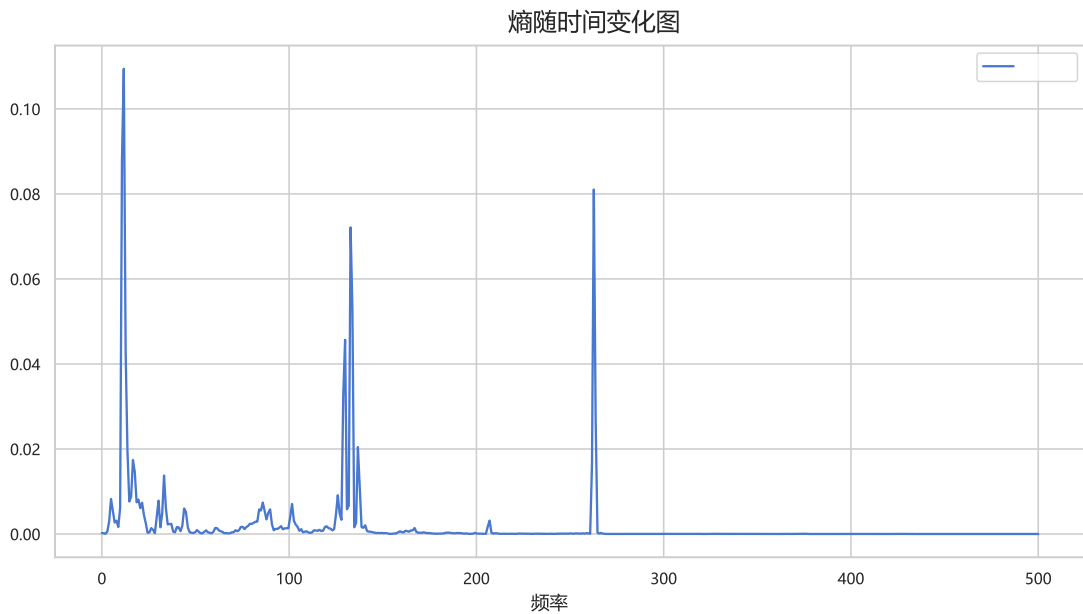


图 4.5 频谱熵随时间变化图

4.4.3 时频域特征提取

除了对轴承信号进行时域和频域分析获得对应特征，还应对信号的时间历程变化进行分析以提取瞬态信息 [9]，且轴承故障的振动信号往往是不平稳信号，其频谱会跟随时间而变化，时频域分析可以有效弥补传统时域和频域分析无法全面描述这一变化的缺点。利用短时傅里叶变换、小波变换等可以将振动信号从时域信息转换到时频域，相比于短期傅里叶变换，小波变换对非平稳信号和突发信号更加有效，且更加灵活，因此以下选用小波变换来获取轴承振动的时频域信息，以下从短时能量、短时频域重心、时频熵、时频均值频率、小波能量方面对时频域特征进行分析。其中 $Z(f, t)$ 表示振动信号在频率 f 和时间 t 上的幅度谱； f_k 表示第 k 条谱线的频率； $p(f_k)$ 表示振动信号在频率 f 和时间 t 上的概率密度函数； c_i 表示第 i 层小波变换的系数。

- **短时能量 (ST Energy)** 表示每个时刻的短时能量，计算公式如下：

$$\text{ST Energy} = \sum |Z(f, t)|^2 \quad (4.24)$$

- **短时频域重心 (ST Frequency)** 反映信号频率的集中位置，计算公式如下：

$$\text{ST Frequency} = \frac{\sum f_i \cdot |Z(f_k, t)|^2}{\sum |Z(f_k, t)|^2} \quad (4.25)$$

- **时频熵 (TF Entropy)** 反映时频能量的复杂度，计算公式如下：

$$\text{TF Entropy} = - \sum p(f, t) \log(p(f, t)) \quad (4.26)$$

- **时频均值频率 (TF Mean Frequency)** 反映能量的频率分布，计算公式如下：

$$\text{TF Mean Frequency} = \sum f_k \cdot p(f_k) \quad (4.27)$$

- **小波能量 (Wavelet Energy)** 反映了信号在不同频带上的能量分布，计算公式如下：

$$\text{Wavelet Energy} = \sum c_i^2 \quad (4.28)$$

4.4.4 包络谱分析特征提取

信号的包络是指信号幅度随时间的变化趋势，通过包络分析可以在从复杂的振动信号中提取出低频调制成分，从而可以获得隐藏在高频噪声中的低频特征。由于轴承故障故障往往伴随着周期性的冲击，产生调幅信号，因此通过包络分析，可以有效的提取这一特征，进而识别轴承故障。

对振动信号进行包络分析后得到包络信号，可以通过傅里叶变化等进一步对其进行频谱分析，得到包络谱。包络谱可以能够揭示信号的频率成分，重点关注由于轴承故障产生的低频部分。以下介绍包络谱的具体获取方法：

假设原始信号为 $x(t)$, 信号总时长是 T , 首先对其进行去均值处理: $x_{\text{mean-removed}}(t) = x(t) - \bar{x}$, 其中 $\bar{x} = \frac{1}{T} \int_0^T x(t) dt$; 然后对去均值后的信号进行希尔伯特变换获得解析信号:

$$z(t) = x_{\text{mean-removed}}(t) + i \cdot \mathcal{H}\{x_{\text{mean-removed}}(t)\} \quad (4.29)$$

$$\mathcal{H}\{x_{\text{mean-removed}}(t)\} = \frac{1}{\pi} \int_{-\infty}^{\infty} \frac{x_{\text{mean-removed}}(\tau)}{t - \tau} d\tau \quad (4.30)$$

其中 $\mathcal{H}\{x_{\text{mean-removed}}(t)\}$ 是 $x_{\text{mean-removed}}(t)$ 的希尔伯特变换, 此变换可以将信息中的包络提取出来。

再对解析信号进行取模, 得到只包含正频率的包络信号:

$$e(t) = |z(t)| = \sqrt{x_{\text{mean-removed}}^2(t) + (\mathcal{H}\{x_{\text{mean-removed}}(t)\})^2} \quad (4.31)$$

之后再对包络信号进行去均值: $e_{\text{mean-removed}}(t) = e(t) - \bar{e}$, 其中 $\bar{e} = \frac{1}{T} \int_0^T e(t) dt$; 最后对去均值的包络信号进行快速傅里叶变换 (FFT) 得到包络谱:

$$E(f) = \mathcal{F}\{e_{\text{mean-removed}}(t)\} = \int_{-\infty}^{\infty} e_{\text{mean-removed}}(t) \cdot e^{-i2\pi ft} dt \quad (4.32)$$

对提取到的包络信号进行进一步分析, 选取其最大最小值、均值、标准差、峰度和偏度进行分析; 对包络谱进行频谱均值、频谱方差分析、包络能量和包络熵谱进行分析, 由于计算公式与上文 4.4.1、4.4.2 和 4.4.3 节所示公式相同, 这里就不再赘述。

4.5 特征选择与分析

本文从时域、频域、时频域和包络谱三个角度进行特征提取, 得到如表 4.3 中所示的初始特征。通过对原始信号做小波变换并在 33 个尺度上汇聚能量, 得到短时能量作为时频域二维特征。

4.5.1 描述统计性分析

从热力图 4.6 中可以看出, 部分时域与包络谱特征与轴承故障类别呈显著相关, 说明这些变量对类别区分具有较强的判别力。同时, 特征之间出现成簇的高相关, 反映出明显的冗余与多重共线性。

4.5.2 基于皮尔逊与斯皮尔曼的相关性分析

本文从时域、频域和包络谱三个维度共提取了 39 个特征。为了深入了解这些特征与标签之间的关系, 我们首先采用了皮尔逊相关系数 (Pearson Correlation Coefficient) 和斯皮尔曼秩相关系数 (Spearman Rank Correlation Coefficient) 两种方法, 从不同的角度计算了每个特征与故障类型之间的相关性, 为后续特征选择和模型构建提供指导。

- **皮尔逊相关系数**常用来衡量两组数据之间的相关性, 取值范围为 $[-1, 1]$, 正值表示正相关, 负值表示负相关。可以通过以下取值范围判断变量的相关强度: 0.0-0.2 为极弱

表 4.3 本文提取的 39 个初始特征（其中 ith 表示小波变换的第 i 层）

特征	对应名称	特征	对应名称
mean	均值	std	标准差
var	方差	rms	均方根
peak	峰值	p2p	峰对峰值
mean_abs	均值绝对值	shape_factor	形状因子
crest_factor	峰值因子	impulse_factor	冲击因子
margin_factor	边缘因子	clearance_factor	清除因子
kurtosis	峰度	skewness	偏度
spectrum_mean	频谱均值	spectrum_variance	频谱方差
spectrum_centroid	频谱重心	spectrum_bandwidth	频谱带宽
spectrum_skewness	频谱偏度	spectrum_kurtosis	频谱峰度
spectrum_entropy	频谱熵	st_energy	短时能量
st_freq	短时频域重心	tf_entropy	时频熵
tf_mean_freq	时频均值频率	wavelet_energy_0	小波 1th 近似系数
wavelet_energy_1	小波 1th 细节系数	wavelet_energy_2	小波 2th 细节系数
wavelet_energy_3	小波 3th 细节系数	wavelet_energy_4	小波 4th 细节系数
envelope_max	包络最大值	envelope_min	包络最小值
envelope_mean	包络均值	envelope_std	包络标准差
envelope_kurt	包络峰度	envelope_skew	包络偏度
envelope_spectrum	包络谱	envelope_energy	包络能量
envelope_spectrum_entropy	包络谱熵		

相关，0.2-0.4 为弱相关，0.4-0.6 为中等程度相关，0.6-0.8 为强相关，0.8-1.0 为极强相关。计算公式如下：

$$\rho_{X,Y} = \frac{\text{cov}(X,Y)}{\sigma_X \sigma_Y} = \frac{E(XY) - E(X)E(Y)}{\sqrt{E(X^2) - E^2(X)}\sqrt{E(Y^2) - E^2(Y)}} \quad (4.33)$$

其中， $\text{cov}(X,Y)$ 表示 X 特征和 Y 标签的协方差， σ_X 、 σ_Y 分别表示 X 和 Y 的标准差。

- 斯皮尔曼秩相关系数则是一种用于衡量两个变量之间的单调关系的非参数统计指标。与皮尔逊相关系数不同，斯皮尔曼相关系数不依赖于变量的分布和数据的线性特征即

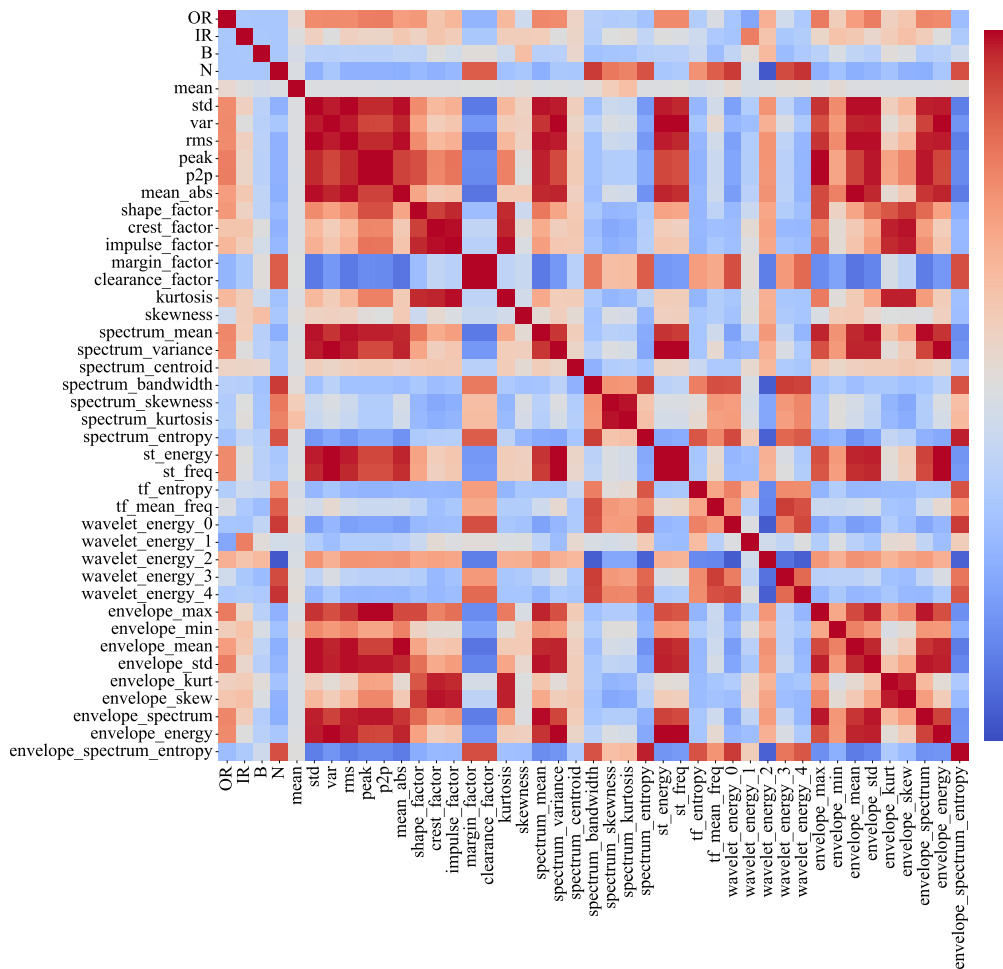


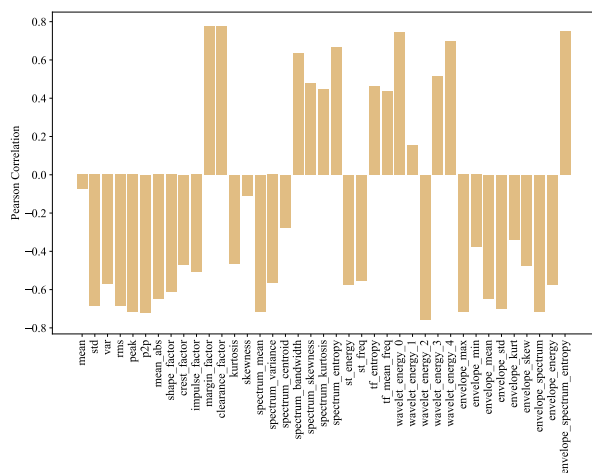
图 4.6 特征与故障类别之间的相关性热力图

不要求数据满足正态分布，而是基于变量值的排序来计算它们之间的相关性。计算公式如下：

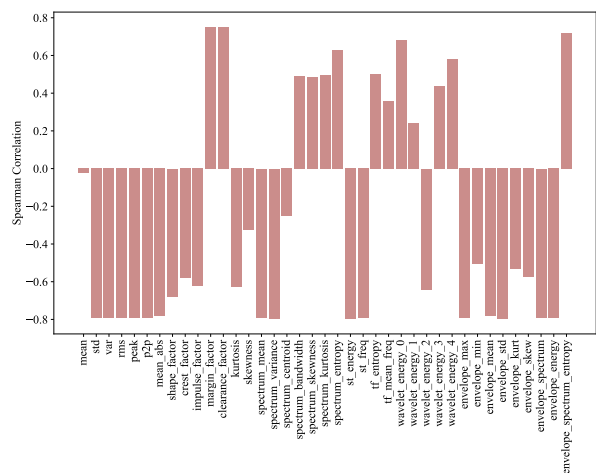
$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)} \quad (4.34)$$

其中， d_i 表示第 i 个特征标签数据对的位次值之差； n 表示总的数据个数。

由图 4.7 可知，在 39 个特征中，有多数的特征与标签具有相关性，只有均值、偏度、小波第一层细节系数、频谱重心等特征呈现弱相关性，其他特征均呈现中等或强相关性。从 39 个特征值中选取皮尔逊相关系数和斯皮尔曼相关系数均大于 0.6 的特征，这些特征均与标签呈强相关。如表 4.4 所示。边缘因子和清除因子在两个指标上均表现出了显著的正相关与负相关，这些特征可能对轴承故障的分类起到了关键作用。此外，小波 1th 近似系数和小波 2th 细节系数在皮尔逊相关系数的检测中，都表现出了较强的相关性，不同程度地反映了在该项指标上不同故障类别存在一定的差异。在斯皮尔曼秩相关系数检测中，除小波 1th 近似系数、小波 2th 细节系数、频谱熵和形状因子外，其他特征也表现出了较强的正相关或负相关，说明它们在轴承故障分类上的影响呈现单调上升或下降的趋势。



(a) 皮尔逊



(b) 斯皮尔曼

图 4.7 皮尔逊和斯皮尔曼

表 4.4 相关系数表

特征	皮尔逊相关性	斯皮尔曼相关性
边缘因子	0.773622	0.75248
清除因子	0.773622	0.75248
峰对峰值	-0.719427	-0.791771
包络最大值	-0.717433	-0.791595
峰值	-0.716431	-0.791967
包络谱	-0.715739	-0.789802
频谱均值	-0.713177	-0.790015
包络标准差	-0.697581	-0.799314
均方根	-0.685688	-0.794511
标准差	-0.684825	-0.790494
包络谱熵	0.749044	0.718594
均值绝对值	-0.648146	-0.782302
包络均值	-0.64748	-0.780653
小波 1 - th 近似系数	0.744356	0.682807
小波 2 - th 细节系数	-0.755096	-0.641074
频谱熵	0.665463	0.630574
形状因子	-0.613561	-0.678278

4.5.3 PCA 主成分分析

主成分分析 (Principal Component Analysis, PCA)，又称主分量分析或矩阵数据分析，是一种无监督降维技术，常用于降低数据的维度。PCA 通过变量变换的方法将相关变量变为若干不相关的综合指标变量 [10]，依次选取方差最大的变量作为主成分，这些主成分尽可能解释数据的最大方差，即最大程度保留了数据中的差异信息。同时去除了冗余特征，有效地降低了模型过拟合的风险，减少了后续的特征处理工作量。详细流程如图 4.8 所示。

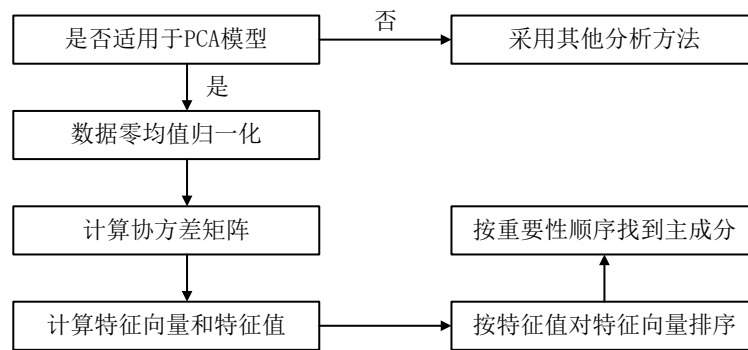


图 4.8 主成分流程图

PCA 的结果如图 4.9 所示，可以看出，前几个主成了解释了绝大部分的方差，PC1 占据了 51.63% 的方差，PC2 和 PC3 分别贡献了 17.35% 和 11.46%。累计方差贡献达到 96.35%，这意味着我们可以通过前 9 个主成分来表示数据的绝大部分信息。PC1-PC3 主成分中各特征的贡献情况如表 4.5 所示。其中，PC1 主要反映了信号的幅度特征，包络标准差、峰值、峰对峰值、均方根等与信号的能量、波动和幅度变化相关的特征有较大的贡献。PC2 则主要反映了频率特征，特别是与频率相关的时频均值频率、第三层小波变换细节因子等特征，反映了信号频谱的变化和形态特征。PC3 主要反映了信号的形态学特征，如峰度和冲击因子，这些特征与信号的极端值和波形形态相关。

综上所述，本文最终选择 9 个主成分来作为分类特征，这些主成分保留了数据的大部分变异性。通过选择贡献较大的特征来确保分类任务中使用到的特征是最有代表性的，具体如下：

- **幅度相关特征**：包络标准差、峰值、峰对峰值、均方根、标准差、频谱均值、包络最大值、包络谱、均值绝对值、包络均值。
- **频率相关特征**：时频均值频率、小波能量、频谱偏度、频谱峰度、频谱带宽。
- **形态学特征**：峰度、冲击因子、形状因子、包络峰度、包络偏度、频谱熵。

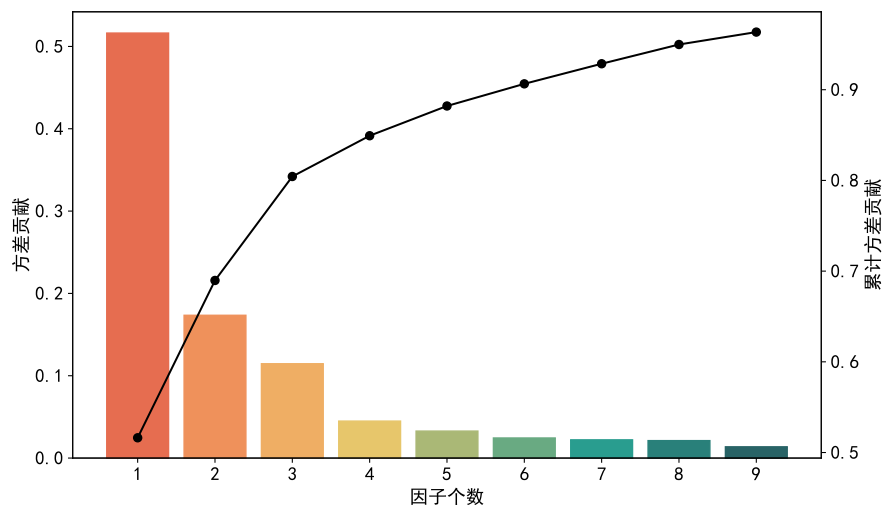


图 4.9 PCA

表 4.5 Top10 主成分特征贡献

PC1	贡献值	PC2	贡献值	PC3	贡献值
包络标准差	0.2100	时频均值频率	0.2688	峰度	0.2901
峰值	0.2097	小波 3th 系数	0.2537	冲击因子	0.2731
峰对峰值	0.2096	包络峰度	-0.2530	形状因子	0.2728
均方根	0.2096	频谱偏度	0.2440	包络峰度	0.2634
标准差	0.2096	峰值因子	-0.2375	包络偏度	0.2541
频谱均值	0.2078	包络偏度	-0.2325	频谱熵	0.2506
包络最大值	0.2068	频谱峰度	0.2264	峰值因子	0.2446
包络谱	0.2055	频谱带宽	0.2116	频谱带宽	0.2346
均值绝对值	0.2020	冲击因子	-0.2099	偏度	-0.2186
包络均值	0.2019	峰度	-0.2090	小波 3th 系数	0.2074

4.5.4 基于随机森林算法的特征重要性分析

传统的特征筛选方法通常依赖于数据的统计特性，例如卡方检验、相关性分析等，这些方法通常假设特征之间是线性或独立的，可能忽略掉特征之间的复杂非线性关系，从而导致潜在的重要特征未能被识别。因此，为了更全面地评估每个特征的贡献，本文采用了基于随机森林的特征筛选方法。通过训练模型，我们可以更准确地量化每个特征对任务的贡献，并据此进行特征选择。

随机森林是一种基于决策树的集成学习方法，框架如图 4.10 所示，其核心思想来源于 Bagging 算法。每次训练时，随机森林从训练集中有放回地抽取 n 个样本形成新的子集，

并根据这些子集训练 m 个决策树模型。最终的预测结果通过对各个树的预测结果进行投票或平均来决定。这种方法通过引入随机性，减小了模型的方差，避免了过拟合，从而提高了预测的稳定性。随机森林的核心优势在于其内置了特征重要性评估机制，通过对每个特征在决策树分裂过程中所带来的信息增益或不纯度减少量进行计算，评估特征对模型预测结果的影响。因此，随机森林非常适合用来进行特征筛选，帮助我们识别对模型性能贡献最大的特征。其基本流程如下：

1. **数据抽样**：通过有放回地从原始训练集中抽取样本，形成多个不同的训练子集。
2. **决策树构建**：每个子集都用于训练一棵决策树。在训练过程中，随机选择部分特征用于分裂节点，这种特征选择的随机性使得每棵树的结构不同，增加了模型的多样性。
3. **集成投票**：最终，随机森林通过集成多个决策树的结果投票决定最终的预测结果，有效避免了单棵树的过拟合问题。
4. **特征重要性评估**：通过计算每个特征在多个决策树中的分裂贡献，评估各个特征对模型预测的重要性。

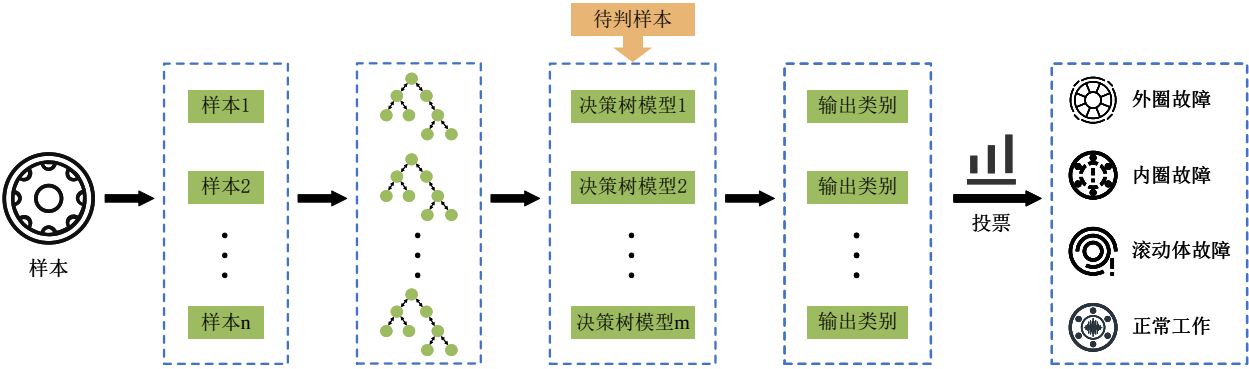


图 4.10 随机森林结构图

根据我们应用随机森林算法进行特征筛选的结果，模型为每个特征分配了一个重要性评分，如图 4.11 所示。这些评分代表了每个特征在分类任务中的贡献大小。可以发现，时域特征（如峰对峰、峰值、均方根）和包络谱特征（如包络谱标准差、包络谱最大值）对模型的预测能力起到了关键作用，在模型中具有较高的重要性。尤其是峰对峰、包络谱标准差等特征，它们在多个决策树的训练过程中频繁作为分裂节点，显示出它们在区分不同类别样本时的重要性。与此同时，也有部分特征对模型的贡献较小，甚至在多个决策树中几乎没有被选中。例如，频域特征中的一些特征（如频谱重心、频谱峰值）在本次分析中得到了较低的评分。这表明这些特征可能对模型分类效果的提升作用有限，或者它们的贡献可以被其他特征所替代。

最终，根据随机森林提供的特征重要性评分，我们筛选出了对分类任务贡献最大的 20 个特征。这些特征覆盖了时域特征、频域特征、包络谱特征和时频域特征，并按照重要性进行了排序。筛选出的特征如表 4.6 所示。

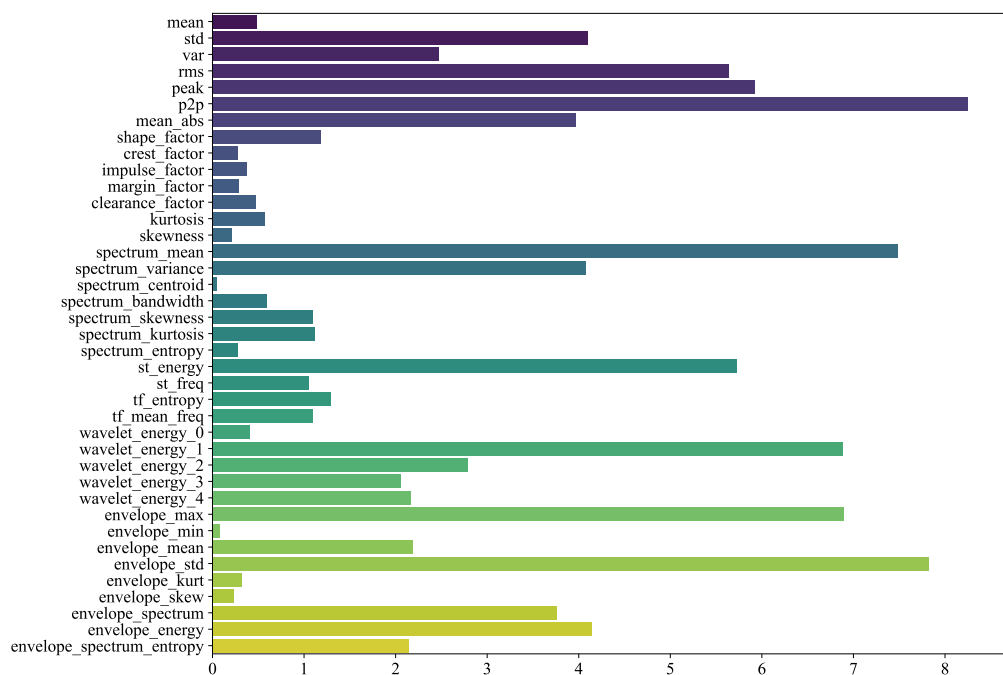


图 4.11 随机森林特征筛选

表 4.6 随机森林筛得 20 个特征

分类	特征	分类	特征
包络谱特征	包络标准差	时频域特征	1th 小波细节系数
	包络最大值		短时能量
	包络能量		2th 小波细节系数
	包络谱		3th 小波细节系数
	包络均值		4th 小波细节系数
	包络谱熵		时频熵
时域谱特征	峰对峰值	频域特征	
	峰值		
	均方根		频谱均值
	标准差		频谱方差
	均值绝对值		
	方差		

4.5.5 基于互信息与卡方检验的特征筛选

时频域特征通常通过短时傅里叶变换（STFT）或其他时频分析方法从信号中提取。这些特征在时间和频率的双重维度上提供了丰富的信息，可以帮助揭示信号中的频谱变化和时间依赖性。由于时频域数据具有二维结构（时间和频率），它们比时域和频域特征更加复杂，包含了信号的局部信息。因此，考虑到时频域数据的复杂性与多维特性，本文采用互信息（Mutual Information）和卡方检验（Chi-Square Test）对时频域特征单独进行特征筛选。

互信息（Mutual Information）又称转移信息，是用来衡量两个随机变量之间依赖关系的。在特征选择和数据分析中，互信息可以用来检验特征与标签（目标变量）之间的依赖性。具体来说，互信息衡量了在知道特征的值的条件下，能够减少标签的不确定性的程度。它不仅能够捕捉线性关系，还能够捕捉非线性关系，因此，互信息是特征与标签之间关联度的有效度量。对于特征 X 与标签 Y 的互信息计算公式如下：

$$I(X; Y) = \sum_{y \in Y} \sum_{x \in X} p(x, y) \log \left(\frac{p(x, y)}{p(x)p(y)} \right) \quad (4.35)$$

其中 $p(x, y)$ 是特征 X 和标签 Y 的联合概率分布函数，而 $p(x)$ 和 $p(y)$ 分别是特征 X 和标签 Y 的边缘概率分布函数。卡方检验是一种常用的假设检验方法。它基于实际频率与期望频率的偏差来计算卡方统计量，可以通过卡方分布来评估两者之间是否存在显著性差异。对于本文时频域特征和轴承运动状态标签来说，卡方检验的原假设 H_0 ：此特征与标签之间没有相关性。备择假设 H_1 ：此特征与标签之间有相关性。卡方列联表卡方统计量计算公式如下：

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i} \quad (4.36)$$

其中表示 O_i 表示观测频数； E_i 表示理论频数。自由度计算公式如下：

$$df = (R - 1)(C - 1) \quad (4.37)$$

其中 R 表示列联表的行数， C 表示列联表的列数。

互信息检验与卡方检验的实验结果如图 4.12 所示。特征 2、特征 18、特征 14 和特征 30 的互信息值最强，它们与故障类型有较强的依赖关系，表明这些特征在分类任务中有较强的贡献。卡方检验提供了每个特征的卡方值与 p 值，由于特征的 p 值都较小，取对数能够使其之间的差异放大，更好观察和分析。结果显示所有特征与标签均具有显著相关性：其中特征 5、特征 21 和特征 28 与标签相关性最高；特征 1、特征 6、特征 16、特征 22、特征 32 与其他特征相比，这些特征与标签相关性较低，但依旧与标签显著性相关。结合互信息和卡方检验可知，时频域特征与标签都有较强的依赖关系和显著的相关性，表明了时频域特征在轴承故障分析中的重要地位。在这些特征中特征 2、特征 5、特征 14、特征 18、特征 21 和特征 28 相比与其他特征，与标签的依赖性和相关性更强。

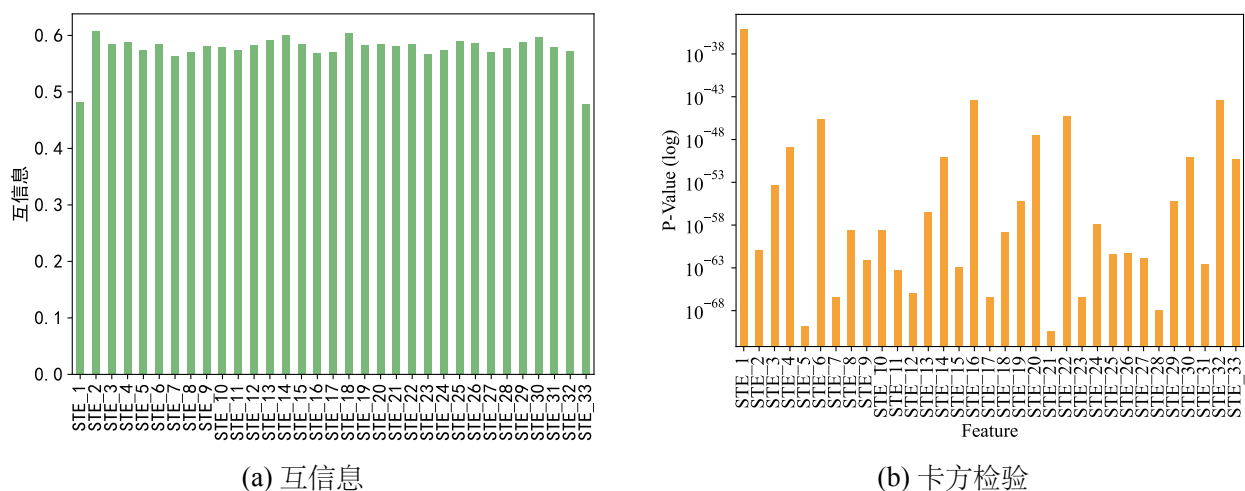


图 4.12 互信息与卡方检验

表 4.7 时频域卡方与互信息

显著性排名	互信息值	特征	卡方值	特征
1	0.60783	STE_2	366.942441*	STE_21
2	0.603627	STE_18	363.884526*	STE_5
3	0.599876	STE_14	355.094033*	STE_28
4	0.597829	STE_30	348.136611*	STE_7
5	0.59165	STE_13	348.136611*	STE_23
6	0.589456	STE_25	348.128114*	STE_17
7	0.589109	STE_4	345.46094*	STE_12
8	0.588903	STE_29	333.160212*	STE_11
9	0.586752	STE_26	331.205329*	STE_15
10	0.584815	STE_22	329.538586*	STE_31

4.6 本章小结

本章首先构建了源域数据集，涵盖了在 **48kHz** 采样频率下采集的四类典型轴承故障数据：外圈故障、内圈故障、滚动体故障以及正常状态。为保证数据的代表性和可迁移性，对样本进行了进一步筛选，即在故障尺寸上选择了 **0.007、0.014、0.021 英寸** 三种典型故障尺寸；在采样位置上选取 **DE** 端作为主要监测点；在外圈故障中保留 **6 点钟** 方向的信号。在数据预处理方面，采用**抗混叠重采样方法**将源域原始信号降至 **32kHz**，对齐目标域采样频率以便后续进行迁移任务，并采样**滑动窗口切分策略**扩充样本数量。通过设置不同的窗口重叠率在提升样本规模的同时又有效缓解了类别不平衡问题，最终形成了一个**相对均衡**

且具有代表性的数据集。随后开展特征提取工作，从四个维度全面表征轴承故障特征：在时域上提取了均值、标准差、均方根等 7 个有量纲特征和峰度、偏度、形状因子等 7 个无量纲特征；在频域上选取了频谱均值、频谱方差等共 7 个特征；在时频域上提取了短时频域重心等 9 个特征以及短时能量二维数据；在包络谱中获得了包络最大值、最小值等 9 个特征。为了去除冗余、保留关键特征，本文结合多种方法进行了筛选与验证。首先，通过基于皮尔逊和斯皮尔曼系数的相关性分析筛选出 17 个显著相关特征，采用主成分分析提取出贡献度较高的 9 个主成分（涵盖 21 个特征），根据随机森林模型识别出对分类性能贡献最大的 20 个特征。同时，互信息与卡方检验表明所有时频域特征均与故障模式高度相关。经过对比多个特征筛选方案的交集，最终得到 14 个兼具物理可解释性与判别力的有效特征（如表 4.8 所示），为后续源域诊断与跨域迁移学习奠定了坚实基础。

表 4.8 筛选得到最终 14 个有效特征

序号	特征	序号	特征
1	频谱均值	2	包络标准差
3	包络最大值	4	包络谱
5	峰对峰值	6	峰值
7	均方根	8	标准差
9	包络均值	10	均值绝对值
11	频谱熵	12	形状因子
13	包络谱熵	14	小波 2th 细节系数

5 问题二求解及分析

5.1 问题分析

在机械设备，尤其是轴承的故障诊断领域，传统的故障检测方法主要依赖于人工经验和基于规则的算法，这些方法往往在复杂环境下表现出较低的诊断准确性，尤其是在面对多种工况和多类故障时，传统方法的适应性和鲁棒性较差。随着智能化和自动化的发展，数据驱动的故障诊断技术逐渐成为主流，尤其是机器学习和深度学习方法，它们能够通过大量历史数据自动学习并提取特征，从而实现高效、精准的故障诊断。这些先进技术不仅能够突破人工经验的局限，而且具有较强的适应性，能够处理复杂的、非线性的故障模式。

根据问题二的要求，本文在问题一中提取的故障特征基础上，基于机器学习和深度学习学习方法设计了一个有效的轴承故障诊断模型。通过训练模型并评估其在测试集上的表现，能够实现对轴承故障的高效识别。本文不仅采用了问题一中筛选得到的特征数据作为输入特征来训练模型，还探索了时频域数据的潜力，将其转化为二维图像，以供深度学习模型进行训练学习。时频域数据包含了时间和频率两个维度的信息，能够更全面地刻画故障信号的变化规律，从而为故障诊断提供更加细致的特征支持。为了全面评估不同模型的表现，我们采用了多项性能评价指标，包括准确率 (ACC)、精确率 (Precision)、召回率 (Recall)、F1 值 (F1-Score) 和 AUC (曲线下面积)。此外，为了进一步分析不同模型的分类效果，本文还通过绘制混淆矩阵、Acc 曲线等方法对模型性能进行了可视化。通过实验，我们不仅比较了多种机器学习和深度学习方法在源域轴承故障诊断中的效果，也为后续跨域迁移学习研究提供了理论基础。整体思路流程如图 5.1 所示。

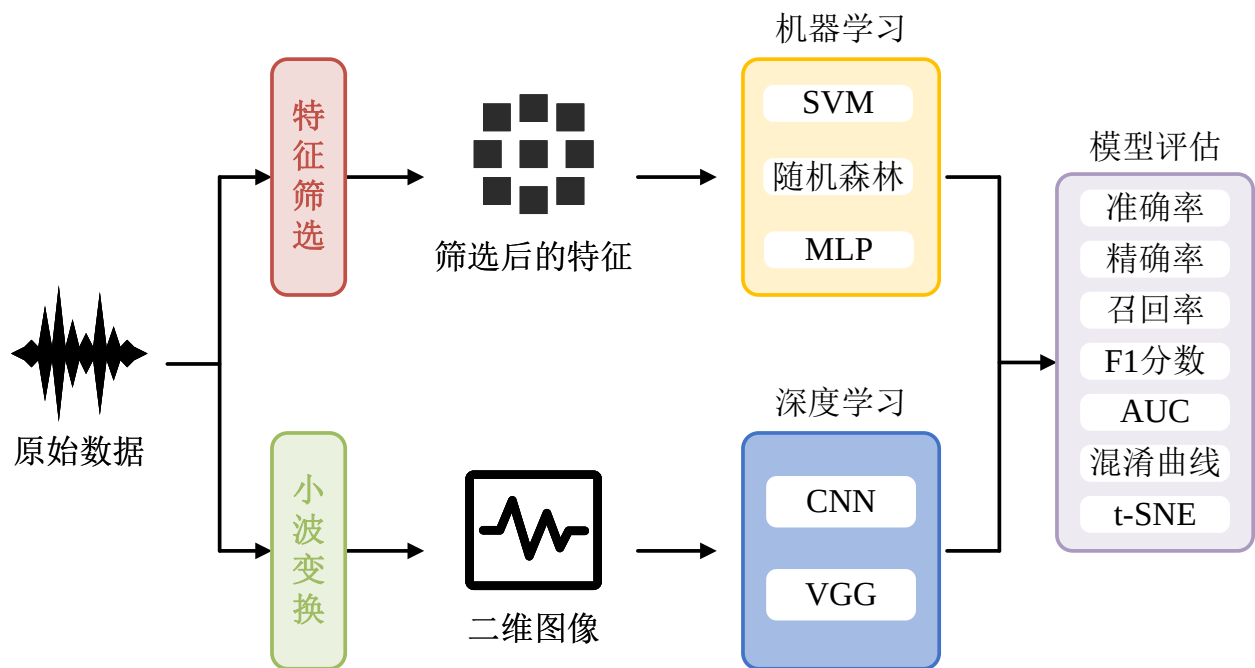


图 5.1 问题 1 求解流程图

5.2 源域特征输入

如 4.5 所示，本文通过**提取轴承的故障特征**，成功筛选出了与故障类型高度相关的特征。这些特征为后续的源域故障诊断任务提供必要支撑。与此同时，在问题一中本文还注意到**时频域数据**在故障诊断中具有较强的相关性，因此本文决定将时频域数据作为一种新的输入形式，用于训练和评估不同的机器学习和深度学习模型。

5.2.1 提取后的特征

在问题一的特征选择过程中，本文采用了皮尔逊相关系数、斯皮尔曼相关系数、PCA 主成分分析以及基于随机森林的重要性排序方法来筛选有效特征。通过皮尔逊相关系数和斯皮尔曼相关系数计算了各特征与故障类型之间的相关性，分别评估了特征与故障类型之间的线性关系和非线性关系，从而筛选出那些与故障类型关系较强的特征，并剔除掉不相关或冗余的特征。PCA 主成分分析方法则通过寻找数据中方差最大的方向，将原始高维特征转换为一组新的主成分，从而提取出最具代表性的特征。PCA 的优势在于能够保留数据中最大的信息量，同时减少特征空间的维度，有效避免模型过拟合。此外，本文还利用随机森林算法通过训练多棵决策树，并依据每棵树中的特征重要性排序，进一步评估了各特征对故障诊断任务的贡献，筛选出最具判别力的特征。

在特征筛选过程中，本文对不同方法筛选得到的特征进行了对比（如图 5.2 所示）。结果表明，部分特征在多个方法中均得到了验证。例如，频谱均值、包络标准差、包络最大值等 10 个特征在皮尔逊相关系数、斯皮尔曼相关系数、PCA 和随机森林方法中都得到了有效验证，表明这些特征能够有效反映故障信号的变化规律，因此它们在不同方法中均表现出较强的诊断能力。此外，频谱熵和形状因子也通过了相关性分析和 PCA 的检验，显示出了它们在故障诊断中的重要性。频谱熵能够衡量信号的复杂度，形状因子则能够描述信号波形的规律性，这两个特征有助于区分不同的故障模式。包络谱和第二层小波变换细节系数则在皮尔逊相关系数和随机森林方法中得到了验证，它们与轴承的故障状态具有密切关系。包络谱能够揭示轴承故障的频率特征，而小波变换细节系数则能够描述信号在多尺度下的特征，尤其适用于非平稳信号的分析。

通过以上方法的综合验证，本文最终选择了 14 个特征作为源域故障诊断任务中的有效输入特征。这些特征涵盖了频率域、时域和时频域的各类信息，能够全面地反映轴承故障的不同方面。

5.2.2 二维图像表示与处理

除了基于特征选择的输入数据外，本文还充分利用了时频域数据在故障诊断中的优势。传统的一维时序信号虽然能够反映轴承运行状态，但往往难以全面揭示信号在时间和频率两个维度上的变化规律。为此，本文将原始振动信号通过小波变换 (Wavelet Transform) 的方法转化为二维时频图像，从而在时频域的角度对信号进行综合分析。

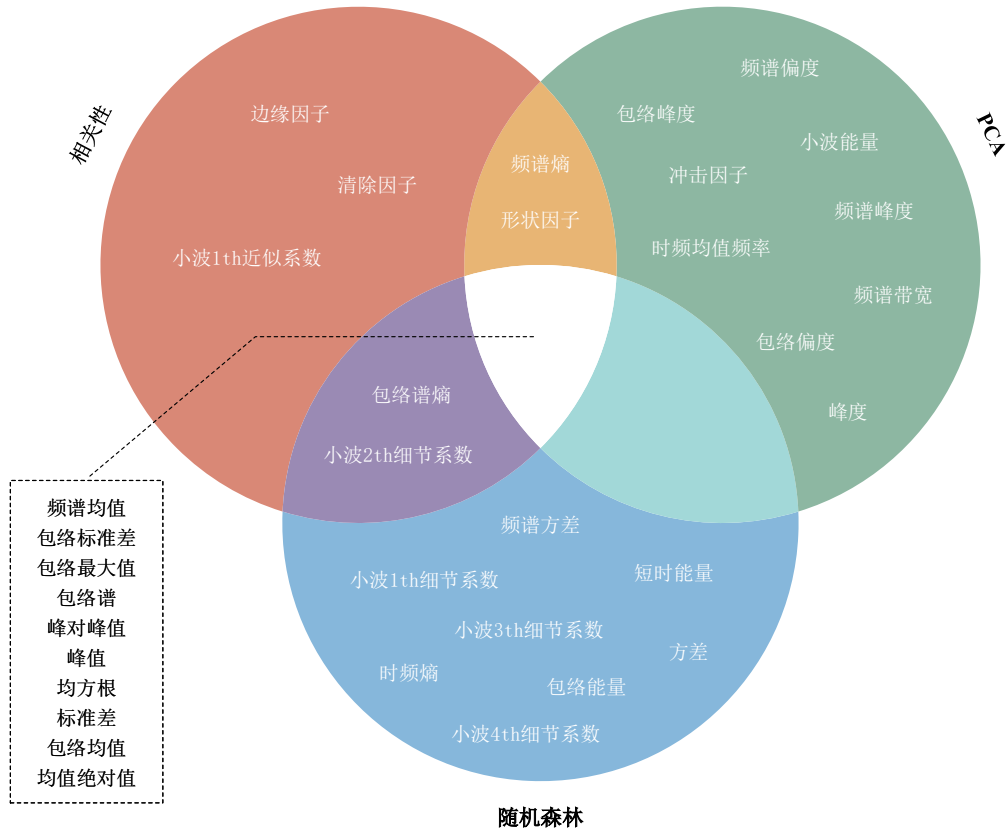


图 5.2 特征筛选结果

小波变换是一种能够同时在时域和频域对信号进行局部化分析的工具，与傅里叶变换只能提供整体频谱信息不同，小波变换能够对非平稳信号进行多尺度分解。其基本原理是利用一组具有伸缩和平移性质的小波基函数，对原始信号进行分解，从而在不同尺度上捕捉信号的局部特征。这种方法特别适合用于轴承故障信号的分析，因为这些信号通常具有时变的频率成分，且是非平稳的。小波变换的数学公式可以表示为：

$$\psi_{a,b}(t) = \int_{-\infty}^{\infty} x(t) \psi_{a,b}(t) dt = \int_{-\infty}^{\infty} x(t) |\alpha|^{1/2} \psi \left(\frac{t-b}{\alpha} \right) dt \quad (5.1)$$

其中， $f(a,b)$ 为连续小波变换系数， $\psi_{a,b}(t)$ 为小波基函数， a 和 b 分别为尺度因子和平移因子， R 为尺度空间。通过小波变换，我们可以得到信号在时频域中的局部特征，如图 5.3 所示，展示了不同故障类型下的二维时频图像。

本文对原始振动信号进行小波分解，提取其在多个尺度下的细节系数和近似系数，并将这些系数通过时频能量映射的方式转换为二维图像表示。二维图像不仅能够揭示不同时间段内信号频率成分的变化，还能凸显出故障特征在时频空间中的表现形式。与一维特征相比，二维图像化的数据更适合于深度学习模型的训练，尤其是 VGG 等能够直接对图像进行空间特征提取的模型。通过捕捉时频图像中的局部模式和整体分布，深度学习模型能够更好地理解故障信号的复杂特征，从而提升诊断的准确性和稳定性。

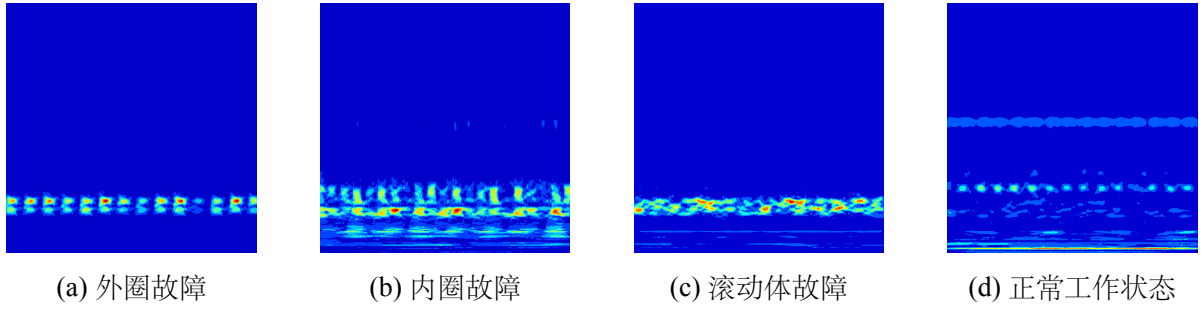


图 5.3 32kHz 采样频率下不同故障类别的二维时频图

5.3 模型建立

5.3.1 支持向量机 (Support Vector Machine, SVM)

SVM 是一种强大且广泛应用于模式识别和机器学习领域的监督学习算法，通过在特征空间中找到一个最优超平面来实现二分类和多分类任务。其基本原理是将训练数据映射到高维特征空间，然后在该空间中找到一个最优超平面，以最大化分类边界的间隔，原理如图 5.4 所示。

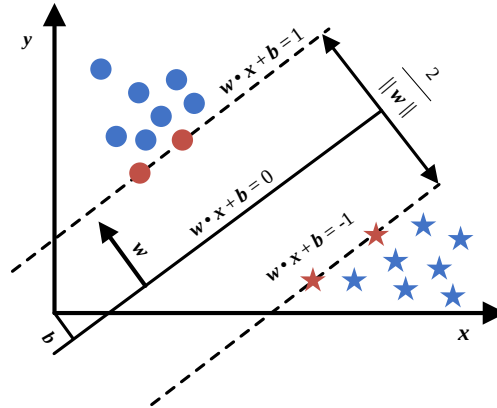


图 5.4 SVM 原理图

图中 $w \cdot x + b = 0$ 为分离超平面， $w \cdot x + b = 1$ 和 $w \cdot x + b = -1$ 中的 1 和 -1 为二分类的标签，是为了确保 $y_i(w \cdot x + b)$ 始终大于 0。支持向量机的最优问题可表示为：

$$\max_{w, b} \left(\min_{x_i} \left(\frac{y_i (w x_i + b)}{\|w\|} \right) \right) \quad (5.2)$$

令最近点到分离超平面的函数距离为 1，即 $\frac{y_i (w x_i + b)}{\|w\|} = 1$ ，则最优问题可转化为：

$$\min_{w, b} \frac{1}{2} \|w\|^2 \quad \text{s.t.}, y_i (w x_i + b) \geq 1 \quad (5.3)$$

建立融入约束条件后的拉格朗日公式：

$$L(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^n \alpha_i (y_i (w x_i + b) - 1) \quad \text{s.t.}, \alpha_i \geq 0 \quad (5.4)$$

可将最优问题转换为以下形式：

$$\min_{w,b} \left(\max_{\alpha_i \geq 0} (L(w, b, \alpha)) \right) \quad (5.5)$$

如满足 KKT 条件则需进一步求对偶问题：

$$\max_{\alpha_i \geq 0} \left(\min_{w,b} (L(w, b, \alpha)) \right) \quad (5.6)$$

当 w 和 b 取得最小值时可得：

$$w = \sum_{i=1}^n \alpha_i y_i x_i \quad \sum_{i=1}^n \alpha_i y_i = 0 \quad (5.7)$$

$$\max_{\alpha_i \geq 0} W(\alpha) = \max_{\alpha_i \geq 0} \left(\sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1,j=1}^n \alpha_i \alpha_j y_i y_j x_j x_i \right), \text{ s.t. } \sum_{i=1}^n \alpha_i y_i = 0 \quad (5.8)$$

最后通过对 a_i 求导得到 $W(\alpha)$ 的极大值。

5.3.2 多层感知机 (Multilayer Perceptron, MLP)

多层感知机 **MLP** 是一种前馈人工神经网络，由多个神经元（神经节点）组成，这些神经元按照层次结构排列，包括输入层、隐藏层和输出层，模型框架如图 5.5 所示。层与层之间的神经元通过权重连接，这些权重在模型训练过程中不断调整和优化，以使得模型能够学习到输入数据和输出目标之间的映射关系。**MLP** 的工作原理主要包括前向传播和反向传播两个过程。

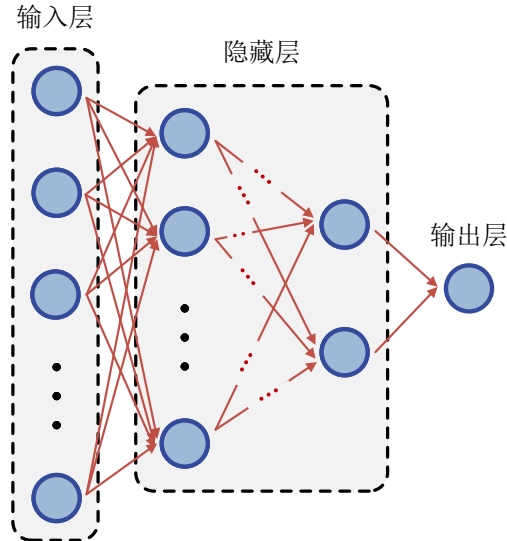


图 5.5 MLP 模型图

设第 l 层权重为 $W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ ，偏置为 $b^{(l)} \in \mathbb{R}^{n_l}$ ，非线性激活为 $f(\cdot)$ 。记输入 $a^{(0)} = x$ ，则第 l 层的线性-非线性变换为：

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)}) \quad (5.9)$$

输出层采用 Softmax 得到每一类的后验概率：

$$\hat{y} = \text{Softmax} (z^{(L)}) = \frac{e^{z_k^{(L)}}}{\sum_{j=1}^K e^{z_j^{(L)}}}. \quad (5.10)$$

利用链式法则最小化损失 $\mathcal{L} = -\sum y_i \log a_i^{(L)}$ ，输出层梯度可表示为：

$$\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial z^{(L)}} = \hat{y} - y \quad (5.11)$$

对任一隐藏层 l ，有：

$$\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial z^{(l)}} = (W^{(l+1)\top} \delta^{(l+1)}) \odot f'(z^{(l)}) \quad (5.12)$$

进而得到参数梯度：

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} a^{(l-1)\top}, \quad \frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)} \quad (5.13)$$

以优化器更新参数 $\theta = W^{(l)}, b^{(l)}$ ，其中 η 为学习率：

$$\theta \leftarrow \theta - \eta, \nabla_{\theta} \mathcal{L} \quad (5.14)$$

5.3.3 VGG (Visual Geometry Group) 模型

VGG-16 是一种经典卷积神经网络，其显著特点是使用多个 3×3 卷积核堆叠，在保证感受野扩大的同时减少参数数量，提升网络深度和表达能力。如图 5.6 所示，VGG-16 总共包含 16 个有权重的层，其中包括 13 个卷积层和 3 个全连接层，具体配置见表 5.1。

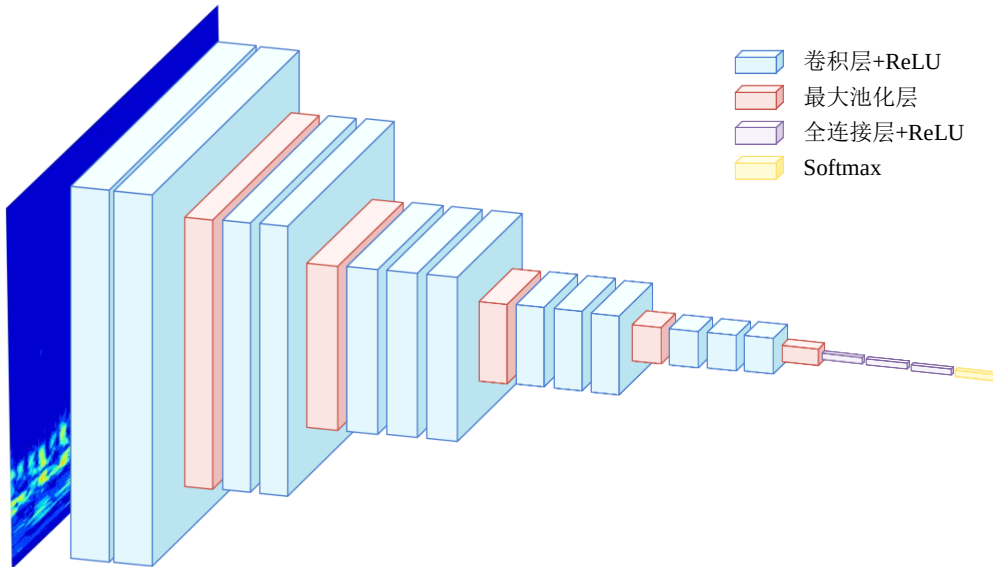


图 5.6 VGG 模型图

相比早期采用大卷积核的网络，VGG 的小卷积核叠加策略不仅提升了模型的非线性表达能力，还增强了网络对复杂模式的适应性。不仅如此，VGG-16 的结构还具有良好的

表 5.1 VGG-16 网络结构配置

Block	卷积层配置	池化层
1	Conv(64, 3×3) $\times 2$	MaxPool(2×2)
2	Conv(128, 3×3) $\times 2$	MaxPool(2×2)
3	Conv(256, 3×3) $\times 3$	MaxPool(2×2)
4	Conv(512, 3×3) $\times 3$	MaxPool(2×2)
5	Conv(512, 3×3) $\times 3$	MaxPool(2×2)
FC	4096 $\rightarrow$ 4096 $\rightarrow$ 4096 $\rightarrow$ softmax	

层次性和可移植性。由于各卷积区块的设计模式高度一致，整个网络框架非常规则，这使得其易于在不同任务中迁移和复用。在本研究中，VGG-16 可以直接作为特征提取器，将二维时频图像中的局部时频模式和全局分布特征逐层提炼出来，从而为轴承故障诊断提供深层次的表示能力。

5.4 实验设置与评价指标

在实验设计方面，考虑到数据集规模及模型训练的稳定性，本文采用五折交叉验证的方法对各类模型进行评估。在五折交叉验证中，原始数据集被随机划分为五个大小相等的子集，其中每一次实验都会选择其中一折作为测试集，其余四折作为训练集。为避免因类别分布不均衡而导致的训练偏差，在随机划分数据集时，本文采用了分层抽样（Stratified Sampling）的方式，即保证每一折中的样本类别比例与整体数据集保持一致。整个过程将循环五次，确保每一折数据都恰好作为一次测试集。最终的实验结果取五次实验的平均值，以此减小因数据划分带来的偶然性影响，从而更加客观地评估模型性能。

在评价指标的选取上，本文从多角度对模型的诊断性能进行量化，采用了以下几类主流评估方式。

1. **准确率（Accuracy）** 是分类模型中最直观的评价指标，表示正例和负例中预测正确数量占总数量的比例，计算公式如下：

$$\text{准确率} = \frac{TP + TN}{TP + TN + FP + FN} \quad (5.15)$$

2. **精确率（Precision）** 以预测结果为判断依据，表示预测为正例的样本中预测正确的比例。可以衡量的是模型对正类预测的准确性，计算公示如下：

$$\text{精确率} = \frac{TP}{TP + FP} \quad (5.16)$$

3. **召回率（Recall）** 以实际样本为判断依据，表示正确识别出的正类样本占有所有实际正类

样本的比例。可以衡量模型对正类的覆盖能力，计算公式如下：

$$\text{召回率} = \frac{TP}{TP + FN} \quad (5.17)$$

4. **F1 分数**是精确率和召回率的调和平均数，用于平衡精确率和召回率。它是一个综合的评价指标，特别适用于数据集不平衡或者错误类型成本不均等的情况，计算公式如下：

$$\text{F1 Score} = 2 \cdot \frac{\text{精确率} \cdot \text{召回率}}{\text{精确率} + \text{召回率}} \quad (5.18)$$

5. **AUC 曲线 (Area Under the ROC Curve)** 是指 ROC 曲线下的面积，表示模型区分正负类样本的能力，是所有可能的分类阈值的效果的综合衡量。AUC 取值范围在是 $[0, 1]$ ROC 曲线描绘的是假正例率和真正例率之间的关系，其中真正例率 $\text{TRP} = \frac{TP}{TP+FN}$ ，假正例率 $\text{FRP} = \frac{FP}{FP+TN}$ 。以假正例率为横坐标、真正例率为纵坐标即可得到 ROC 曲线。
6. **混淆矩阵**是一个二维误差矩阵，用于评估分类模型性能。能够非常直观地显示分类模型在数据集上的预测结果与实际值之间的关系，能够全面展示模型的性能。

其中，TP：真阳性，即模型正确预测为正的样本数。TN：真阴性，即模型正确预测为负的样本数。FP：假阳性，即模型错误预测为正的样本数。FN：假阴性，即模型错误预测为负的样本数。

5.5 实验结果分析与可视化

如表 5.2 所示，所选模型在各个评价指标上都达到了很好的预期效果：每种模型在每种指标上都达到了 98% 以上，尤其是多层感知机 (MLP) 模型和 VGG 模型，在各个指标上都达到了 99% 以上。可见在问题 1 中使用三种方法选出的 14 个特征具有很好的代表性，与轴承故障密切相关，进一步说明了所选特征的正确性和有效性。不同于机器学习模型只对一维特征进行提取，VGG 模型是根据二维时频图像，通过卷积神经网络对二维时频图像进行多次卷积，完成对数据特征进行深度提取，得到更加丰富的局部特征和全局特征。此外，作为一种深度神经网络，VGG 在大规模数据集上进行训练，能够学习到更深层更具有普适性的特征，且模型结构简单易于修改，因此 VGG 具有良好的迁移学习能力，通过超参数调节可以在新的相似任务上取得优秀的表现。

如图 5.7 所示，可以得出三种机器学习模型和两种深度学习模型在轴承故障诊断上都有很好的结果：SVM 模型和 CNN 模型在滚动体故障 (B) 和外圈故障 (OR) 类别出现了错误分类；RF 模型和 MLP 模型在滚动体故障 (B) 和内圈故障 (IR) 类别出现了错误分类；VGG 模型仅在内圈故障 (IR) 类别上出现了错误分类。总体来说，MLP 模型和 VGG 模型在轴承故障诊断上表现最好，说明这些特征在故障诊断中有重要的区分能力。

如图 5.8 和 5.9 所示，VGG 和 CNN 的训练准确率迅速上升然后逐渐趋向于 1，表明两个模型能够有效的判断轴承的运动状态，在训练集上有较强的拟合能力；而训练损失在一开始就迅速下降然后逐渐趋向稳定，这表明两个模型在训练过程中，损失函数得到优化，

表 5.2 不同模型的诊断结果

类别	方法	Acc	Pre	Recall	F1	AUC
机器学习	SVM	98.11	98.12	98.11	98.11	99.87
	RF	98.86	98.62	98.62	98.62	99.97
	MLP	99.66	99.65	99.65	99.65	99.94
深度学习	CNN	98.26	98.29	98.26	98.27	99.90
	VGG	99.14	99.14	99.14	99.14	99.93

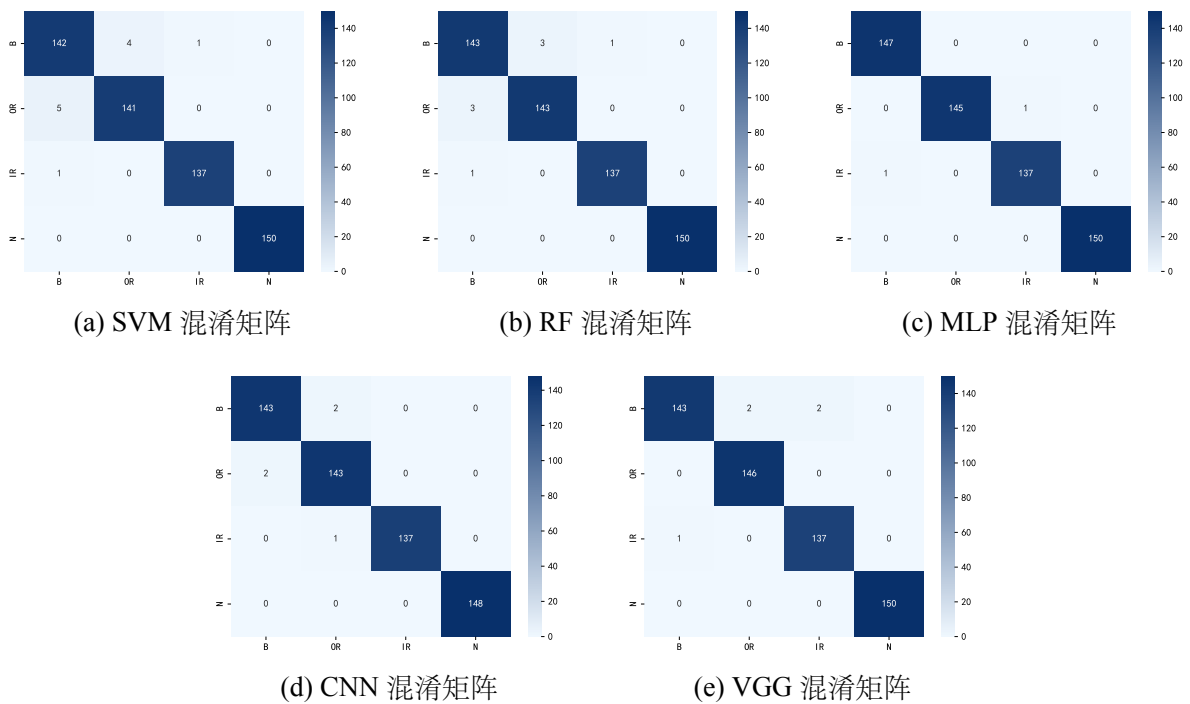
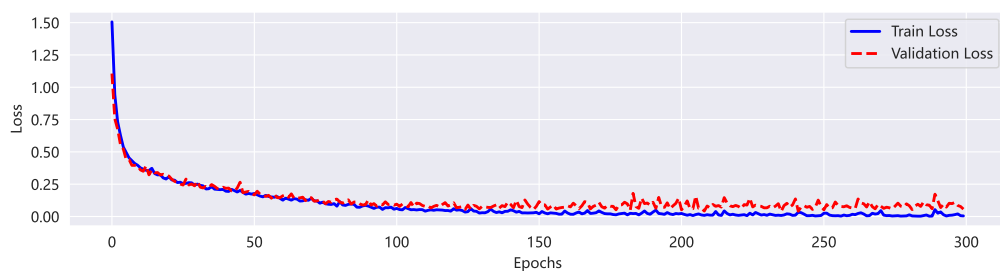


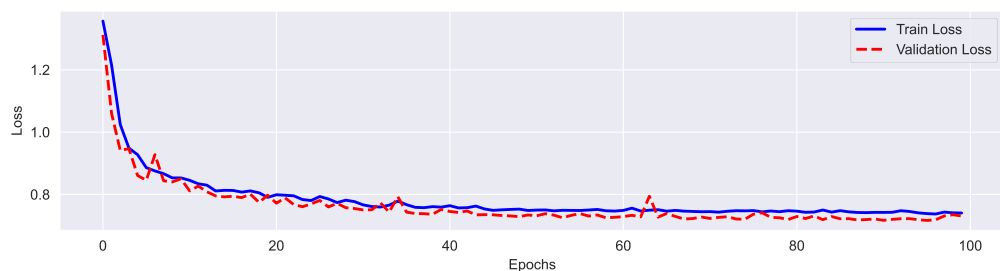
图 5.7 混淆矩阵

逐渐趋向稳定。验证集准确率曲线和损失曲线与训练集相似，但是在前期出现了一些波动，可能与前期模型在验证集上尚未稳定有关。到后期两条曲线几乎平行，表明两个模型逐渐稳定并获得了很好的泛化能力。整体来说，训练集和验证集的准确率接近，两个模型均未出现过拟合。

为了评估 VGG 在轴承故障上的分类效果，可采用 t-SNE[11] 将数据集中分类节点映射在二维空间。如图 5.10 所示，可以明显看到数据集被分为四块，对应轴承的四种状态。其中绿的模块（滚动体故障）、红色模块（内圈故障）、蓝色模块（外圈故障）和黄色模块（正常状态）均没有与其他模块重合，VGG 在轴承故障上展现出了优秀的分类能力，进一步说明了 VGG 在时频域上学习到的深层特征具有很好的辨别能力，与轴承状态密切相关。

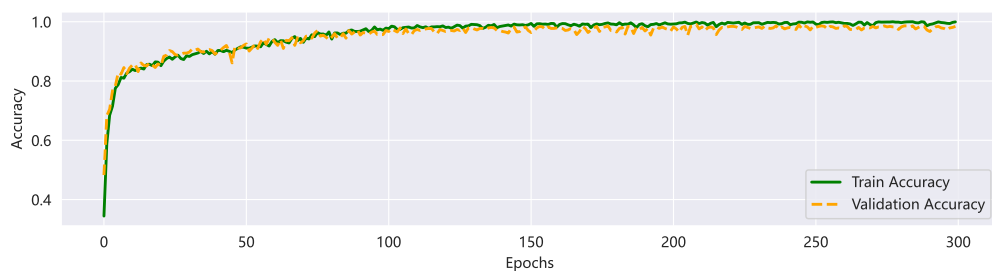


(a) CNN

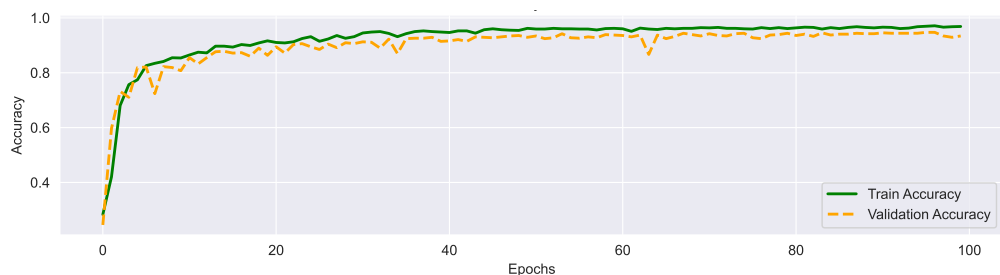


(b) VGG

图 5.8 Loss 曲线



(a) CNN



(b) VGG

图 5.9 Acc 曲线

5.6 本章小结

本章基于源域数据构建了两类特征输入，一类为问题一中筛选得到的**时域、频域、时频域及包络谱特征**，另一类为**通过小波变换生成的二维时频图像**。针对第一类特征，根据第一问中多种特征选择方法得到频谱均值、包络标准差、包络最大值等 14 个最具有代表

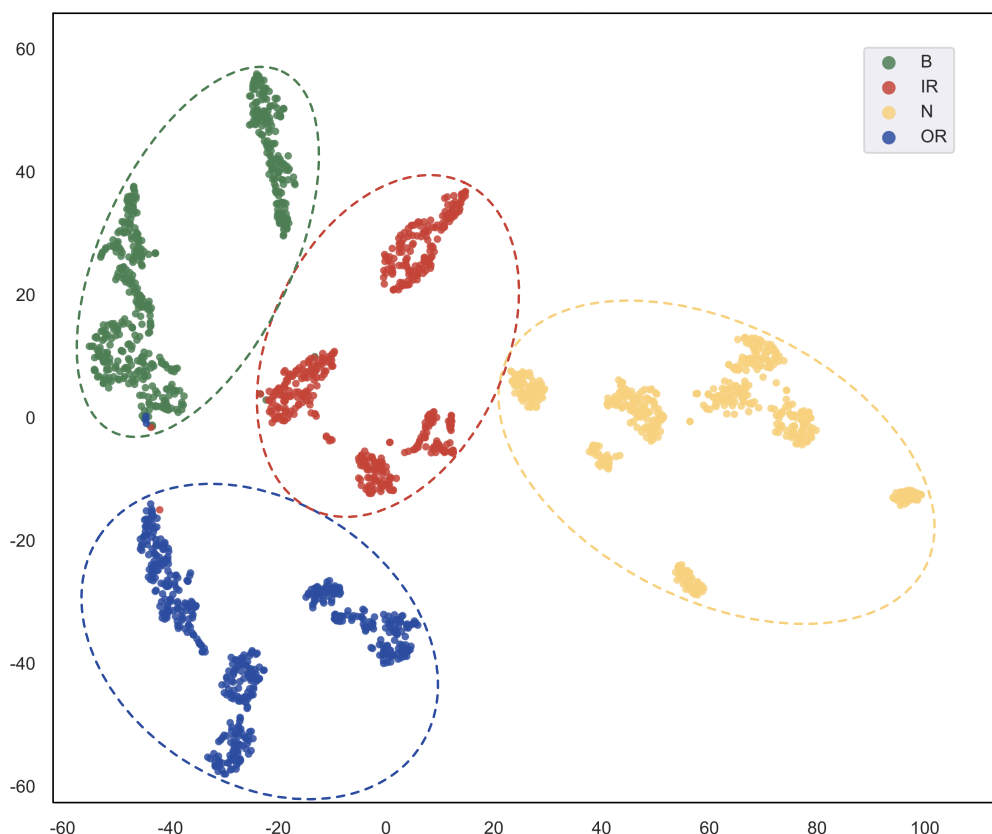


图 5.10 VGG 的 t-SNE 可视化

性的关键特征。针对第二类特征，通过小波变换得到包含时频信息的二维图像以供深度学习模型深入挖掘潜在特征。在模型设计方面，对第一类特征采用**支持向量机 (SVM)**、**随机森林 (RF)** 和**多层感知机 (MLP)** 三种机器学习方法进行诊断。对于二维时频图像则采用**VGG 与卷积神经网络 (CNN)** 进行特征提取与分类任务。为确保模型评估的可靠性，将源域数据集平均划分为五个子集，采用**五折交叉验证**的方式进行训练与测试。经过**准确率 (ACC)**、**精确率 (Precision)**、**召回率 (Recall)**、**F1 值 (F1-Score)** 与 **AUC** 等多项评估指标检验，并通过**混淆矩阵**和 **Acc 曲线**对分类效果进行可视化分析，表 5.2 中结果表明，五类模型均表现出良好的分类性能，尤其是 **MLP 模型**，达到了 **99.6% 的最佳准确率**，这表明特征工程在机器学习中的重要性。通过精心筛选的手工特征，机器学习能够实现优秀的效果。然而，深度学习方法也能取得接近的性能，而且不依赖于特征工程。仅使用基础的震动特征或时频域特征，深度学习便能取得同样出色的诊断效果。尤其是在数据集增大的场景下，深度学习能够自动挖掘复杂的特征，其**强大的特征学习能力**使得效率远超传统机器学习方法。由此可见，深度学习模型能够自动捕捉**更深层次的判别性特征**，展现出更强的适应性和鲁棒性，尤其是在处理复杂故障模式时。本章的研究为后续跨域迁移学习提供了有效的模型选择参考和理论支撑。

6 问题三求解及分析

6.1 问题分析

轴承故障诊断的任务常常面临源域与目标域之间的共性与差异，传统的方法难以应对这些差异。而**域迁移学习**能够克服这种挑战，使得在源域中学到的知识能够有效应用于目标域。这种迁移方法能够有效利用源域中丰富的**标注数据**，在目标域中提供较为准确的故障诊断。尤其在轴承故障诊断中，源域和目标域的差异（如设备类型、工况差异等）往往对传统算法的诊断性能造成显著影响，而**迁移学习能够有效缩小源域与目标域之间的知识差距**，提高在目标域中的诊断效果。

在本研究中，基于问题二中获得的深度学习模型，本文决定在迁移学习框架下进一步提升轴承故障诊断的性能。**卷积神经网络 CNN** 在特征学习方面具有显著优势，能够同时从时域和频域中提取有效特征，这使得 CNN 在**处理复杂故障模式和高维数据**时表现出了优异的性能。然而，源域和目标域之间的分布差异会影响模型的迁移效果。为了解决这一问题，本文采用了**对齐决策边界**的方法 [12]，以提高域迁移的性能，如图 6.1 所示。对齐决策边界的核心思想是通过调整源域和目标域数据的特征分布，使得源域和目标域共享更多的特征信息，从而使得模型能够在目标域中获得更好的分类性能。具体而言，这一策略通过对**源域和目标域的决策边界进行对齐**，使得模型能够减少由于分布差异带来的性能下降，更好地适应目标域中可能出现的变化，提升在目标域中的识别能力。

综上，本文通过首先在源域上训练 CNN 模型，再通过对齐决策边界来适应目标域特征，最终完成目标域数据的分类检测。这一方法通过利用目标域的无标签数据进行微调，有效地提升了模型在目标域中的诊断性能，并且通过可视化展示了迁移结果，为轴承故障诊断的跨域应用提供了理论和实践基础。

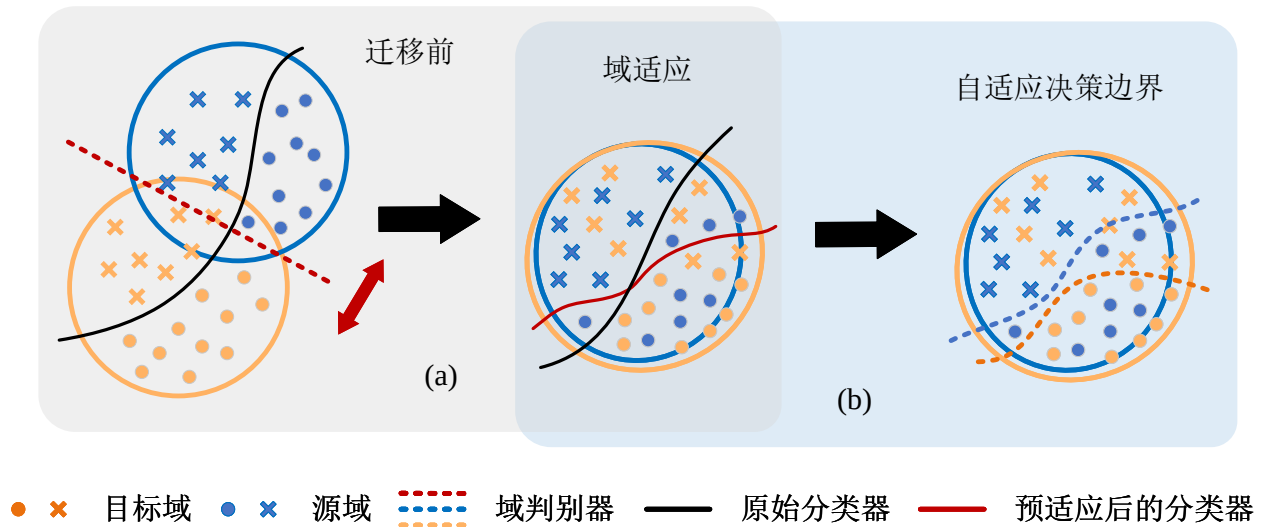


图 6.1 决策边界对齐示意图

6.2 域适应 (Domain Adaptation, DA)

域适应 (Domain Adaptation, DA) 是迁移学习的一种形式, 旨在将从一个领域 (源域) 学到的知识迁移到另一个领域 (目标域), 以应对目标域样本不足或无标签的情况。这是由于在实际应用中, 标注数据昂贵或者难以获得, 而无标签的目标领域数据却非常丰富。为此, 域适应提供了一种有效的方案, 有助于对目标域进行精准预测。

域适应任务中主要分为以下两个领域:

- **源域 (Source Domain, $\mathcal{D}_S$)**: 有标签数据的领域, 包含特征 X_S 和标签 Y_S 。
- **目标域 (Target Domain, $\mathcal{D}_T$)**: 无标签数据的领域, 只有特征 X_T 。

域适应中的学习任务通常是通过源域的标注数据来训练分类器, 目标是使得学习到的模型能够有效地应用于目标域。具体来说, 给定源域的训练数据和目标域是无标签数据, 我们希望学习到一个映射 $f(X)$, 使得它能够根据目标域数据做出准确的预测。

域适应中的一个关键问题是如何使源域和目标域的分布尽可能相似。联合分布适应 (Joint Distribution Adaptation, JDA) 结合了两种主要的方法:

- **边际分布适应 (MDA)**: 要求源领域和目标领域的特征分布相同, 即 $P(X_S) = P(X_T)$ 。
- **条件分布适应 (CDA)**: 要求源领域和目标领域中每个类别的样本具有相同的条件概率分布, 即 $P(Y_S|X_S) = P(Y_T|X_T)$ 。

其中, P 为特征空间的边际概率分布。为了减少这两种分布之间的差异, 联合分布适应通过优化目标函数来学习一个特征表示 T , 使得源领域和目标领域的边际分布和条件分布尽可能相似。目标函数可以表示为:

$$L_{JDA} = \left\| \mathbb{E}_{P(X_S)} [T(X_S)] - \mathbb{E}_{P(X_T)} [T(X_T)] \right\|^2 + \sum_{c=1}^C \left\| \mathbb{E}_{P(X_S|Y_S=c)} [T(X_S) | Y_S = c] - \mathbb{E}_{P(X_T|Y_T=c)} [T(X_T) | Y_T = c] \right\|^2 \quad (6.1)$$

其中, C 表示类别数量, $\mathbb{E}$ 表示期望值。目标函数的第一项优化源域和目标域的特征分布相似度, 第二项优化源域和目标域在每个类别的条件分布相似度。

6.3 基于决策边界优化的迁移学习模型

根据问题二实验结果可见, 在分别使用机器学习和深度学习方法对源域数据集进行分类实验后, 深度学习方法, 在特征提取和分类准确性上表现出较优的性能。机器学习方法虽然也能实现分类, 但往往难以充分挖掘数据中的深层次特征, 而深度学习方法, 凭借其强大的特征学习能力, 能够有效地从数据中提取高维特征, 特别适用于处理复杂和大规模数据集。由于 CNN 相较于 VGG 在计算资源消耗和模型训练时间上更为高效, 且对于我们的任务而言, CNN 能够提供足够的特征提取能力, 因此本文最终选择具有强大的特征学习能力的卷积神经网络作为特征提取器。本文所采用的模型框架如图 6.2 所示。通过五个

Conv1D 模块、一个全局平均池化层（GAP）和两个全连接层（FC）来提取源域和目标域的共享特征。每个 Conv1D 模块由卷积层、批量归一化（BN）层和最大池化层组成，这种设计不仅有助于加速网络的收敛，还能缓解过拟合问题，具体参数设置见表 6.1。通过这种结构设计使源域和目标域之间能够进行高效的知识迁移，提升目标域的预测准确性。

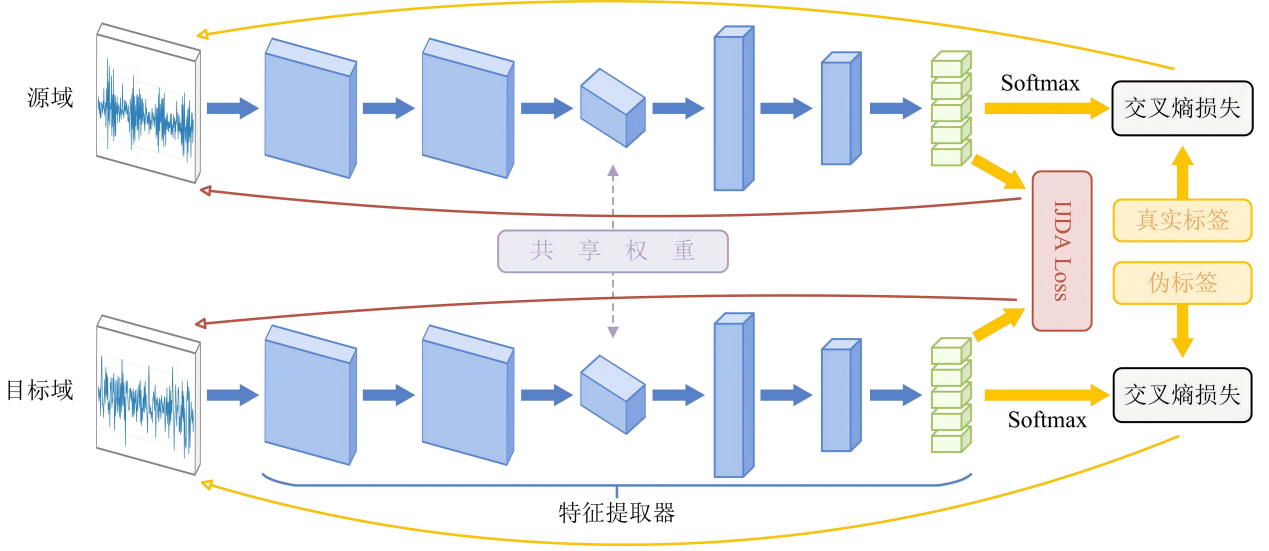


图 6.2 基于决策边界优化的迁移学习模型框架

为了克服原始 CDA 中存在的近似误差，本文采用了优化后的 CDA 机制，其主要目的是对源域和目标域的条件概率分布进行对齐，具体公式如下：

$$P(Y = c | X) = \frac{P(Y = c) \cdot P(X | Y = c)}{P(X)} \quad (6.2)$$

其中， $P(Y = c)$ 表示类的先验概率， $P(X|Y = c)$ 表示类条件概率分布。改进的 CDA 机制优化了条件概率分布的对齐，目标函数表示为：

$$L_{CDA} = \sum_{c=1}^C \left\| \mathbb{E}_{P(X_S|Y_S=c)} [T(X_S) | Y_S = c] P(Y_S = c) - \mathbb{E}_{P(X_T|Y_T=c)} [T(X_T) | Y_T = c] P(Y_T = c) \right\|^2 \quad (6.3)$$

该损失函数旨在减小源域和目标域之间类条件分布的差异。结合了 MDA 和 CDA 的目标，最终形成了 IJDA 目标函数，表示为：

$$L_{IJDA} = \left\| \mathbb{E}_{P(X_S)} [T(X_S)] - \mathbb{E}_{P(X_T)} [T(X_T)] \right\|^2 + \sum_{c=1}^C \left\| \mathbb{E}_{P(X_S|Y_S=c)} [T(X_S) | Y_S = c] P(Y_S = c) - \mathbb{E}_{P(X_T|Y_T=c)} [T(X_T) | Y_T = c] P(Y_T = c) \right\|^2 \quad (6.4)$$

此外，为评估源域和目标域边际分布的差异，本文还引入了新的分布距离度量 DDM (Domain Distribution Metric)，进一步提升域适应的性能，公式如下：

$$DDM(A, B) = MMD(A, B) + CORAL(A, B) \quad (6.5)$$

表 6.1 模型参数设置

	层类型	大小/步长/通道	输入尺寸	输出尺寸
Conv1D-1	卷积	64/16/32	(4096, 1)	(192, 32)
	批归一化	/	(192, 32)	(192, 32)
	最大池化	2/2/32	(192, 32)	(96, 32)
Conv1D-2	卷积	3/1/64	(96, 32)	(96, 64)
	批归一化	/	(96, 64)	(96, 64)
	最大池化	2/2/32	(96, 64)	(48, 64)
Conv1D-3	卷积	3/1/128	(48, 64)	(48, 128)
	批归一化	/	(48, 128)	(48, 128)
	最大池化	2/2/128	(48, 128)	(24, 128)
Conv1D-4	卷积	3/1/256	(24, 128)	(24, 256)
	批归一化	/	(24, 256)	(24, 256)
	最大池化	2/2/256	(24, 256)	(12, 256)
Conv1D-5	卷积	3/1/512	(12, 256)	(12, 512)
	批归一化	/	(12, 512)	(12, 512)
	最大池化	2/2/512	(12, 512)	(6, 512)
	全局平均池化	/	(6,512)	512
	1th 全连接层	1*/512	512	512
	2th 全连接层	1*/4	512	4

该度量结合了最大均值差异 (MMD) 和类间相关 (CORAL) 度量来进一步减少域混淆, 提高模型的鲁棒性。最终的目标函数被定义为:

$$L_{IJDA} = DDM(T(X_S), T(X_T)) + \sum_{c=1}^C DDM[P(Y_S = c) \cdot (T(X_S) | Y_S = c), P(Y_T = c) \cdot (T(X_T) | Y_T = c)] \quad (6.6)$$

在多分类任务中, 传统的 Softmax 损失函数虽然有概率解释和计算简单的优点, 但在某些情况下, 无法满足类内紧凑性 (intra-class compactness) 和类间可分离性 (inter-class separability) 的要求。因此, 本文采用了改进后的 Softmax 损失, 旨在学习更可分的特征并提高迁移任务的性能。改进后的 Softmax 损失可表示为:

$$L_y = \begin{cases} -\frac{1}{n} \sum_{i=1}^n \log \left(\frac{e^{F^i(c)/m-k}}{e^{F^i(c)/m-k} + \sum_{j \neq c} e^{F^i(j)}} \right), & F^i(c) > 0 \\ -\frac{1}{n} \sum_{i=1}^n \log \left(\frac{e^{mF^i(c)-k}}{e^{mF^i(c)-k} + \sum_{j \neq c} e^{F^i(j)}} \right), & F^i(c) \leq 0 \end{cases} \quad (6.7)$$

其中, F_i 代表通过特征提取器输出的特征向量, $F^i(c) = W_c^T Z^i = \|W_c\| \|Z^i\| \cos(\theta_c)$ 和 $F^{(i)}(j)$ 分别表示 F_i 中的第 c 个元素和其他元素, c 为标签, $k \geq 0$ 和 $m \geq 1$ 是控制决策边界的超参数。

为了提高决策边界的严格性, Softmax 损失要求在训练过程中遵循以下不等式:

$$\begin{cases} W_1^T Z > W_1^T Z/m - k > W_2^T Z, W_1^T Z > 0 \\ W_1^T Z > W_1^T Z/m - k > W_2^T Z, W_1^T Z \leq 0 \end{cases} \quad (6.8)$$

这表明通过调整权重和特征之间的关系,改进后的 Softmax 损失能够缓解传统 Softmax 损失中决策边界不严格的问题,提升类间可分性和类内紧凑性。为了进一步优化特征学习,使得模型更易于学习到可分离的特征,本文对权重向量进行 L2 范数归一化,公式如下:

$$\widehat{W}_c = \frac{W_c}{\|W_c\|_2} \quad (6.9)$$

这种归一化确保了权重向量的长度一致,从而避免了过大或过小的权重影响模型的学习稳定性,并且帮助模型学习到更加平衡且易于区分的特征。最终 Softmax 损失可以表示为如下公式:

$$L_y = \begin{cases} -\frac{1}{n} \sum_{i=1}^n \log \left(\frac{e^{\widehat{W}_c^T Z^i / m - k}}{e^{\widehat{W}_c^T Z^i / m - k} + \sum_{j \neq c} e^{\widehat{W}_j^T Z^i}} \right), \widehat{W}_c^T Z^i > 0 \\ -\frac{1}{n} \sum_{i=1}^n \log \left(\frac{e^{m \widehat{W}_c^T Z^i - k}}{e^{m \widehat{W}_c^T Z^i - k} + \sum_{j \neq c} e^{\widehat{W}_j^T Z^i}} \right), \widehat{W}_c^T Z^i \leq 0 \end{cases} \quad (6.10)$$

改进后的 Softmax 损失不仅能够增强模型对目标域数据的适应性,还能够提高决策边界的严格性,从而实现更强的类间分离能力。

6.4 结果展示

表 6.2 基于迁移学习的预测结果

样本	类型	样本	类型
A	滚动体故障 B	I	内圈故障 IR
B	正常工作状态 N	J	内圈故障 IR
C	内圈故障 IR	K	外圈故障 OR
D	滚动体故障 B	L	滚动体故障 B
E	内圈故障 IR	M	外圈故障 OR
F	滚动体故障 B	N	外圈故障 OR
G	正常工作状态 N	O	正常工作状态 N
H	外圈故障 OR	P	滚动体故障 B

基于迁移学习模型得到的目标域类别如表 6.2 所示。其中，目标域共有 4 个外圈故障 (OR)、4 个内圈故障 (IR)、5 个滚动体故障 (B) 和 3 个正常工作状态 (N)。

如图 6.3-6.6 所示，左上子图是原始信号与重构信号的对比图；右上子图是自带包络超峰值图；左下子图是重构信号的包络时域；右下子图是重构信号的包络谱。对于 H、K、M 和 N 来说重构时域图符合外圈故障振动信号呈周期性冲击，且包络信号变化相对剧烈，说明信号中存在周期性波动；从频谱中可以得出，信号有明显能量集中在 100Hz 到 500Hz 的频段，这些频率与外圈故障有关，判断为发生了外圈故障。对 C、I 和 J 来说重构时域图符合内圈故障振动信号呈周期性冲击；包络波形出现了剧烈波动，可能与内圈故障有关。判断为发生了内圈故障。对于 A、D、F、L 和 P 来说重构时域图符合滚动体故障振动信号没有明显周期性；频域中显示信号在 100Hz 到 500Hz 频段能力集中，是轴承故障的典型特征，可能是发生了滚动体故障小波变换图进一步说明了能量集中在高频段，表明信号中存在局部缺陷，判断为发生了滚动体故障。对于 B、G 和 O 来说重构时域图符合正常运动的振动信号，且中低频端能量集中，判断为正常状态。

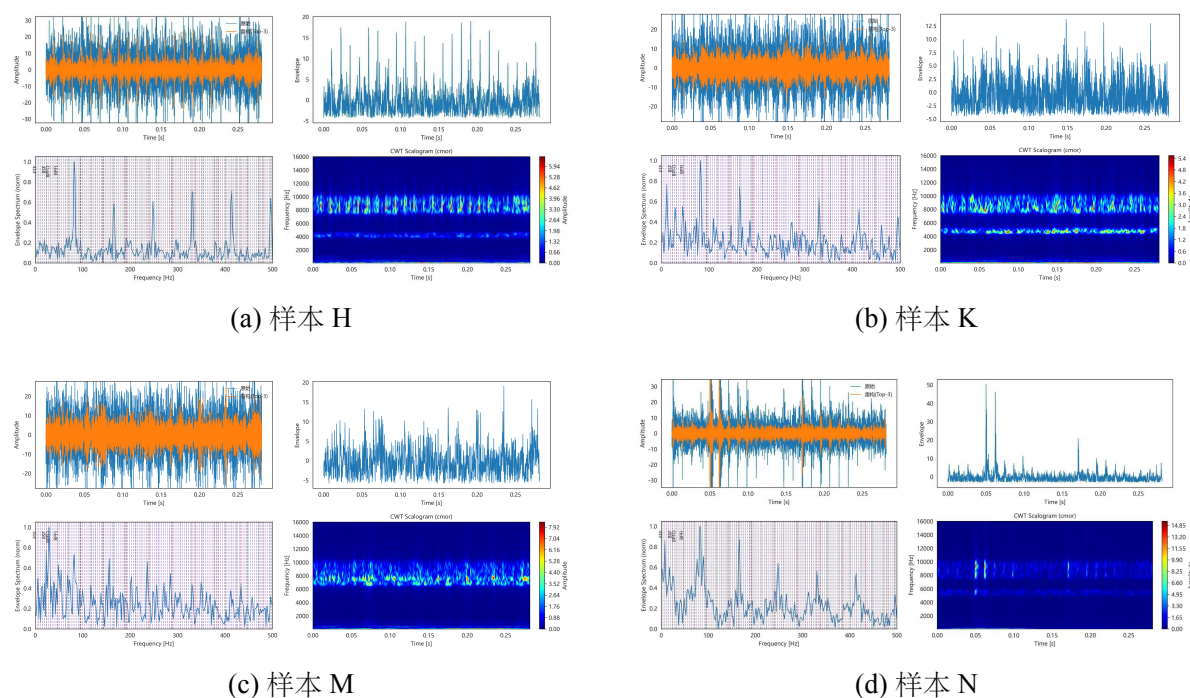


图 6.3 预测为外圈故障 OR 的样本特征可视化

6.5 本章小结

在源域与目标域之间常常存在较大的数据分布差异，这使得传统模型在目标域上的泛化能力不足，进而影响了故障诊断的准确性。因此，采用域适应方法能够有效地将源域中的知识迁移到目标域，从而提升目标域数据的诊断性能。基于这一背景，本研究选择了卷

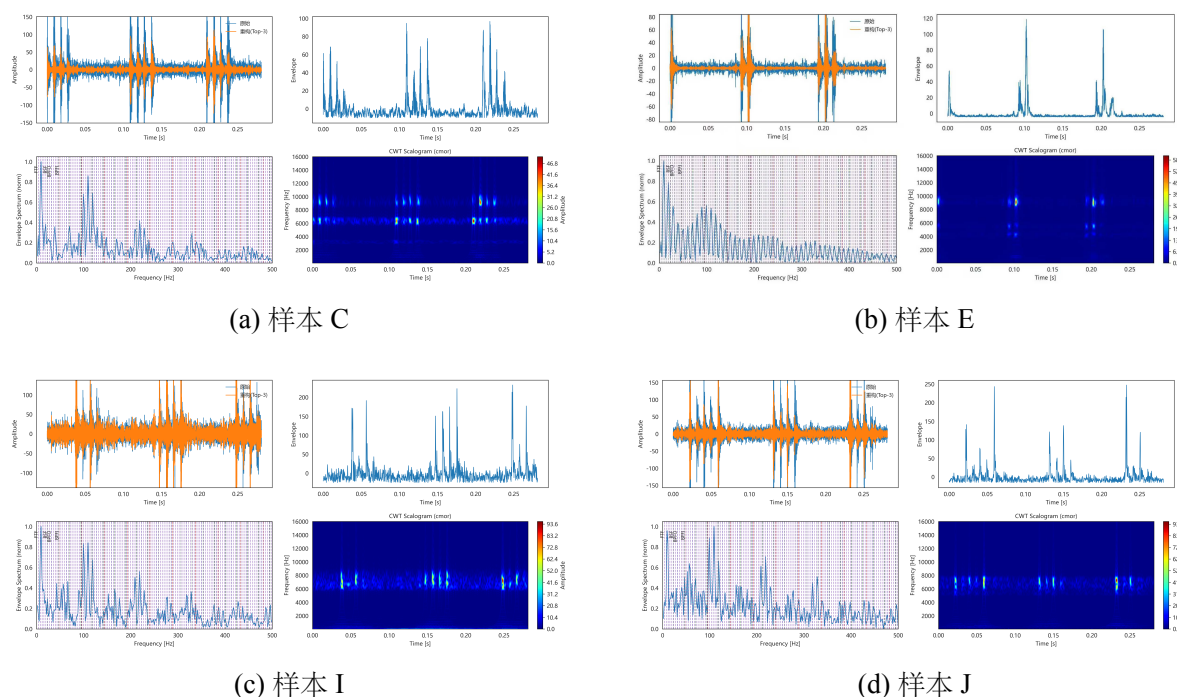
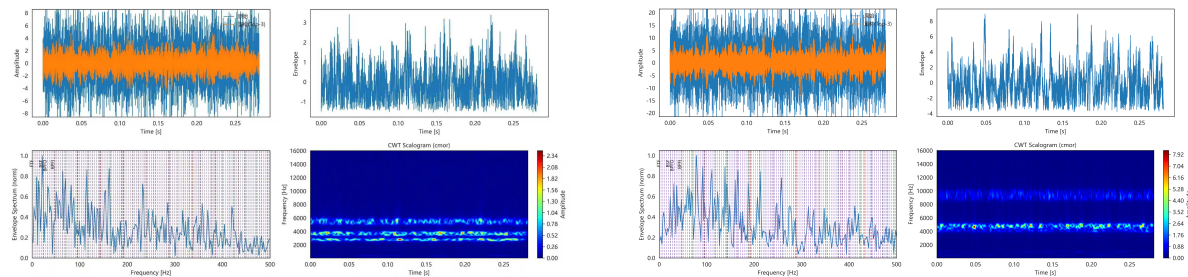


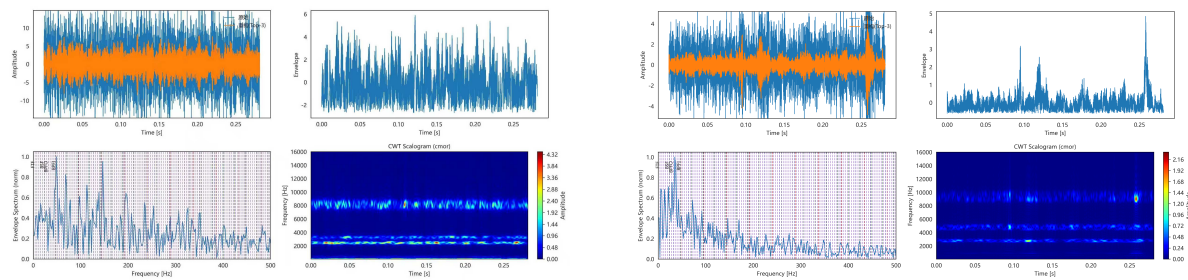
图 6.4 预测为内圈故障 IR 的样本特征可视化

积神经网络 (CNN) 作为迁移学习的基线模型。CNN 能够通过多层卷积操作自动提取时域和频域特征，在处理复杂故障模式时表现出了良好的性能。然而，由于源域和目标域之间的特征分布差异，模型在目标域的表现往往会受到影响。因此，为了提升迁移学习的效果，本文引入了**对齐决策边界策略**，通过优化源域和目标域的决策边界，使得模型能够更好地将源域中学到的特征迁移到目标域，从而提高分类性能。此外，本文还进一步引入了**分布距离度量 (DDM)**，作为优化后的 **CDA 机制**，以进一步减少源域和目标域之间的分布差异。这一机制能够有效提升模型的域适应能力，增强其在目标域中的**鲁棒性和准确性**。通过这种方法，迁移学习模型能够在目标域中实现较高的分类精度和较强的泛化能力。最终，迁移学习模型成功地对目标域数据进行了分类，识别出了**4 个外圈故障 (OR)**、**4 个内圈故障 (IR)**、**5 个滚动体故障 (B)** 和 **3 个正常工作状态 (N)**，如表 6.2 所示。对目标域数据的可视化结果显示，样本特征在各类别间具有较好的区分性，且类间差异较小，进一步验证了迁移学习模型在目标域中的有效性和可靠性。



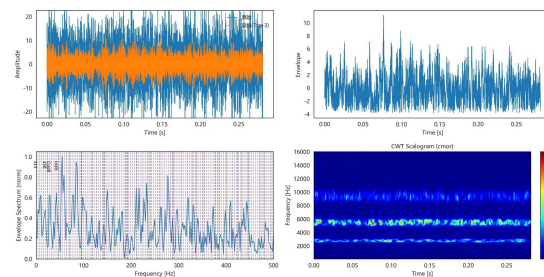
(a) 样本 A

(b) 样本 D



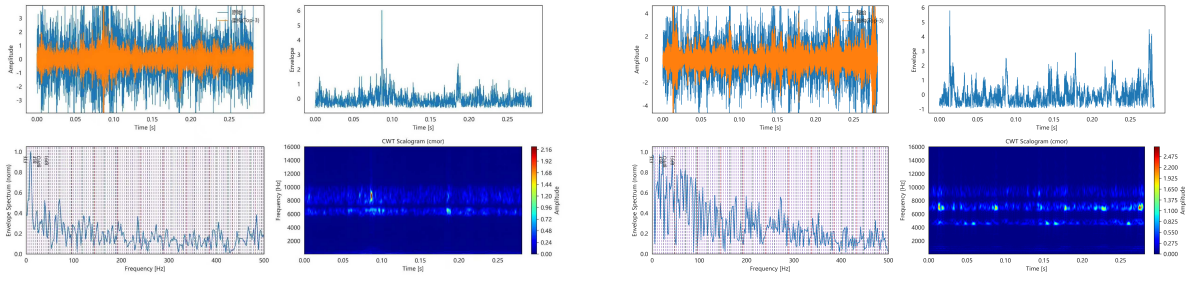
(c) 样本 F

(d) 样本 L



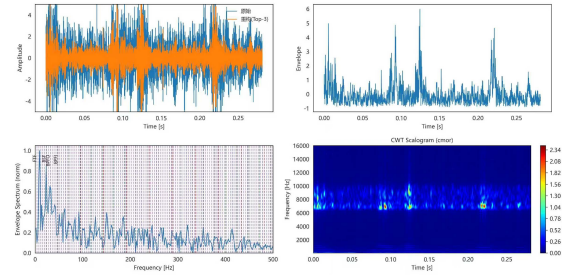
(e) 样本 P

图 6.5 预测为滚动体故障 B 的样本特征可视化



(a) 样本 B

(b) 样本 G



(c) 样本 O

图 6.6 预测为正常工作状态 N 的样本特征可视化

7 问题四求解及分析

7.1 问题分析

在迁移学习应用于跨工况、跨设备的故障诊断任务中，**模型的可解释性**是一个亟待解决的重要问题。**机器学习模型通常被视为“黑箱”**，其决策过程和迁移过程往往难以被直观理解，这不仅导致诊断人员对模型结果缺乏信任，还可能影响模型的实际应用效果。因此，提升迁移诊断的可解释性，帮助用户理解模型在迁移学习过程中的决策逻辑与特征转移，对于故障诊断的准确性与信任度至关重要。本文的研究重点之一是提高迁移诊断过程中的可解释性，尤其是问题三中的迁移学习模型。迁移学习模型在不同设备和工况之间进行知识迁移，其**透明度问题**尤为突出。通过深入分析**模型结构设计、迁移过程以及决策机制**，本研究结合轴承故障的物理机理和故障特征，从**迁移过程**进行可解释性分析，以增强模型的透明度。

在迁移学习过程中，如何确保源域的特征能够有效地传递至目标域，且迁移后的特征依然具有诊断价值，是一个关键问题。为了解决这个问题，本文基于类条件**最大均值差 (MMD)** 和**相关对齐度量 (CORAL)** 对源域与目标域之间的分布对齐情况进行分析，并通过 **t-SNE** 展示迁移过程中**类别间的对齐情况**。这些可视化结果帮助我们理解不同类别在迁移过程中面临的挑战，揭示了哪些类别在源域和目标域之间难以对齐，进一步说明模型如何适应新的工况和设备。

综上所述，**迁移学习中的可解释性分析是确保故障诊断模型在实际应用中能够被广泛接受的关键因素**。通过对迁移过程的细致分析，本研究不仅提升了模型的可解释性，也增强了模型在不同设备和工况下的适应性和可信度，为迁移诊断的实际应用提供了理论基础。

7.2 基于迁移过程的可解释性分析

在迁移学习中，模型的可解释性分析对于理解源域与目标域之间的知识迁移过程至关重要。为了增强模型的透明性，本研究通过**最大均值差异 (MMD)** 和**类间相关度量 (CORAL)** 来分析迁移过程，并引入**分布距离度量 (DDM)** 来综合评估源域与目标域的边际分布差异，从而进一步提高迁移学习的效果。

- **最大均值差异 (Maximum Mean Discrepancy, MMD)** 是一种衡量两个概率分布之间差异的方法，其核心思想是通过计算源域和目标域样本的均值差异来评估它们的分布差异，计算公式如 4.5 所示。具体来说，MMD 通过对源域和目标域样本映射到一个再生核希尔伯特空间 (RKHS) 中，然后计算这些样本均值之间的差异，从而得到源域与目标域的分布差异。在迁移学习中，MMD 可以帮助我们衡量迁移过程中的源域和目标域的分布对齐程度。较小的 MMD 值意味着源域和目标域的特征分布较为接近，模型的迁移效果较好。因此，通过对源域和目标域之间的 MMD 进行优化，可以使得模型能

够在目标域上更好地泛化。

- **类间相关度量 (Correlation Alignment, CORAL)** 是一种度量源域和目标域特征分布之间相似度的方法，其基本思想是将源域和目标域的协方差矩阵进行对齐，以减少源域和目标域之间的特征分布差异。通过计算源域和目标域样本的协方差矩阵，CORAL 可以量化源域和目标域在特征空间中的对齐程度，从而减少数据分布的偏差。在迁移学习过程中，CORAL 通过对源域和目标域之间的协方差矩阵进行优化，使得两者的特征分布尽可能相似，从而提高迁移学习的性能。CORAL 的优势在于它不仅能够减小均值的差异，还能有效减少源域和目标域特征空间中的相关性差异，使得目标域样本能够更好地适应源域学到的特征。
- **分布距离度量 (DDM)**：为评估源域和目标域之间的分布差异，本文引入了分布距离度量 DDM，结合了 MMD 和 CORAL 这两种度量方法（如公式 6.5 所示），旨在更全面地评估源域和目标域之间的边缘分布差异。具体而言，DDM 通过将 MMD 和 CORAL 的结果相加，综合考虑了均值差异和协方差差异，从而提供一个更为精确的分布对齐度量。DDM 的引入使得迁移学习过程中，源域和目标域的知识迁移能够更加顺畅，减少了由于分布差异带来的性能下降。通过优化 DDM，模型不仅能在源域上获得良好的表现，还能更好地适应目标域的特征分布，从而提升整体的诊断精度和鲁棒性。

MMD、CORAL 和 DDM 为迁移学习的可解释性分析提供了强有力的支持。MMD 主要通过衡量源域和目标域的均值差异来分析迁移过程中的分布对齐，而 CORAL 则通过对协方差矩阵的优化减少了源域和目标域之间的相关性差异。DDM 结合了这两种方法，进一步精细化了分布对齐的度量，为迁移学习提供了更加准确和有效的优化手段。通过这些度量方法，我们能够深入理解迁移学习过程中的决策逻辑，增强模型的可解释性，进而提高故障诊断模型的透明度和可信度。

图 7.1 展示了迁移模型的可视化结果，图中的圆形表示源域数据，星形表示目标域数据。通过 t-SNE 降维后的结果显示，源域数据在特征空间中具有良好的类间分离性，且每一类内部的样本较为紧凑，表现出较好的类别区分度。四种故障类型在二维空间中有明显的分布差异，这表明源域数据的故障类型得到了较好的识别。然而，目标域的数据分布与源域相比略有不同，尽管目标域中的类间差异依然明显，但部分类别的样本之间存在较大的重叠，说明目标域数据的特征分布和源域数据存在一定差异。尽管如此，目标域数据在类别上的分离性仍然较好，且与源域数据之间保持了一定的结构一致性，这表明通过迁移学习，目标域的数据在经过模型处理后，能够保留源域中学到的一些重要特征。通过这种可视化分析，诊断人员可以更加清晰地观察到模型迁移的结果，从而更好地理解迁移过程中的特征对齐和决策逻辑。

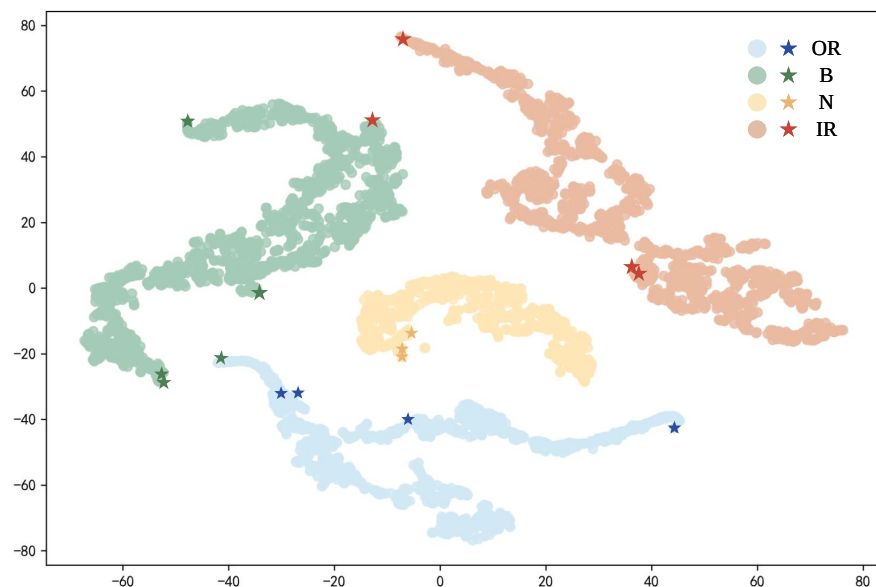


图 7.1 t-SNE 可视化结果

7.3 本章小结

本章主要探讨了迁移学习在轴承故障诊断中的应用，重点分析了迁移学习模型中**基于最大均值差异 (MMD)、类间相关度量 (CORAL) 和分布距离度量 (DDM) 的方法**来优化源域和目标域之间的特征对齐策略的可解释性。在迁移学习过程中，源域和目标域的特征分布差异会影响模型的性能。因此，本章通过这些度量方法深入分析了**源域与目标域之间的分布对齐情况**，并为迁移过程提供了**可解释性支持**。此外，t-SNE 可视化结果表明，源域和目标域数据在特征空间中的分布差异，尽管目标域存在一定的重叠，但**整体上保持了良好的类间分离性**，证明了**迁移学习方法在跨域故障诊断中的有效性**。通过迁移学习，目标域能够较好地保留源域学到的**关键特征**，且分类性能有所提升，为后续跨工况、跨设备的轴承故障诊断提供了理论依据和技术支持。

8 模型评价与展望

8.1 模型优点

1. **所选特征全面且科学**：本研究从时域、频域、时频域和包络谱四个维度提取了 39 个特征，并通过皮尔逊和斯皮尔曼相关系数、PCA 和随机森林等多种方法对重要特征进行筛选。筛选出的 19 个特征在不同方法中均表现出较高的重要性，确保了特征提取的全面性、严谨性和可行性。
2. **多模型高效故障诊断**：本研究采用了 SVM、随机森林和 MLP 三种经典机器学习模型对筛选后的特征进行故障诊断。通过多项性能评价指标的验证，所有模型在不同的评价标准上均表现出了较高的准确性，证明了该模型在轴承故障诊断中的适用性和有效性。
3. **深度学习增强泛化能力**：针对时频谱特征，本文使用小波变换提取的特征并采用深度学习中的 CNN 和 VGG 模型进行故障诊断。实验结果表明，深度学习模型在特征学习和故障分类中表现出较强的泛化能力，证明了其在复杂故障模式识别中的优势。
4. **使用新颖的 CDA 解决域迁移**：使用一种新型的 CDA 机制，来有效对齐两个域的概率分布并使用结合了 MDA 和 CDA 机制的 IJDA 通过改进的 MMD 和 CORAL 来解决域迁移问题，并从均值和协方差减少分布差异。随后，通过零活的 I-Softmax，进一步提升了故障特征可比性，从而在轴承诊断任务中取得良好结果。

8.2 模型缺点

1. **迁移学习中的模型选择需进一步优化**：在问题三的迁移学习中，本文采用了单一的深度学习模型，未来应尝试多种深度学习模型并进行调参，以筛选出最优模型。这样可以更全面地评估迁移学习在不同工况和设备中的表现，进一步提升模型的鲁棒性和适应性。
2. **可解释性仍有待提升**：在问题四的研究中，虽然通过 MMD、CORAL 等方法对迁移过程进行了一定的可解释性分析，但深度学习模型的“黑箱”问题依然存在。未来的研究应进一步加强模型的可解释性，尤其是在迁移学习的过程中，帮助诊断人员更好地理解模型决策依据，增强对模型输出结果的信任。

8.3 未来展望

本文采用的 VGG 和 CNN 模型在图像分类和故障诊断等领域已经有了广泛应用，尤其在机械设备故障检测和医学影像分析等领域取得了显著成果。未来，深度学习模型与迁移学习、自适应学习等方法相结合，能够更好地应对跨设备、跨工况的故障诊断任务，提高模型的泛化能力和迁移能力。进一步加强模型的可解释性，并结合领域知识，可能为机械设备的故障检测提供更加精准和透明的解决方案。此外，随着数据量的不断增加和计算能力的提升，未来的模型有望更加高效、精准地识别复杂的故障模式，并拓展到更多领域。

参考文献

- [1] 宁滨, 董海荣, 郑伟, 等, 高速铁路运行控制与动态调度一体化的现状与展望, 自动化学报, 45(12):2208-2217, 2019.
- [2] 黄玲, 轨道交通行业发展综述, 中国基础科学, 6, 2018.
- [3] 李军宁, 罗文广, 陈武阁, 面向振动信号的滚动轴承故障诊断算法综述, 西安工业大学学报, 42(2):105-122, 2022.
- [4] Hoang D T, Kang H J, A survey on deep learning based bearing fault diagnosis, Neurocomputing, 335:327-335, 2019.
- [5] 王敏, 邓艾东, 马天霆, 等, 基于双注意力机制的 MSCN-BiGRU 的滚动轴承故障诊断方法, 振动与冲击, 43(06):84-92+103, 2024.
- [6] 陈鹏, 基于振动信号的滚动轴承故障诊断方法综述, 轴承(06):1-6, 2022.
- [7] 刘鑫鹏, 栾悉道, 谢毓湘, 等, 迁移学习研究和算法综述, 长沙大学学报, 32(05):28-31+36, 2018.
- [8] 金雨纯, 甄元元, 刘平, 基于迁移改进 MobileNetV3 的玉米叶片病害识别研究, 中国农学通报, 41(23):145-154, 2025.
- [9] 李舜酩, 侯钰哲, 李香莲, 滚动轴承振动故障时频域分析方法综述, 重庆理工大学学报(自然科学), 35(10):85-93, 2021.
- [10] 韩小孩, 张耀辉, 孙福军, 等, 基于主成分分析的指标权重确定方法, 四川兵工学报, 33(10):124-126, 2012.
- [11] Van der Maaten L, Hinton G, Visualizing data using t-SNE, Journal of Machine Learning Research, 9(11):2579-2605, 2008.
- [12] Qian Q, Qin Y, Luo J, et al., Deep discriminative transfer learning network for cross-machine fault diagnosis, Mechanical systems and signal processing, 186:109884, 2023.

附录 A Python 源程序

A.1 第 1 问程序

Q1.py

```
import pandas as pd
import numpy as np
import scipy.io as sio
import scipy.signal as signal
from pathlib import Path
import chardet

summary_path =
    r"E:\WYN\Python_project\cwru_process\source_domain_summary.csv"

with open(summary_path, "rb") as f:
    raw_bytes = f.read(10000)
    enc_detect = chardet.detect(raw_bytes)
    encoding_used = enc_detect["encoding"] or "utf-8"

df_summary = pd.read_csv(summary_path, encoding=encoding_used)
print("表格基本信息：")
print(df_summary.head())

mask = (df_summary["轴承类型"] == "DE") & (df_summary["采样频率Hz"] ==
    48000)
is_or = df_summary["故障类型"].str.upper() == "OR"
mask = mask & (~is_or | (is_or & (df_summary["点位"] == 6)))
df_filtered = df_summary[mask].copy().reset_index(drop=True)
print(f"筛选后文件数：{len(df_filtered)}")

def load_signal_from_mat(mat_path, ID, bearing_type):
    key = f"{ID}_{bearing_type}_time"
    mat = sio.loadmat(mat_path)
    if key not in mat:
        raise KeyError(f"{mat_path} 中未找到键 {key}")
    sig = mat[key].squeeze()
    return sig

def resample_to_32k(sig, fs_in=48000, fs_out=32000):
```

```

gcd = np.gcd(fs_in, fs_out)
up = fs_out // gcd
down = fs_in // gcd
sig_ds = signal.resample_poly(sig, up, down)
return sig_ds

def segment_signal(sig, win_size, overlap=0.5):
    step = int(win_size * (1 - overlap))
    segments = []
    for start in range(0, len(sig) - win_size + 1, step):
        seg = sig[start:start+win_size]
        segments.append(seg)
    return np.array(segments)

def normalize_min_max(sig):
    sig_min = np.min(sig)
    sig_max = np.max(sig)
    sig = (sig - sig_min) / (sig_max - sig_min + 1e-12)
    return sig

fs_out = 32000
all_segments_A, meta_A = [], []
for _, row in df_filtered.iterrows():
    mat_path = row["数据文件"]
    ID = row["ID"]
    bearing_type = row["轴承类型"]
    bearing=row['故障类型']

    sig = load_signal_from_mat(mat_path, ID, bearing_type)
    sig_ds = resample_to_32k(sig, fs_in=48000, fs_out=fs_out)
    if bearing == 'N':
        segs_A = segment_signal(sig_ds, win_size=4096, overlap=0.7)
    else:
        segs_A = segment_signal(sig_ds, win_size=4096, overlap=0.5)
    # segs_A = np.array([normalize_min_max(seg) for seg in segs_A])
    for seg in segs_A:
        all_segments_A.append(seg)
    meta_A.append({
        "ID": ID,

```

```

        "bearing_type": bearing_type,
        "fault_type": row["故障类型"],
        "fs": fs_out,
        "stage": "A",
        "origin_file": mat_path
    })

print("Stage A 窗口数: ", len(all_segments_A))
np.save("source_domain_segments.npy", np.array(all_segments_A))
pd.DataFrame(meta_A).to_csv("source_domain_meta.csv", index=False,
    encoding="utf-8-sig")

def extract_time_domain_features(sig):
    mean = np.mean(sig)
    std = np.std(sig)
    var = np.var(sig)
    rms = np.sqrt(np.mean(sig**2)) # RMS
    peak = np.max(sig) # 峰值
    p2p = np.ptp(sig) # P2P, 峰对峰值
    mean_abs = np.mean(np.abs(sig)) # 均值绝对值
    sqr_mean = np.mean(sig**2) # 均方值
    shape_factor = rms / (mean_abs + 1e-12) # 形状因子
    crest_factor = peak / (rms + 1e-12) # 峰值因子
    impulse_factor = peak / (mean_abs + 1e-12) # 冲击因子
    margin_factor = peak / (sqr_mean + 1e-12) # 边缘因子
    clearance_factor = peak / (sqr_mean + 1e-12) # 清除因子
    kurt = kurtosis(sig) # 峰度
    skewness = skew(sig) # 偏度

    return [mean, std, var, rms, peak, p2p, mean_abs, shape_factor,
        crest_factor, impulse_factor, margin_factor, clearance_factor, kurt,
        skewness ]

def extract_frequency_domain_features(sig, fs):
    N = len(sig)
    freqs = np.fft.fftfreq(N, 1/fs)
    fft_vals = np.abs(fft(sig))

    spectrum_mean = np.mean(fft_vals) # 频谱均值
    spectrum_variance = np.var(fft_vals) # 频谱方差

```

```

spectrum_centroid = np.sum(freqs * fft_vals) / np.sum(fft_vals) # 频谱重心
spectrum_bandwidth = np.sqrt(np.sum(((freqs - np.mean(freqs))**2) *
    fft_vals) / np.sum(fft_vals)) # 频谱带宽
spectrum_skewness = skew(fft_vals) # 频谱偏度
spectrum_kurtosis = kurtosis(fft_vals) # 频谱峰度
spectrum_entropy = -np.sum((fft_vals / np.sum(fft_vals)) * np.log(fft_vals
    / np.sum(fft_vals) + 1e-12)) # 频谱熵

return [spectrum_mean, spectrum_variance, spectrum_centroid,
    spectrum_bandwidth, spectrum_skewness, spectrum_kurtosis,
    spectrum_entropy ]

def extract_time_frequency_domain_features(sig, fs):
    f, t, Zxx = signal.stft(sig, fs, nperseg=1024)

    st_energy = np.sum(np.abs(Zxx)**2, axis=0)
    st_freq = np.sum(np.abs(Zxx)**2 * f[:, None], axis=0)
    power = np.abs(Zxx)**2
    power = power / (np.sum(power) + 1e-12)
    tf_entropy = -np.sum(power * np.log(power + 1e-12))
    f_norm = f / np.max(f)
    tf_mean_freq = np.sum(np.mean(power, axis=1) * f_norm)
    coeffs = pywt.wavedec(sig, 'db4', level=4)
    energy = np.array([np.sum(c**2) for c in coeffs])
    energy_ratio = energy / (np.sum(energy) + 1e-12)
    wavelet_energy_feats = [energy_ratio[i] for i in range(len(energy_ratio))]

    stft_feats = [np.mean(st_energy), np.mean(st_freq), tf_entropy,
        tf_mean_freq]

    return stft_feats + wavelet_energy_feats

def extract_envelope_spectrum_features(sig, fs):
    analytic_signal = signal.hilbert(sig)
    envelope = np.abs(analytic_signal)
    N = len(envelope)
    freqs = np.fft.fftfreq(N, 1/fs)
    fft_vals = np.abs(np.fft.fft(envelope))
    envelope_max = np.max(envelope)

```

```

envelope_min = np.min(envelope)
envelope_mean = np.mean(envelope)
envelope_std = np.std(envelope)
envelope_kurt = kurtosis(envelope)
envelope_skew = skew(envelope)
envelope_spectrum = np.mean(fft_vals)
envelope_frequency = freqs[np.argmax(fft_vals)]
envelope_energy = np.sum(fft_vals**2)
power_spectrum = fft_vals / np.sum(fft_vals)
envelope_spectrum_entropy = -np.sum(power_spectrum * np.log(power_spectrum
    + 1e-12))

return [envelope_max, envelope_min, envelope_mean, envelope_std,
        envelope_kurt, envelope_skew, envelope_spectrum, envelope_frequency,
        envelope_energy, envelope_spectrum_entropy]

segments = np.load('source_domain_segments.npy')
meta = pd.read_csv('source_domain_meta.csv', encoding='gb18030')

all_features = []

for index, row in meta.iterrows():
    seg = segments[index]
    time_domain_features = extract_time_domain_features(seg)
    freq_domain_features = extract_frequency_domain_features(seg, fs_out)
    time_freq_features = extract_time_frequency_domain_features(seg, fs_out)
    envelope_spectrum_features = extract_envelope_spectrum_features(seg, fs_out)
    all_features.append(
        list(time_domain_features) + list(freq_domain_features) +
        list(time_freq_features) + list(envelope_spectrum_features)
    )
    meta.at[index, [
        "mean", "std", "var", "rms", "peak", "p2p", "mean_abs", "shape_factor",
        "crest_factor",
        "impulse_factor", "margin_factor", "clearance_factor", "kurtosis",
        "skewness",
        "spectrum_mean", "spectrum_variance", "spectrum_centroid",
        "spectrum_bandwidth",
        "spectrum_skewness", "spectrum_kurtosis", "spectrum_entropy",

```

```

"st_energy", "st_freq", "tf_entropy", "tf_mean_freq",
"wavelet_energy_0", "wavelet_energy_1", "wavelet_energy_2",
    "wavelet_energy_3", "wavelet_energy_4",
"envelope_max", "envelope_min", "envelope_mean", "envelope_std",
    "envelope_kurt", "envelope_skew",
"envelope_spectrum", "envelope_frequency", "envelope_energy",
    "envelope_spectrum_entropy"
]] = all_features[-1]

meta.to_csv('source_domain_meta.csv', index=False, encoding='utf-8-sig')

def compute_short_time_energy(sig, fs=32000, nperseg=256):
    _, _, Zxx = stft(sig, fs=fs, nperseg=nperseg)
    st_energy = np.sum(np.abs(Zxx) ** 2, axis=0)
    return st_energy

def extract_all_features(signal, fs=32000, nperseg=256):
    features = {}
    short_time_energy = compute_short_time_energy(signal, fs, nperseg)
    features["short_time_energy"] = short_time_energy
    return features

all_features = []
all_short_time_energies = []
segments = np.load('source_domain_segments.npy')
for seg in segments:
    features = extract_all_features(seg)
    short_time_energy = features["short_time_energy"]
    all_short_time_energies.append(short_time_energy)
short_time_energy_matrix = np.array(all_short_time_energies)
np.save("short_time_energies.npy", short_time_energy_matrix)
print(f"短时能量矩阵维度: {short_time_energy_matrix.shape}")
energy_column_names = [f"short_time_energy_{i+1}" for i in
    range(short_time_energy_matrix.shape[1])]

new_column_names = energy_column_names
print(new_column_names)
df_combined = pd.DataFrame(short_time_energy_matrix,
    columns=new_column_names)

```

```

df_meta = pd.read_csv("source_domain_meta.csv", encoding="gb18030")
df_meta_combined = pd.concat([df_meta, df_combined], axis=1)
df_meta_combined.to_csv("source_domain_meta.csv", index=False,
                        encoding="utf-8-sig")
print("新文件已保存为 source_domain_meta_stageA_combined.csv")

```

A.2 第2问程序

Q2.py

```

import pandas as pd
import numpy as np
from sklearn.model_selection import StratifiedShuffleSplit
from joblib import dump

def make_datasets(data_file_csv, segments_file, split_rate=[0.4, 0.2, 0.4]):
    origin_data = pd.read_csv(data_file_csv, encoding="gb18030")
    segments = np.load(segments_file)
    labels = origin_data['fault_type'].values
    sss = StratifiedShuffleSplit(n_splits=1, test_size=(split_rate[1] +
                                                         split_rate[2]), random_state=123)

    for train_index, remaining_index in sss.split(segments, labels):
        train_segments, remaining_segments = segments[train_index],
            segments[remaining_index]
        train_labels = labels[train_index]
        remaining_labels = labels[remaining_index]

    sss_remaining = StratifiedShuffleSplit(n_splits=1, test_size=split_rate[2]
                                           / (split_rate[1] + split_rate[2]), random_state=123)

    for val_index, test_index in sss_remaining.split(remaining_segments,
                                                       remaining_labels):
        val_segments, test_segments = remaining_segments[val_index],
            remaining_segments[test_index]
        val_labels = remaining_labels[val_index]
        test_labels = remaining_labels[test_index]

    train_set = pd.DataFrame(train_segments)
    train_set['label'] = train_labels

```

```

val_set = pd.DataFrame(val_segments)
val_set['label'] = val_labels

test_set = pd.DataFrame(test_segments)
test_set['label'] = test_labels

dump(train_set, 'train_set.joblib')
dump(val_set, 'val_set.joblib')
dump(test_set, 'test_set.joblib')

print("数据集保存成功: train_set.joblib, val_set.joblib, test_set.joblib")
return

make_datasets('source_domain_meta.csv', 'source_domain_segments.npy')

import matplotlib.pyplot as plt
import numpy as np
import matplotlib
matplotlib.rc("font", family='Microsoft YaHei')
import pywt
data_set = [train_set, val_set, test_set]
train_path = 'e2_1/FaultClassificationData/train/'
val_path = 'e2_1/FaultClassificationData/val/'
test_path = 'e2_1/FaultClassificationData/test/'
path_list = [train_path, val_path, test_path]

def makeTimeFrequencyImage(data, img_path, img_size):
    sampling_length = len(data)
    sampling_period = 1.0 / 32000
    totalscal = 256
    wavename = 'cmor100-1'
    fc = pywt.central_frequency(wavename)
    cparam = 2 * fc * totalscal
    scales = cparam / np.arange(totalscal, 0, -1)
    coefficients, frequencies = pywt.cwt(data, scales, wavename,
        sampling_period)
    amp = abs(coefficients)
    t = np.linspace(0, sampling_period, sampling_length, endpoint=False)

```

```

plt.contourf(t, frequencies, amp, cmap='jet')
plt.axis('off')
plt.subplots_adjust(top=1, bottom=0, right=1, left=0, hspace=1, wspace=0)
plt.margins(0, 0)
plt.gcf().set_size_inches(img_size/100, img_size/100)
plt.savefig(img_path, dpi=100)
plt.clf()
plt.close()

def GenerateImageDataset(path_list, data_set):
    for index in range(len(data_set)):
        dataset = data_set[index]
        path = path_list[index]
        count = 1
        for _, row in dataset.iterrows():
            time_series = row[:-1].values
            label = row['label']
            img_path = path + str(label) + '_' + str(count)
            os.makedirs(os.path.dirname(img_path), exist_ok=True)
            makeTimeFrequencyImage(time_series, img_path, 512)
            count += 1
        GenerateImageDataset(path_list, data_set)

import os
from PIL import Image
train_path = 'e2_1/FaultClassificationData/train/'
val_path = 'e2_1/FaultClassificationData/val/'
test_path = 'e2_1/FaultClassificationData/test/'
path_list = [train_path, val_path, test_path]

for path in path_list:
    all_images = os.listdir(path)
    for image in all_images:
        image_path = os.path.join(path, image)
        img = Image.open(image_path)
        print(img.format, img.size, img.mode)

import os
import PIL

```

```

import torch
import pandas
import torchvision
from joblib import dump, load

class MyDataset(torch.utils.data.Dataset):
    def __init__(self, data_path:str, train=True, transform=None):
        self.data_path = data_path
        self.train_flag = train
        if transform is None:
            self.transform = torchvision.transforms.Compose(
                [
                    torchvision.transforms.Resize(size = (224,224)),
                    torchvision.transforms.ToTensor(),
                    torchvision.transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
                ]
            )
        else:
            self.transform = transform
        self.path_list = os.listdir(data_path)

    def __getitem__(self, idx: int):
        img_path = self.path_list[idx]
        if img_path.split('_')[0] == 'B':
            label = 0
        elif img_path.split('_')[0] == 'IR':
            label = 1
        elif img_path.split('_')[0] == 'N':
            label = 2
        elif img_path.split('_')[0] == 'OR':
            label = 3
        label = torch.tensor(label, dtype=torch.int64)
        img_path = os.path.join(self.data_path, img_path)
        img = PIL.Image.open(img_path)
        img = self.transform(img)
        return img, label

    def __len__(self) -> int:
        return len(self.path_list)

```

```

train_path = 'e2_1/FaultClassificationData/train/'
val_path = 'e2_1/FaultClassificationData/val/'
test_path = 'e2_1/FaultClassificationData/test/'

train_dataset = MyDataset(train_path)
val_dataset = MyDataset(val_path, train=False)
test_dataset = MyDataset(test_path, train=False)

dump(train_dataset, 'train_dataset')
dump(val_dataset, 'val_dataset')
dump(test_dataset, 'test_dataset')

import os
import PIL
import torch
import pandas
import torchvision
import matplotlib.pyplot as plt
import matplotlib.pyplot as plt
from joblib import dump, load
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

train_dataset = load('train_dataset')
val_dataset = load('val_dataset')
test_dataset = load('test_dataset')

batch_size = 32
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=32,
                                             shuffle=True, pin_memory=True, num_workers=0)
validate_loader = torch.utils.data.DataLoader(val_dataset, batch_size=32,
                                              shuffle=True, pin_memory=True, num_workers=0)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=32,
                                           shuffle=False, pin_memory=True, num_workers=0)

import torch
from joblib import dump, load
import torch.utils.data as Data
import numpy as np
import pandas as pd

```

```

import torch
import torch.nn as nn

def vgg_block(num_convs, in_channels, out_channels):
    layers = []
    for _ in range(num_convs):
        layers.append(torch.nn.Conv2d(in_channels, out_channels, kernel_size=3,
                                       padding=1))
    layers.append(torch.nn.ReLU())
    in_channels = out_channels
    layers.append(torch.nn.MaxPool2d(kernel_size=2, stride=2, ceil_mode=False))
    return torch.nn.Sequential(*layers)

def VGGModel(conv_arch, num_classes):
    conv_blks = []
    in_channels = 3
    for (num_convs, out_channels) in conv_arch:
        conv_blks.append(vgg_block(num_convs, in_channels, out_channels))
        in_channels = out_channels

    return torch.nn.Sequential(
        *conv_blks,
        torch.nn.Flatten(),
        torch.nn.Linear(in_channels * 7 * 7, 4096),
        torch.nn.ReLU(),
        torch.nn.Dropout(0.5),
        torch.nn.Linear(4096, 4096),
        torch.nn.ReLU(),
        torch.nn.Dropout(0.5),
        torch.nn.Linear(4096, num_classes),
        torch.nn.Softmax(dim=1))

def init_normal(m):
    if type(m) == torch.nn.Linear:
        torch.nn.init.xavier_uniform_(m.weight)
        torch.nn.init.zeros_(m.bias)
    if type(m) == torch.nn.Conv2d:
        torch.nn.init.xavier_uniform_(m.weight)
        torch.nn.init.zeros_(m.bias)

```

```

conv_arch = ((2, 64), (2, 128), (3, 256), (3, 512), (3, 512))
num_classes = 4
model = VGGModel(conv_arch, num_classes)
model.apply(init_normal)
loss_function = nn.CrossEntropyLoss(reduction='sum')
learn_rate = 1e-5
optimizer = torch.optim.Adam(model.parameters(), lr=learn_rate)

def count_parameters(model):
    params = [p.numel() for p in model.parameters() if p.requires_grad]
    for item in params:
        print(f'{item:>6}')
    print(f'_____\n{sum(params):>6}')

count_parameters(model)

from joblib import dump, load
import matplotlib.pyplot as plt

def train_model(model, epoch, train_loader, validate_loader, cost,
                optimezer, train_size, val_size):
    model = model.to(device)
    now_loss = 1e9
    train_ls = []
    train_acc = []
    val_ls = []
    val_acc = []

    for i in range(epoch):
        loss_epoch = 0.
        correct_epoch = 0
        for j, (data, label) in enumerate(train_loader):
            model.train()
            data, label = data.to(device), label.to(device)
            pre = model(data)
            correct_epoch += torch.sum(pre.argmax(dim = 1).view(-1) ==
                                      label.view(-1)).item()
            loss = cost(pre, label)

```

```

loss_epoch += loss.item()
optimizer.zero_grad()
loss.backward()
optimizer.step()

train_ls.append(loss_epoch/train_size)
train_acc.append(correct_epoch/train_size)
print(f'batch_loss:{loss_epoch/train_size},
      batch_acc:{correct_epoch/train_size}%')

with torch.no_grad():
    loss_validate = 0.
    correct_validate = 0
    for j, (data, label) in enumerate(validate_loader):
        data, label = data.to(device), label.to(device)
        pre = model(data)
        correct_validate += torch.sum(pre.argmax(dim = 1).view(-1) ==
                                     label.view(-1)).item()
        loss = cost(pre, label)
        loss_validate += loss.item()

    print(f'validate_Loss:{loss_validate/val_size},
          validate_Acc:{correct_validate/val_size}%')

val_ls.append(loss_validate/val_size)
val_acc.append(correct_validate/val_size)
if now_loss > loss_validate:
    now_loss = loss_validate
    print(f"保存模型参数--epoch:{i}")
    torch.save(model.state_dict(), 'model.params')

plt.figure(figsize=(10, 6))
ax = plt.gca()
ax.set_facecolor('white')
plt.subplot(2, 1, 1)
plt.plot(range(epoch), train_ls, color='b', label='Train Loss',
         linestyle='-', linewidth=2)
plt.plot(range(epoch), val_ls, color='r', label='Validation Loss',
         linestyle='--', linewidth=2)

```

```

plt.title('Loss Curve')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend(loc='upper right')
plt.grid(True)
plt.savefig('figures/e2/loss2.pdf', format='pdf')

plt.subplot(2, 1, 2)
plt.plot(range(epoch), train_acc, color='g', label='Train Accuracy',
         linestyle='-', linewidth=2)
plt.plot(range(epoch), val_acc, color='orange', label='Validation
         Accuracy', linestyle='--', linewidth=2)
plt.title('Accuracy Curve')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend(loc='lower right')
plt.grid(True)
plt.tight_layout()
plt.savefig('figures/e2/acc2.pdf', format='pdf')
plt.show()

train_size = len(train_loader) * batch_size
val_size = len(validate_loader) * batch_size
epoch = 100
train_model(model, epoch, train_loader, validate_loader, loss_function,
            optimizer, train_size, val_size)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model.load_state_dict(torch.load('model.params'))
class_labels = []
predicted_labels = []

loss_function = nn.CrossEntropyLoss(reduction='sum')

with torch.no_grad():
    for j, (data, label) in enumerate(test_loader):
        model.eval()
        data, label = data.to(device), label.to(device)
        pre = model(data)

```

```

predicted = torch.argmax(pre, dim=1)
class_labels.extend(label.tolist())
predicted_labels.extend(predicted.tolist())

confusion_mat = confusion_matrix(class_labels, predicted_labels)

from sklearn.metrics import classification_report
report = classification_report(class_labels, predicted_labels, digits=4)
print(report)

df = pd.read_csv('source_domain_meta.csv', encoding='ISO-8859-1')

feature_columns =
    df.columns[df.columns.get_loc('mean'):df.columns.get_loc('short_time_energy_1')]
print(feature_columns)

X = df[feature_columns]
y = df['fault_type']

y_encoded = y.map({'B': 0, 'OR': 1, 'IR': 2, 'N': 3})

X_train, X_test, y_train, y_test = train_test_split(X, y_encoded,
    test_size=0.2, stratify=y_encoded, random_state=123)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

pca = PCA(n_components=10)
X_train_pca = X_train_scaled
X_test_pca = X_test_scaled

param_grid = {
    'C': [1,10,20,30],
    'gamma': [0.01,0.05,0.1,0.5],
    'kernel': ['rbf'],
    'probability': [True]
}

```

```

grid_search = GridSearchCV(SVC(), param_grid, cv=5, scoring='accuracy',
                           n_jobs=1, verbose=1)
grid_search.fit(X_train_pca, y_train)

print("Best parameters found: ", grid_search.best_params_)

svm_best = grid_search.best_estimator_

y_pred = svm_best.predict(X_test_pca)
y_pred_prob = svm_best.predict_proba(X_test_pca)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='macro')
recall = recall_score(y_test, y_pred, average='macro')
f1 = f1_score(y_test, y_pred, average='macro')
fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob[:, 1], pos_label=1)
roc_auc = auc(fpr, tpr)

metrics_results = {
    'Accuracy': accuracy,
    'Precision (Macro)': precision,
    'Recall (Macro)': recall,
    'F1 Score (Macro)': f1,
    'ROC AUC': roc_auc
}

print(metrics_results)

cm = confusion_matrix(y_test, y_pred)
print(cm)

df = pd.read_csv('source_domain_meta.csv', encoding='ISO-8859-1')

feature_columns =
    df.columns[df.columns.get_loc('mean'):df.columns.get_loc('short_time_energy_1')]

X = df[feature_columns]
y = df['fault_type']

```

```

y_encoded = y.map({'B': 0, 'OR': 1, 'IR': 2, 'N': 3})

X_train, X_test, y_train, y_test = train_test_split(X, y_encoded,
                                                    test_size=0.2, stratify=y_encoded, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

pca = PCA(n_components=20)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

param_grid = {
    'n_estimators': [100],
    'max_depth': [10, 20, None],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2],
    'bootstrap': [True, False]
}

grid_search = GridSearchCV(RandomForestClassifier(random_state=42),
                            param_grid, cv=5, scoring='accuracy', n_jobs=1, verbose=1)
grid_search.fit(X_train_pca, y_train)

print("Best parameters found: ", grid_search.best_params_)

rf_best = grid_search.best_estimator_

y_pred = rf_best.predict(X_test_pca)
y_pred_prob = rf_best.predict_proba(X_test_pca)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='macro')
recall = recall_score(y_test, y_pred, average='macro')
f1 = f1_score(y_test, y_pred, average='macro')
fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob[:, 1], pos_label=1)
roc_auc = auc(fpr, tpr)

```

```

metrics_results = {
    'Accuracy': accuracy,
    'Precision (Macro)': precision,
    'Recall (Macro)': recall,
    'F1 Score (Macro)': f1,
    'ROC AUC': roc_auc
}

print(metrics_results)

cm = confusion_matrix(y_test, y_pred)
print(cm)

df = pd.read_csv('source_domain_meta.csv', encoding='ISO-8859-1')

feature_columns =
    df.columns[df.columns.get_loc('mean'):df.columns.get_loc('short_time_energy_1')]

X = df[feature_columns]
y = df['fault_type']

y_encoded = y.map({'B': 0, 'OR': 1, 'IR': 2, 'N': 3})

X_train, X_test, y_train, y_test = train_test_split(X, y_encoded,
    test_size=0.2, stratify=y_encoded, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

pca = PCA(n_components=20)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

param_grid = {
    'hidden_layer_sizes': [(50,), (100,), (150,)],
    'activation': ['relu'],
    'solver': ['adam', 'sgd'],

```

```

        'learning_rate': ['constant', 'adaptive'],
        'max_iter': [1000, 2000]
    }

    grid_search = GridSearchCV(MLPClassifier(random_state=123), param_grid,
                               cv=5, scoring='accuracy', n_jobs=1, verbose=1)
    grid_search.fit(X_train_pca, y_train)

    print("Best parameters found: ", grid_search.best_params_)

    mlp_best = grid_search.best_estimator_

    y_pred = mlp_best.predict(X_test_pca)
    y_pred_prob = mlp_best.predict_proba(X_test_pca)

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average='macro')
    recall = recall_score(y_test, y_pred, average='macro')
    f1 = f1_score(y_test, y_pred, average='macro')
    fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob[:, 1], pos_label=1)
    roc_auc = auc(fpr, tpr)

    metrics_results = {
        'Accuracy': accuracy,
        'Precision (Macro)': precision,
        'Recall (Macro)': recall,
        'F1 Score (Macro)': f1,
        'ROC AUC': roc_auc
    }

    print(metrics_results)

    cm = confusion_matrix(y_test, y_pred)
    print(cm)

```

A.3 第3问程序

Q2.py

```
import numpy as np
```

```

import torchmetrics
from sklearn.preprocessing import StandardScaler
from pytorch_lightning.utilities.seed import seed_everything
from pytorch_lightning import LightningModule, Trainer, seed_everything
from sklearn.metrics import confusion_matrix
import torch
import torch.nn as nn
import torch.nn.functional as F
import argparse
from torch.utils import data as da
from torchmetrics import MeanMetric, Accuracy
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.manifold import TSNE

def parse_args():
    parser = argparse.ArgumentParser(description='Train')
    parser.add_argument('--cru_data', type=str,
                        default=r"E:\WYN\Python_project\cru_process\cru114.npy", help='')
    parser.add_argument('--cru_label', type=str,
                        default=r"E:\WYN\Python_project\cru_process\cru114_label.npy",
                        help='')
    parser.add_argument('--bjtu_data', type=str,
                        default=r"E:\WYN\Python_project\cru_process\target_domain_nosegments.npy",
                        help='')
    parser.add_argument('--bjtu_label', type=str,
                        default=r"E:\WYN\Python_project\cru_process\DDTLN\jnu600_label.npy",
                        help='')
    parser.add_argument('--batch_size', type=int, default=64, help='batchsize
                        of the training process')
    parser.add_argument('--epoch', type=int, default=200, help='max number of
                        epoch')
    parser.add_argument('--s_m', type=int, default=3, help='')
    parser.add_argument('--s_n', type=int, default=16, help='')
    parser.add_argument('--num_classes', type=int, default=4, help='')
    parser.add_argument('--lr', type=float, default=0.001, help='')
    parser.add_argument('--weight_decay', type=float, default=0.0001,
                        help='initialization list')
    args = parser.parse_args()

```

```

return args

class Dataset(da.Dataset):
def __init__(self, X, y):
self.Data = X
self.Label = y

def __getitem__(self, index):
txt = self.Data[index]
label = self.Label[index]
return txt, label

def __len__(self):
return len(self.Data)

def load_data():
source_data = np.load(args.cwru_data)
source_label = np.load(args.cwru_label).argmax(axis=-1)
target_data = np.load(args.bjtu_data)
target_label = np.load(args.bjtu_label).argmax(axis=-1)
source_data = StandardScaler().fit_transform(source_data.T).T
target_data = StandardScaler().fit_transform(target_data.T).T
source_data = np.expand_dims(source_data, axis=1)
target_data = np.expand_dims(target_data, axis=1)
Train_source = Dataset(source_data, source_label)
Train_target = Dataset(target_data, target_label)
return Train_source, Train_target

class MMD(nn.Module):
def __init__(self, m, n):
super(MMD, self).__init__()
self.m = m
self.n = n

def _mix_rbf_mmd2(self, X, Y, sigmas=(10,), wts=None, biased=True):
K_XX, K_XY, K_YY, d = self._mix_rbf_kernel(X, Y, sigmas, wts)

```

```

return self._mmd2(K_XX, K_XY, K_YY, const_diagonal=d, biased=biased)

def _mix_rbf_kernel(self, X, Y, sigmas, wts=None):
    if X.shape[0] == 0 or Y.shape[0] == 0:
        raise ValueError("Input tensors X and Y must not be empty")
    device = X.device
    Y = Y.to(device)
    if wts is None:
        wts = [1] * len(sigmas)
    XX = torch.matmul(X, X.t())
    XY = torch.matmul(X, Y.t())
    YY = torch.matmul(Y, Y.t())

    X_sqnorms = torch.diagonal(XX)
    Y_sqnorms = torch.diagonal(YY)

    r = lambda x: torch.unsqueeze(x, 0)
    c = lambda x: torch.unsqueeze(x, 1)

    K_XX, K_XY, K_YY = 0., 0., 0.
    for sigma, wt in zip(sigmas, wts):
        gamma = 1 / (2 * sigma ** 2)
        K_XX += wt * torch.exp(-gamma * (-2 * XX + c(X_sqnorms) + r(X_sqnorms)))
        K_XY += wt * torch.exp(-gamma * (-2 * XY + c(X_sqnorms) + r(Y_sqnorms)))
        K_YY += wt * torch.exp(-gamma * (-2 * YY + c(Y_sqnorms) + r(Y_sqnorms)))
    return K_XX, K_XY, K_YY, torch.sum(torch.tensor(wts))

def _mmd2(self, K_XX, K_XY, K_YY, const_diagonal=False, biased=False):
    if biased:
        mmd2 = torch.sum(K_XX) / (self.m * self.m) + torch.sum(K_YY) / (self.n *
            self.n) - 2 * torch.sum(K_XY) / (self.m * self.n)
    else:
        if const_diagonal is not False:
            trace_X = self.m * const_diagonal
            trace_Y = self.n * const_diagonal
        else:
            trace_X = torch.trace(K_XX)
            trace_Y = torch.trace(K_YY)

```

```

mmd2 = ((torch.sum(K_XX) - trace_X) / (self.m * (self.m - 1))
+ (torch.sum(K_YY) - trace_Y) / (self.n * (self.n - 1))
- 2 * torch.sum(K_XY) / (self.m * self.n))
return mmd2

def _if_list(self, list):
if len(list) == 0:
list = torch.tensor(list)
else:
list = torch.vstack(list)
return list

def _classification_division(self, data, label):
N = data.size()[0]
a, b, c, d = [], [], [], []
for i in range(N):
if label[i] == 0:
a.append(data[i])
elif label[i] == 1:
b.append(data[i])
elif label[i] == 2:
c.append(data[i])
elif label[i] == 3:
d.append(data[i])

return self._if_list(a), self._if_list(b), self._if_list(c),
self._if_list(d)

def MDA(self, source, target, bandwidths=[10]):
if source.shape[0] == 0 or target.shape[0] == 0:
raise ValueError("Source and target tensors must not be empty")
kernel_loss = self._mix_rbf_mmd2(source, target, sigmas=bandwidths) * 100.
eps = 1e-5
d = source.size()[1]
ns, nt = source.size()[0], target.size()[0]

tmp_s = torch.matmul(torch.ones(size=(1, ns)).cuda(), source)
cs = (torch.matmul(torch.t(source), source) - torch.matmul(torch.t(tmp_s),

```

```

        tmp_s) / (ns + eps)) / (ns - 1 + eps) * (ns / (self.m))

tmp_t = torch.matmul(torch.ones(size=(1, nt)).cuda(), target)
ct = (torch.matmul(torch.t(target), target) - torch.matmul(torch.t(tmp_t),
        tmp_t) / (nt + eps)) / (nt - 1 + eps) * (nt / (self.n))

loss = torch.norm((cs - ct))
loss = loss / (4 * d * d) * 10.

return loss + kernel_loss

def CDA(self, output1, source_label, output2, pseudo_label):
    s_0, s_1, s_2, s_3 = self._classification_division(output1, source_label)
    t_0, t_1, t_2, t_3 = self._classification_division(output2, pseudo_label)

    CDA_loss = 0.
    if t_0.size()[0] != 0:
        CDA_loss += self.MDA(s_0, t_0)
    if t_1.size()[0] != 0:
        CDA_loss += self.MDA(s_1, t_1)
    if t_2.size()[0] != 0:
        CDA_loss += self.MDA(s_2, t_2)
    if t_3.size()[0] != 0:
        CDA_loss += self.MDA(s_3, t_3)
    return CDA_loss / 4.

class I_Softmax(nn.Module):

    def __init__(self, m, n, source_output1, source_label):
        super().__init__()
        self.m = torch.tensor([m]).cuda()
        self.n = torch.tensor([n]).cuda()
        self.source_output1 = source_output1
        self.source_label = source_label
        self.la, self.lb, self.lc, self.ld = [], [], [], []
        self.a, self.b, self.c, self.d = [], [], [], []
        self.data_set = []
        self.label_set = []

```

```

def _combine(self):
    for i in range(self.source_label.size()[0]):
        if self.source_label[i] == 0:
            self.a.append(self.source_output1[i])
            self.la.append(self.source_label[i])
        elif self.source_label[i] == 1:
            self.b.append(self.source_output1[i])
            self.lb.append(self.source_label[i])
        elif self.source_label[i] == 2:
            self.c.append(self.source_output1[i])
            self.lc.append(self.source_label[i])
        elif self.source_label[i] == 3:
            self.d.append(self.source_output1[i])
            self.ld.append(self.source_label[i])

    a = self._class_angle(self.a, self.la)
    b = self._class_angle(self.b, self.lb)
    c = self._class_angle(self.c, self.lc)
    d = self._class_angle(self.d, self.ld)

    if len(a) != 0:
        self.data_set.append(a)
        self.label_set.append(torch.tensor(self.la).unsqueeze(1))
    if len(b) != 0:
        self.data_set.append(b)
        self.label_set.append(torch.tensor(self.lb).unsqueeze(1))
    if len(c) != 0:
        self.data_set.append(c)
        self.label_set.append(torch.tensor(self.lc).unsqueeze(1))
    if len(d) != 0:
        self.data_set.append(d)
        self.label_set.append(torch.tensor(self.ld).unsqueeze(1))
    data = torch.vstack(self.data_set)
    label = torch.vstack(self.label_set)
    return data.cuda(), label.squeeze().cuda()

def _class_angle(self, a, la):
    if len(la) == 0:

```

```

    return a
else:
    index = la[0]
    for i in range(len(a)):
        c = a[i]
        part1 = c[:index]
        part2 = c[index + 1:]
        if c[index] > 0:
            val = c[index] / (self.m + 1e-5) - self.n
        elif c[index] <= 0:
            val = c[index] * (self.m + 1e-5) - self.n
        if i == 0:
            new_tensor = torch.cat((part1, val, part2))
        else:
            tensor = torch.cat((part1, val, part2), dim=0)
            new_tensor = torch.vstack([new_tensor, tensor])

    return new_tensor

def forward(self):
    data, label = self._combine()
    loss = F.nll_loss(F.log_softmax(data, dim=-1), label)
    return data, label, loss

class TLN1(nn.Module):
    def __init__(self):
        super(TLN1, self).__init__()
        self.p1_1 = nn.Sequential(nn.Conv1d(1, 32, kernel_size=64, stride=16,
            padding=24),
            nn.BatchNorm1d(32),
            nn.ReLU())
        self.p1_2 = nn.MaxPool1d(2, 2)
        self.p2_1 = nn.Sequential(nn.Conv1d(32, 64, kernel_size=3, stride=1,
            padding=24),
            nn.BatchNorm1d(64),
            nn.ReLU())
        self.p2_2 = nn.MaxPool1d(2, 2)
        self.p3_1 = nn.Sequential(nn.Conv1d(64, 128, kernel_size=3, stride=1,

```

```

        padding=24),
nn.BatchNorm1d(128),
nn.ReLU())
self.p3_2 = nn.MaxPool1d(2, 2)
self.p4_1 = nn.Sequential(nn.Conv1d(128, 256, kernel_size=3, stride=1,
    padding=24),
nn.BatchNorm1d(256),
nn.ReLU())
self.p4_2 = nn.MaxPool1d(2, 2)
self.p5_1 = nn.Sequential(nn.Conv1d(256, 512, kernel_size=3, stride=1,
    padding=24),
nn.BatchNorm1d(512),
nn.ReLU())
self.p5_2 = nn.MaxPool1d(2, 2)
self.p6 = nn.AdaptiveAvgPool1d(1)
self.p7_1 = nn.Sequential(nn.Linear(512, 512),
nn.ReLU())
self.p7_2 = nn.Sequential(nn.Linear(512, 4))

def forward(self, x, y):
    x = self.p1_2(self.p1_1(x))
    x = self.p2_2(self.p2_1(x))
    x = self.p3_2(self.p3_1(x))
    x = self.p4_2(self.p4_1(x))
    x = self.p5_2(self.p5_1(x))
    x = self.p7_1(self.p6(x).squeeze())
    x = self.p7_2(x)
    y = self.p1_2(self.p1_1(y))
    y = self.p2_2(self.p2_1(y))
    y = self.p3_2(self.p3_1(y))
    y = self.p4_2(self.p4_1(y))
    y = self.p5_2(self.p5_1(y))
    y = self.p7_1(self.p6(y).squeeze())
    y = self.p7_2(y)
    return x, y

def predict(self, y):
    y = self.p1_2(self.p1_1(y))
    y = self.p2_2(self.p2_1(y))

```

```

y = self.p3_2(self.p3_1(y))
y = self.p4_2(self.p4_1(y))
y = self.p5_2(self.p5_1(y))
y = self.p7_1(self.p6(y).squeeze())
y = self.p7_2(y)
return y

def _weights_init(self):
for L in self.modules():
if isinstance(L, nn.Conv1d):
n = L.kernel_size[0] * L.out_channels
L.weight.data.normal_(mean=0, std=np.sqrt(2.0 / float(n)))
elif isinstance(L, nn.BatchNorm1d):
L.weight.data.fill_(1)
L.bias.data.fill_(0)
elif isinstance(L, nn.Linear):
L.weight.data.normal_(0, 0.01)
if L.bias is not None:
L.bias.data.fill_(1)

losses = []

def train(model, source_loader, target_loader, optimizer):
model.train()
iter_source = iter(source_loader)
iter_target = iter(target_loader)
num_iter = len(source_loader)
for i in range(0, num_iter):
source_data, source_label = next(iter_source)
target_data, _ = next(iter_target)
source_data, source_label = source_data.cuda(), source_label.cuda()
target_data = target_data.cuda()
optimizer.zero_grad()
output1, output2 = model(source_data.float(), target_data.float())
data, label, clc_loss_step = I_Softmax(args.s_m, args.s_n, output1,
source_label).forward()
pre_pseudo_label = torch.argmax(output2, dim=-1)
pseudo_data, pseudo_label, pseudo_loss_step = I_Softmax(args.s_m, args.s_n,
output2, pre_pseudo_label).forward()
CDA_loss = MMD(source_data.size()[0], target_data.size()[0]).CDA(output1,

```

```

        source_label, output2, pre_pseudo_label)
MDA_loss = MMD(source_data.size()[0], target_data.size()[0]).MDA(output1,
    output2)
loss_step = clc_loss_step + (MDA_loss + 0.1 * CDA_loss) + 0.1 *
    pseudo_loss_step
loss_step.backward()
optimizer.step()
metric_accuracy_1.update(output1.max(1)[1], source_label)
metric_mean_1.update(loss_step)
metric_mean_2.update(criterion(output1, source_label))
metric_mean_4.update(pseudo_loss_step)
metric_mean_5.update(clc_loss_step)
metric_mean_6.update(CDA_loss)
metric_mean_7.update(MDA_loss)
train_acc = metric_accuracy_1.compute()
train_all_loss = metric_mean_1.compute()
train_loss = metric_mean_2.compute()
target_cla_loss = metric_mean_4.compute()
source_cla_loss = metric_mean_5.compute()
cda_loss = metric_mean_6.compute()
mda_loss = metric_mean_7.compute()
metric_accuracy_1.reset()
metric_accuracy_2.reset()
metric_mean_1.reset()
metric_mean_2.reset()
metric_mean_3.reset()
metric_mean_4.reset()
metric_mean_5.reset()
metric_mean_6.reset()
metric_mean_7.reset()
return train_acc, train_all_loss, train_loss, target_cla_loss,
    source_cla_loss, cda_loss, mda_loss

def plot_confusion_matrix(y_true, y_pred, classes, normalize=False,
    title='Confusion matrix', cmap=plt.cm.Blues):
    cm = confusion_matrix(y_true, y_pred)
    if normalize:
        cm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
    print("Normalized confusion matrix")

```

```

else:
    print('Confusion matrix, without normalization')

    print(cm)

plt.imshow(cm, interpolation='nearest', cmap=cmap)
plt.title(title)
plt.colorbar()
tick_marks = np.arange(len(classes))
plt.xticks(tick_marks, classes, rotation=45)
plt.yticks(tick_marks, classes)

fmt = '.2f' if normalize else 'd'
thresh = cm.max() / 2.
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        plt.text(j, i, format(cm[i, j], fmt),
            horizontalalignment="center",
            color="white" if cm[i, j] > thresh else "black")

plt.tight_layout()
plt.ylabel('True label')
plt.xlabel('Predicted label')

def test(model, target_loader):
    global all_predicted_labels
    all_predicted_labels = []
    all_target_labels = []
    global all_features
    collected_features = []
    all_features = []
    model.eval()
    iter_target = iter(target_loader)
    num_iter = len(target_loader)
    for i in range(0, num_iter):
        target_data, target_label = next(iter_target)
        target_data, target_label = target_data.cuda(), target_label.cuda()

    output2 = model.predict(target_data.float())

```

```

feature_output = output2.detach()
collected_features.append(feature_output.cpu().numpy())

metric_accuracy_2.update(output2.max(1)[1], target_label)
metric_mean_3.update(criterion(output2, target_label))

y = F.softmax(output2, dim=1)
predicted_labels = y.argmax(dim=1)
all_predicted_labels.append(predicted_labels.cpu().numpy())
all_predicted_labels1 = np.concatenate(all_predicted_labels)

all_target_labels.append(target_label.cpu().numpy())
all_target_labels1 = np.concatenate(all_target_labels)

all_predicted_labels = np.concatenate(all_predicted_labels)
all_features = np.concatenate(collected_features, axis=0)
all_predicted_labels1 = all_predicted_labels.tolist()

test_acc = metric_accuracy_2.compute()
test_loss = metric_mean_3.compute()
metric_accuracy_2.reset()
metric_mean_3.reset()

return test_acc, test_loss, all_predicted_labels1

if __name__ == '__main__':
seed_everything(42)
args = parse_args()
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
metric_accuracy_1 = torchmetrics.Accuracy(task='multiclass',
    num_classes=4).to('cuda:0')
metric_accuracy_2 = torchmetrics.Accuracy(task='multiclass',
    num_classes=4).to('cuda:0')
metric_mean_1 = MeanMetric().cuda()
metric_mean_2 = MeanMetric().cuda()
metric_mean_3 = MeanMetric().cuda()
metric_mean_4 = MeanMetric().cuda()
metric_mean_5 = MeanMetric().cuda()

```

```

metric_mean_6 = MeanMetric().cuda()
metric_mean_7 = MeanMetric().cuda()
t_test_acc = 0.0
stop = 0
Train_source, Train_target = load_data()
source_loader = da.DataLoader(dataset=Train_source,
                              batch_size=args.batch_size, shuffle=True)
target_loader = da.DataLoader(dataset=Train_target,
                              batch_size=args.batch_size, shuffle=True)
target_loader_test = da.DataLoader(dataset=Train_target,
                                   batch_size=args.batch_size, shuffle=False)

model = TLN1().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=args.lr,
                              weight_decay=args.weight_decay)
train_loss1, test_loss1 = [], []
train_acc1, test_acc1 = [], []
cda_loss1 = []
mda_loss1 = []
target_cla_loss1, source_cla_loss1 = [], []

for epoch in range(0, args.nepoch):
    stop += 1
    train_acc, train_all_loss, train_loss, target_cla_loss, source_cla_loss,
        cda_loss, mda_loss = train(
        model, source_loader, target_loader, optimizer)
    test_acc, test_loss, all_pre_labels1 = test(model, target_loader_test)
    file_name = "predicted_labels_114.txt"
    with open(file_name, "a") as f:
        f.write(f"Epoch {epoch}:\n")
        f.write(str(all_pre_labels1) + "\n\n")
    train_loss1.append(train_loss.item())
    test_loss1.append(test_loss.item())
    train_acc1.append(train_acc.item())
    test_acc1.append(test_acc.item())
    cda_loss1.append(cda_loss.item())
    mda_loss1.append(mda_loss.item())
    target_cla_loss1.append(target_cla_loss.item())

```

```

source_cla_loss1.append(source_cla_loss.item())
print(
'Epoch{}, train_loss is {:.5f},test_loss is {:.5f}, train_accuracy is
 {:.5f},test_accuracy is {:.5f},train_all_loss is {:.5f},target_cla_loss
 is {:.5f},source_cla_loss is {:.5f},cda_loss is {:.5f},mda_loss is
 {:.5f}'.format(
epoch + 1, train_loss, test_loss, train_acc, test_acc, train_all_loss,
 target_cla_loss, source_cla_loss,
 cda_loss, mda_loss))

tsne = TSNE(n_components=2, verbose=1, perplexity=40, n_iter=300)
tsne_results = tsne.fit_transform(all_features)

plt.figure(figsize=(8, 8))
plt.scatter(tsne_results[:, 0], tsne_results[:, 1], c=all_target_labels1,
 cmap='viridis', alpha=0.8)
plt.title('t-SNE visualization of 4D features')
plt.xlabel('t-SNE feature 1')
plt.ylabel('t-SNE feature 2')
plt.show()

classes = ['0', '1', '2', '3']

plt.figure()
plot_confusion_matrix(all_target_labels1, all_predicted_labels1,
 classes=classes, title='Confusion matrix, without normalization')
plt.savefig('1confusion_matrix4812.png', dpi=300)

plt.figure()
plot_confusion_matrix(all_target_labels1, all_predicted_labels1,
 classes=classes, normalize=True, title='Normalized confusion matrix')
plt.savefig('2confusion_matrix4812.png', dpi=300)
plt.show()

plt.figure()
plt.plot(np.array(target_cla_loss1), label='target_cla_loss')
plt.plot(np.array(source_cla_loss1), label='source_cla_loss')
plt.title('loss curve')
plt.legend()

```

```

plt.savefig('loss curve11.png')
plt.show()

plt.figure()
plt.plot(np.array(cda_loss1), label='cda_loss')
plt.title('loss curve')
plt.legend()
plt.savefig('loss curve21.png')
plt.show()

plt.figure()
plt.plot(np.array(mda_loss1), label='mda_loss')
plt.title('loss curve')
plt.legend()
plt.savefig('loss curve31.png')
plt.show()

plt.figure()
plt.plot(np.array(train_loss1), label='train loss')
plt.title('loss curve')
plt.legend()
plt.savefig('loss curve41.png')
plt.show()

plt.figure()
plt.plot(np.array(train_acc1), label='train acc')
plt.plot(np.array(test_acc1), label='test acc')
plt.title('acc curve')
plt.legend()
plt.savefig('acc curve.png')
plt.show()

```

A.4 第4问程序

Q2.py